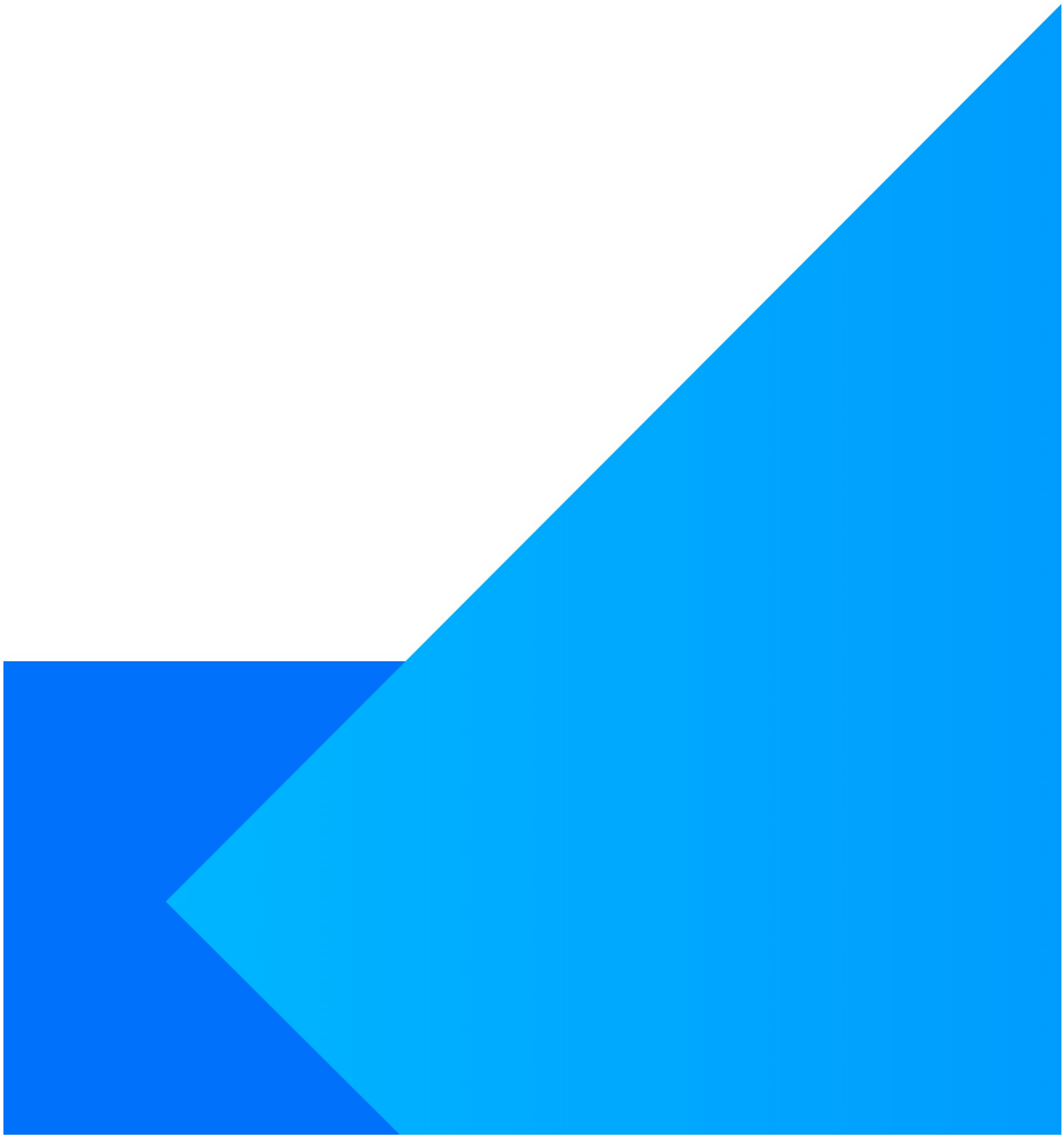
Alipay

Remittance solution for Kakaopay



Contents

Main flows

Alipay+ Remittance enables a Payer to send money instantly to a Beneficiary. The Payer can initiate the money transfer from a wallet, bank account, or by using a cash pickup code. See the following figure for an overview of the money transfer flow:

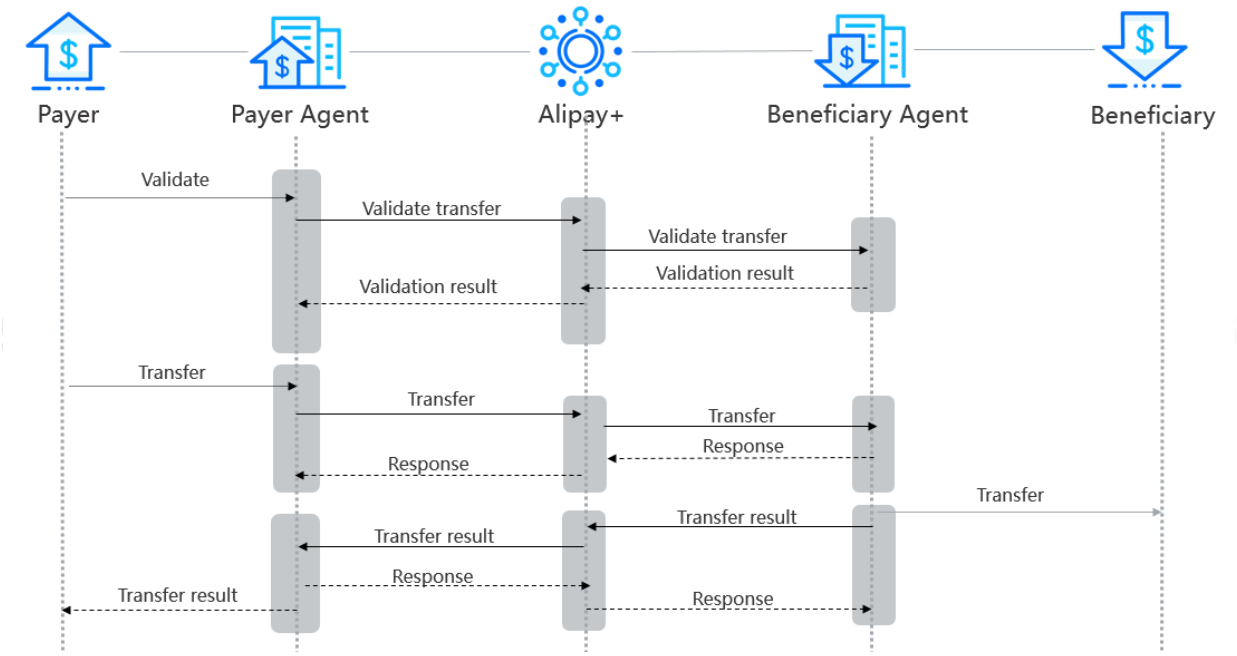


Figure 1. Money transfer flow

Balance management

This chapter describes details of balance management flow, which includes prefund, balance withdrawal, balance withdrawal result notification.

Prefund

For Beneficiary Agents that uses funds flow model 2, to perform remittance, Alipay+ needs first to add funds into the prefund account. Beneficiary Agent sends the prefund result to Alipay+ by email.

The following rules apply to the prefund service:

- **Purpose**: If reversal occurs, or the transfer is canceled, Beneficiary Agent needes to return transfer funds into Alipay+ Balance immediately.
- **Availability:** Prefunded funds can be used any time and every day. Prefund service can be also used any time.
- **Prefund account**: Alipay+ prefund into the accounts that are assigned by Beneficiary Agent.
- **Balance inquiry**: Alipay+ can inquiry the Balance anytime.
- **Prefund result notification**: The prefund result is sent by Beneficiary Agent by emails.
- **Frequency**: Not limited. Alipay+ can perform prefunding according to the specific business requirements.
- **Amount**: No minimal or maximum prefund amount limit exists.

The following figure illustrates the prefund flow of Alipay+.

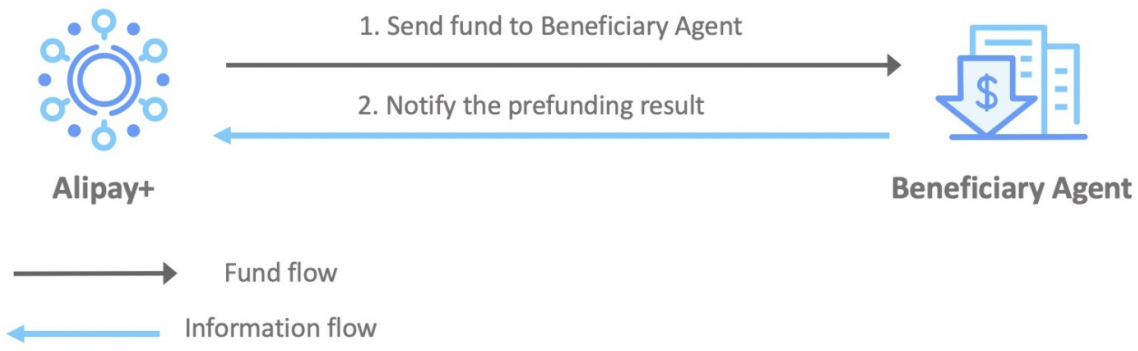


Figure 1. Prefund flow

The prefund flow has the following steps:

1. Alipay+ sends funds to Beneficiary Agent.
2. Beneficiary Agent returns the prefund result to Alipay+ by email.

Beneficiary Agent Balance inquiry

Alipay+ can use inquiryBalance interface to obtain the Balance information.

The following rules apply to Balance inquiry:

- **Purpose**: Provide the Balance information in real-time to Alipay+,

which includes,

  - Balance amount in all the available currencies.
  - The real-time Balance amount.
- **Availability**: Balance inquiry service can be used any time and every day.

The following figure illustrates the balance inquiry flow:

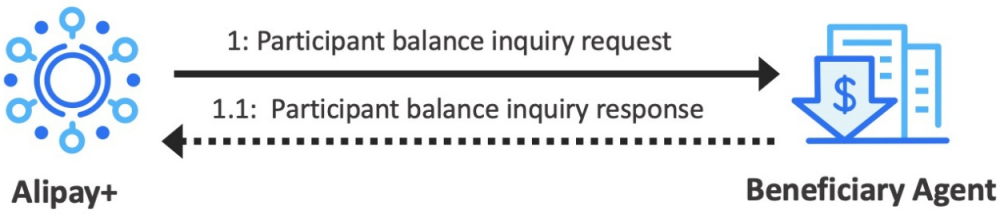


Figure 2. Balance inquiry flow

The balance inquiry flow has the following steps:

1. Alipay+ sends the inquiry request to Beneficiary.
2. Beneficiary responses to Alipay+.

Transfer

For beneficiary Agents, transfer flow includes transfer validation, transfer, transfer result notification, and transfer result inquiry.

Transfer validation

By using validateTransfer interface, Payer Agent can ask Beneficiary Agent to validate the transfer information to ensure that all the information required is contained and valid.

The following rules apply to the prefund:

- **Purpose**: Improve the user experience and the success rate of transfer.
- **Validation items**: Contents that Beneficiary Agent validates includes, but are not limited to the following items:
  - Legality of the beneficiary account.
  - Whether the Beneficiary and the beneficiary account match.
  - Whether Beneficiary meets KYC requirements.
  - Beneficiary collection quota (if available).
  - Legality of the payer (if available).
  - Currency check of incoming and outgoing payments
  - Purpose of the transfer (if available). The valid purposes vary according to the requirements in different countries.
- **Result**: The validation result is returned by the interface in real-time, and the result is either success or failure. No intermediary result exists.

The following figure illustrates the Transfer validation flow:

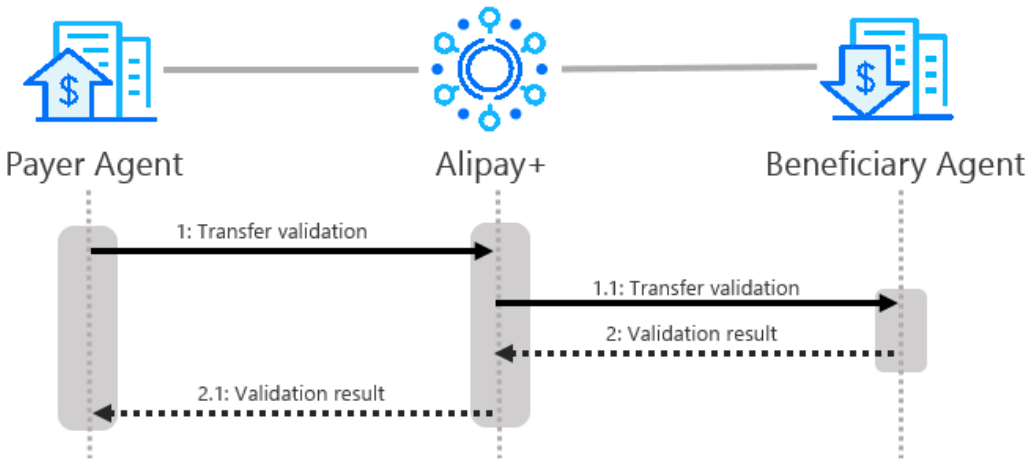


Figure 1. Transfer validation flow

The transfer validation flow has the following steps:

1. Payer Agent sends the transfer information to Alipay+, and then Alipay+ forwards the request to Beneficiary Agent for validation.
2. Beneficiary Agent returns the validation result to Alipay+, and then Alipay+ forwards the result to Payer Agent.

Transfer

Transfer request is initiated by Payer Agent. After a transfer request is accepted by Alipay+, Alipay+ forwards the request to Beneficiary Agent by using transfer interface. Beneficiary Agent processes the request and then returns the request acceptance result to Alipay+.

The final transfer result is returned by notifyTransfer or inquiryTransfer.

The following figure illustrates the transfer flow:

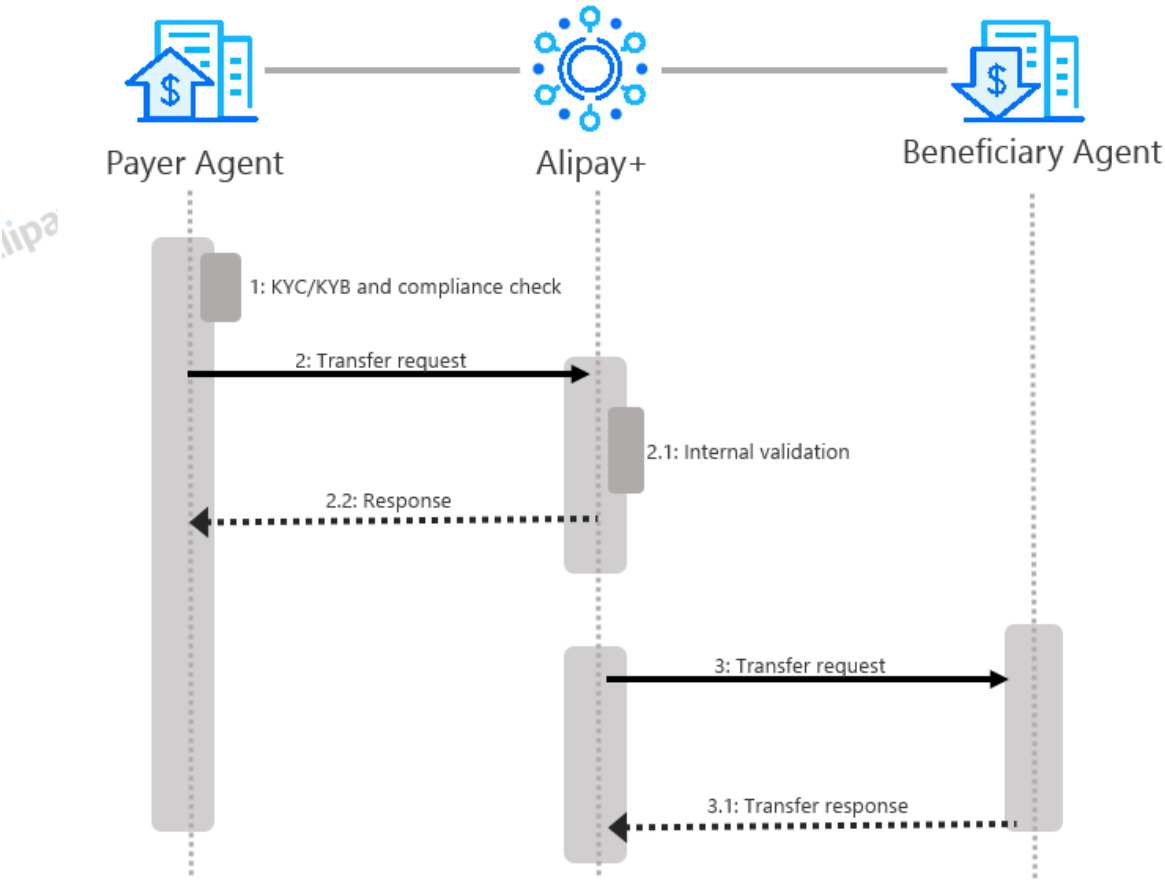


Figure 2. Transfer preparation flow

The transfer flow has the following steps:

1. Payer Agent completes internal preparations, such as validation of user information and AML scanning.
2. Payer Agent sends the transfer request to `Alipay+`.
3. `Alipay+` process the transfer request.
4. `Alipay+` sends the acceptance result to Payer Agent.
5. `Alipay+` sends the transfer request to Beneficiary Agent.
6. Beneficiary Agent processes the request and returns the acceptance result to `Alipay+`.

Transfer result notification

Beneficiary uses notifyTransfer interface to return the transfer result to `Alipay+`. `Alipay+` then forwards the transfer result to Payer Agent. Beneficiary Agent must ensure that the result is received by `Alipay+`. If notification timeout occurs, or `Alipay+` does not return `SUCCESS` to the notification, Beneficiary Agent must retry the notification according to the retrial rules.

The following figure illustrates the transfer result notification flow:

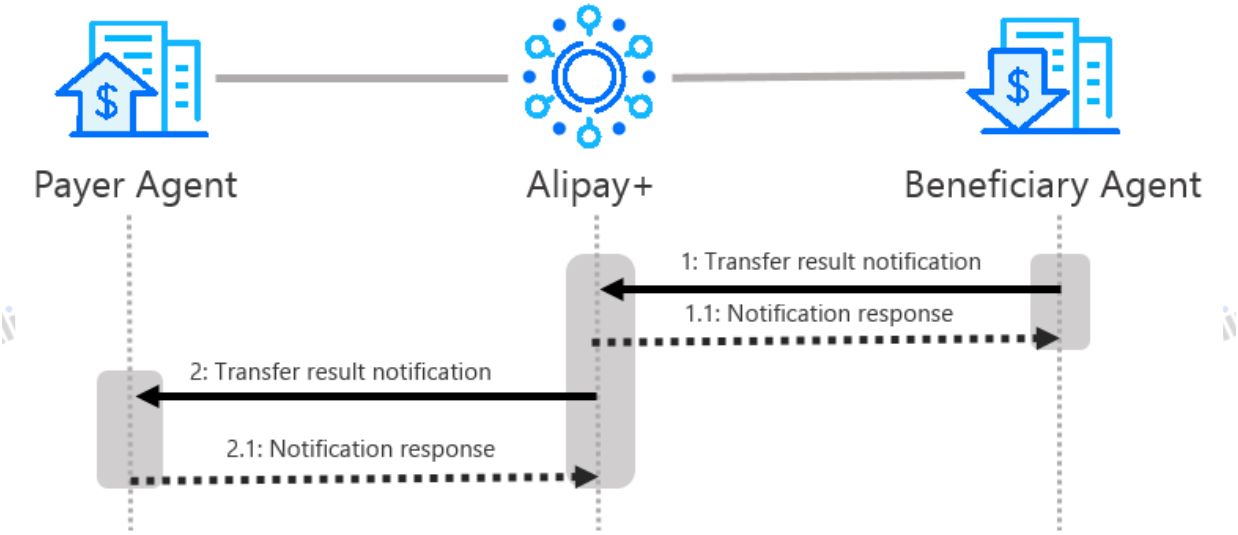


Figure 3. Transfer result notification flow

The transfer result notification flow has the following steps:

1. Beneficiary sends the transfer result to Alipay+.
2. Alipay+ processes the result and responses to Beneficiary Agent.
3. Alipay+ forwards the transfer result notification to Payer Agent.
4. Payer Agent processes the result and responses to Alipay+.

Transfer result inquiry

If Alipay+ does not receive the transfer result notification from the Beneficiary Agent for a specific time period (for example, 60 seconds) after the transfer request is sent, Alipay+ uses inquiryTransfer interface to query the result. If no clear result is returned, Alipay+ will retry the inquiry according to the frequency specified by the Beneficiary Agent during the integration. For example, Alipay+ might retry every 10 seconds for 7 times.

The following figure illustrates the transfer result inquiry flow:

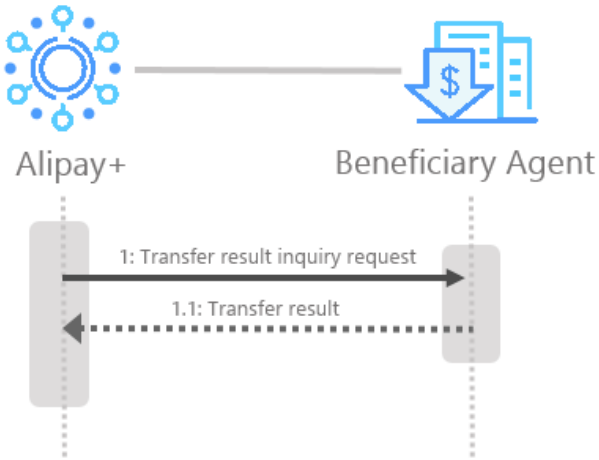


Figure 4. Transfer result inquiry flow

Transfer result inquiry flow has the following steps:

1. Alipay+ send the Transfer result inquiry request to Beneficiary Agent.
2. Beneficiary Agent returns the transfer result.

Transfer cancelation

Alipay+ uses cancelTransfer interface to request ~~that the~~ Beneficiary Agent cancel a transfer. Usually, if no capital loss is caused, Beneficiary Agent should agree the cancelation.

The following rules apply to the transfer cancelation:

- **Cancelable transfers**: Transfers that are in a final status of success or failure cannot be canceled. Only transfers in `COMPLIANCE_PENDING` or `WAIT_BENEFICIARY_CONFIRM` status can be canceled.
- Refund: Beneficiary Agent returns the transfer funds immediately after receiving the cancelation request.
  - If the Beneficiary Agent uses funds flow model 1, The canceled transaction will not be included in the outstanding bill on T+1 day
  - If the Beneficiary Agent uses funds flow model 2, the cancelation amount is automatically refunded to `Alipay+`'s prefunding Balance once the cancelation is processed sucessful.

The following figure illustrates the transfer cancelation flow:

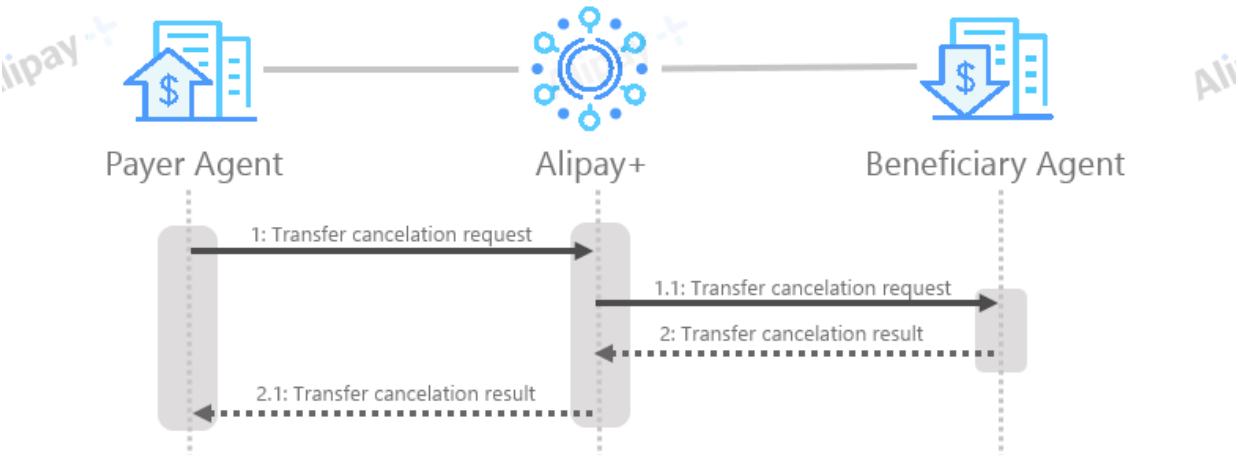


Figure 5. Transfer cancelation flow

The transfer cancelation flow has the following steps:

1. Payer Agent request a transfer cancelation and `Alipay+` sends the transfer cancelation request to Beneficiary Agent.
2. Beneficiary Agent cancels the transfer and then returns the cancelation result to `Alipay+`, who then sends the result to Payer Agent.

Files and reconciliation

Transfer transaction file contains transaction details information on T day, such as **TRANSFER**, **REVERSE**, **PREFUND**, **FX_TRADE** and **WITHDRAW**.

The file is UTF-8 encoded and uses GTM+8 time.

counterParticipantId is the unique ID of a Partner, indicating the receiver of the reconciliation file.

currency is the currency used in the reconciliation file, as agreed in the Partner's contract with Alipay+.

date is the reconciliation day, for example 20190101.

The file contains two sections, which are Balance, TransactionDetail. For more information about each section, see Sample for details.

Structure

Balance

No.	Field	Description
1	counterParticipantId	**Optional** String max(64) The unique ID that is assigned by Alipay+ to identify a counter Partner
2	balanceFundType	**Required** BalanceFundType The fund type of balance
3	currency	**Required** String (3) The three-character ISO-4217 currency code
4	openingAmount	**Required** String max(16) The available balance of Partner at the beginning of T day 00:00:00. The amount in the smallest currency unit
5	closingAmount	**Required** String max(16) The available balance of Partner at the end of T day 23:59:59. The amount in the smallest currency unit
6	transactionDate	**Required** Date The transaction date

7	businessScene	**Optional** BizSceneType
		Business scene type. Valid values are:
		`TRANSFER`
		`B2C_TRANSFER`
		`TUITION`,
		`HOUSE_RENTAL`

TransactionDetail

No.	Field	Description
1	transactionRequestId	**Required** String max(64) The unique ID that is assigned by the transaction initiator to identify a transaction request.
4	transactionId	**Required** String max(64) The unique ID that is assigned by the transaction executor to identify a transaction request.
3	TransactionType	**Required** TransferTransactionType The transaction type. The transfer transaction reconciliation file only contains the transaction type `TRANSFER`, `REVERSE`, `PREFUND`, `FX_TRADE` and `WITHDRAW`.
	originalTransactionRequestId	**Optional** String (64) The ID of the original transaction request. When transationType is REVERSE, the originalTransactionRequestId is the transferRequestId.
	TransactionCreationTime	**Optional** The creation time of a transaction
	transactionTime	**Required** Datetime The time of a successful transaction
7	amount	**Required** String max(16) The amount of the transactions in the smallest currency unit
8	currency	**Required** String (3) The three-character ISO-4217 currency code
	fundDirection	**Required** FundDirectionType The fund direction. Valid values are: • C: Credit • D: Debit
5	counterParticipantId	**Optional** String max(64) The unique ID that is assigned by Alipay+ to identify a counter Partner
	counterParticipantAmountValue	**Optional** String max(16) The transaction amount of the counter Partner. The amount of the transactions in the smallest currency unit .
	counterParticipantAmountCurrency	**Optional** String (3) Currency of counterParticipantAmountValue. A 3-letter currency code as defined in ISO 4217
	quoteId	**Optional** String max(64) The unique ID that is assigned by Alipay+ to identify a quote. Required if `transactionType` is `TRANSFER`.
	quotePrice	**Optional** Decimal max(20) The exchange rate used when a currency conversion occurs. Required if `transactionType` is `TRANSFER`.

	businessScene	**Optional** BizSceneType
		Business scene type. Valid values are:
		`TRANSFER`
		`B2C_TRANSFER`
		`TUITION`,
		`HOUSE_RENTAL`

Samples

The following example is a reconciliation file for a Beneficiary Agent with an ID of 105test000000001000.

The file name is "transactions_105test000000001000_USD_20221017.csv". The file contains three sections, which are **Balance**, **TransactionDetail**.

The following figure shows the sample details:

```
counterParticipantId,balanceFundType,currency,openingAmount,closingAmount,transactionDate
105test000000001000,QUOTA,USD,1000000,500000,2022-10-17T00:00:00+08:00
counterParticipantId,transactionRequestId,transactionType,originalTransactionRequestId,transactionCreationTime,transactionTime,amount,currency,fundDirection,counterParticipantAmountValue,counterParticipantAmountCurrency,quoteId,quotePrice,transactionId,businessScene
105test000000001000,TUITION_sample0001,TRANSFER,,2022-10-17T15:21:41+08:00,2022-10-17T17:21:41+08:00,100000,USD,D,687200,CNY,1.68E+18,6.872,2.02E+30,TUITION
105test000000001000,TUITION_sample0002,TRANSFER,,2022-10-17T11:40:41+08:00,2022-10-17T12:40:41+08:00,200000,USD,D,1374400,CNY,1.68E+18,6.872,2.02E+30,TUITION
105test000000001000,TUITION_sample0003,TRANSFER,,2022-10-17T09:35:37+08:00,2022-10-17T10:35:37+08:00,300000,USD,D,2061600,CNY,1.68E+18,6.872,2.02E+30,TUITION
105test000000001000,reverse_sample0001,REVERSE,transfer_sample0001,2022-10-17T11:52:50+08:00,2022-10-17T13:52:50+08:00,100000,USD,C,687200,CNY,1.68E+18,6.872,2.02E+30,TUITION
105test000000001000,prefund_sample0005,PREFUND,,2022-10-17T18:59:11+08:00,2022-10-17T20:59:11+08:00,100000,USD,C,,,,,,TUITION
105test000000001000,withdraw_sample0007,WITHDRAW,,2022-10-17T10:31:55+08:00,2022-10-17T11:31:55+08:00,100000,USD,D,,,,,,TUITION
<END>
```

Before you begin

Before you start integrating with Alipay, complete the following steps:

- Contact Alipay to retrieve the required integration information for both non-production and production:
  - Alipay outbound IP address
  - Partner ID
  - Client ID
  - Alipay public key
  - Endpoint
- Review API references and the content under appendix to be clear about the requirements of API interactions and data exchange.
- Prepare keys. You must generate your key pair, and exchange public keys with Alipay.
  - You can use the following commands to generate RSA keys.
    - For RSA private key:

```
## generate private key
openssl genrsa -out private_key.pem 2048
```

    - For RSA public key:

```
## export public key
openssl rsa -in bridge_private_key.pem -pubout -out public_key.pem
```

    - For PKCS8 format:

```
## convert to PKCS8 format
openssl pkcs8 -topk8 -inform PEM -in private_key.pem -out private_key_pkcs8.pem -nocrypt
```

  - To exchange public keys with Alipay, contact Alipay+ support.

Development

This section introduces how to integrate the APIs.

Making a request

The following sections describe the API calling processes with examples appended. The sample message contains no sensitive information, therefor encryption is not required and only request signing and signature validation are illustrated.

Sample request

Take the Transfer interface for example, assume the `Client-Id` is `TEST_5X0000000000****` and the gateway is `https://xxx/openapi/transfer/v1/transfer`.

Sample request message body:


```
{
 "transferRequestId": "20111111111111111111111111****",
 "transferFromAmount": {
  "currency": "HKD",
  "value": "8990"
 },
 "payerPaymentMethod": {
  "paymentMethodType": "WALLET",
  "walletDetail": {
   "customerId": "12****",
   "walletName": "HK_WALLET",
   "customerName": {
    "firstName": "William",
    "middleName": "Jefferson",
    "lastName": "Clinton",
    "fullName": "William Jefferson Clinton"
   }
  }
 },
 "transferToAmount": {
  "currency": "PHP",
  "value": "60000"
 },
 "beneficiaryReceiptMethod": {
  "paymentMethodType": "WALLET",
  "walletDetail": {
   "customerId": "927111****",
   "walletName": "PH_WALLET",
   "customerName": {
    "firstName": "FREDY",
    "middleName": "MANALO",
    "lastName": "SANTOS",
    "fullName": "FREDY MANALO SANTOS"
   }
  }
 },
 "instructedAmountType": "TRANSFER_TO",
 "transferQuote": {
  "quoteId": "CDEI1NWPAQ196****"
```

```
 },
"additionalTransferDetails": {
 "payer": {
  "birthDate": "1911****",
  "certificate": {
   "certificateNo": "G000000****",
   "certificateType": "PASSPORT"
  },
  "nationality": "US",
  "userAddress": {
   "address1": "Room 1xx1,Block x,xxx Castle Peak Road",
   "address2": "Tsuen Wan",
   "city": "HK",
   "state": "HK",
   "region": "HK",
   "zipCode": "999077"
  },
  "customerId": "12****",
  "userName": {
   "firstName": "William",
   "middleName": "Jefferson",
   "lastName": "Clinton",
   "fullName": "William Jefferson Clinton"
  },
  "userPhoneNo": "+8521111****"
 },
 "beneficiary": {
  "birthDate": "1911****",
  "certificate": {
   "certificateNo": "G000000****",
   "certificateType": "PASSPORT"
  },
  "nationality": "PH",
  "userAddress": {
   "address1": "Dasmarinas Village",
   "address2": "1xxx Pasay Road",
   "city": "Makati",
   "state": "Metro Manila",
   "region": "PH",
   "zipCode": "999005"
  },
  "userName": {
   "firstName": "FREDY",
   "middleName": "MANALO",
   "lastName": "SANTOS",
   "fullName": "FREDY MANALO SANTOS"
  }
 },
 "transferFromRegion": "HK",
 "transferToRegion": "PH",
 "transferReference": "transfer reference",
 "transferPurpose": "FAMILY_SUPPORT"
}
}
```

Step 1. Signing the request

Complete the following steps to sign a request.

1. Obtain the private key

You need the Partner private key to sign the request. For more information about how to obtain and configure the key, see Technical specification for keys and certificates.

2. Preparing the content to be signed

The content to be signed is:

```
<HTTP-method> <HTTP-URI-with-query-string>
<Client-Id>.<Request-Time|Response-Time>.<HTTP BODY>
```

In the following example, assume the value of `Request-Time` is `2019-05-28T12:12:12+08:00`

```
POST /openapi/transfer/v1/transfer
TEST_5X00000000000000.2019-05-28T12:12:12+08:00.{
 "transferRequestId": "20111111111111111111111111****",
 "transferFromAmount": {
  "currency": "HKD",
  "value": "8990"
 },
 "payerPaymentMethod": {
  "paymentMethodType": "WALLET",
  "walletDetail": {
   "customerId": "12****",
   "walletName": "HK_WALLET",
   "customerName": {
    "firstName": "William",
    "middleName": "Jefferson",
    "lastName": "Clinton",
    "fullName": "William Jefferson Clinton"
   }
  }
 },
 "transferToAmount": {
  "currency": "PHP",
  "value": "60000"
 },
 "beneficiaryReceiptMethod": {
  "paymentMethodType": "WALLET",
  "walletDetail": {
   "customerId": "927111****",
   "walletName": "PH_WALLET",
   "customerName": {
    "firstName": "FREDY",
    "middleName": "MANALO",
    "lastName": "SANTOS",
    "fullName": "FREDY MANALO SANTOS"
   }
  }
 },
 "instructedAmountType": "TRANSFER_TO",
 "transferQuote": {
```

```
 "quoteId": "CDEI1NWPAQ196****"
},
"additionalTransferDetails": {
 "payer": {
  "birthDate": "1911****",
  "certificate": {
   "certificateNo": "G000000****",
   "certificateType": "PASSPORT"
  },
  "nationality": "US",
  "userAddress": {
   "address1": "Room 1xx1,Block x,xxx Castle Peak Road",
   "address2": "Tsuen Wan",
   "city": "HK",
   "state": "HK",
   "region": "HK",
   "zipCode": "999077"
  },
  "customerId": "12****",
  "userName": {
   "firstName": "William",
   "middleName": "Jefferson",
   "lastName": "Clinton",
   "fullName": "William Jefferson Clinton"
  },
  "userPhoneNo": "+8521111****"
 },
 "beneficiary": {
  "birthDate": "1911****",
  "certificate": {
   "certificateNo": "G000000****",
   "certificateType": "PASSPORT"
  },
  "nationality": "PH",
  "userAddress": {
   "address1": "Dasmarinas Village",
   "address2": "1xxx Pasay Road",
   "city": "Makati",
   "state": "Metro Manila",
   "region": "PH",
   "zipCode": "999005"
  },
  "userName": {
   "firstName": "FREDY",
   "middleName": "MANALO",
   "lastName": "SANTOS",
   "fullName": "FREDY MANALO SANTOS"
  }
 },
 "transferFromRegion": "HK",
 "transferToRegion": "PH",
 "transferReference": "transfer reference",
 "transferPurpose": "FAMILY_SUPPORT"
}
}
```

3. Generating the signature

Use the following command to generate the signature:

```
base64UrlEncode(sha256withrsa(<unsignedContent>), <privateKey>))
```

where,

The content to be signed is `<unsignedContent>`, the algorithm used is RSA 256, and the private key value is `<privateKey>`.

The generated signature is:

```
KrwDE9tAPJYBb4cUZU6ALJxGIZgwDXn5UkFPMip09n%2FkYKPhEIII%2Fki2rYY2lPtuKVgMNz%2BtuCU%2FjzRpohDbrO
d8zYriiukpGAxBQDIVbatGI7WYOcc9YVQwdCR6ROuRQvr%2FD1AfdhHd6waAASu5Xugow9w1OW7Ti93LTd0tcyEWQYd2
S7c3A73sHOJNYI8DC1PjasiBozZ%2FADgb7ONsqHo%2B8fKHsLygX9cuMkQYTGIRBQsvfgICnJhh%2BzXV8AQoecJBTrv
6p%2B7MQC4VcqJGcoLE9aqNnPCX1O8G4stEPwspYwShsjhp5AVjeCNvk0ar9JIqQ6VZ0nMQGwZoZ6gcFw%3D%3D
```

Step 2. Constructing the request

A header contains the following fields:

* Client-Id

* Signature

* Content-Type

* Request-Time

* Encrypt (optional)

For more information, see Technical specification for messaging interface.

In this example, the request is sent by using cURL. Add `Client-Id`, `Request-Time`, and `Signature` to the request header:

```
curl -X POST \
  https://xxx/openapi/transfer/v1/transfer \
  -H 'Content-Type: application/json' \
  -H 'Client-Id: TEST_5X0000000000****' \
  -H 'Request-Time: 2019-05-28T12:12:12+08:00' \
  -H 'Signature: algorithm=RSA256, keyVersion=0,
signature=KrwDE9tAPJYBb4cUZU6ALJxGIZgwDXn5UkFPMip09n%2FkYKPhEIII%2Fki2rYY2lPtuKVgMNz%2BtuCU%2FjzRpohDbrOd8zYriiukpGAxBQDIVbatGI7WYOcc9YVQwdCR6ROuRQvr%2FD1AfdhHd6waAASu5Xugow9w1OW7Ti93LTd0tcyEWQYd2S7c3A73sHOJNYI8DC1PjasiBozZ%2FADgb7ONsqHo%2B8fKHsLygX9cuMkQYTGIRBQsvfgICnJhh%2BzXV8AQoecJBTrv6p%2B7MQC4VcqJGcoLE9aqNnPCX1O8G4stEPwspYwShsjhp5AVjeCNvk0ar9JIqQ6VZ0nMQGwZoZ6gcFw%3D%3D' \
  -d '{
 "transferRequestId": "201111111111111111111111111****",
 "transferFromAmount": {
  "currency": "HKD",
  "value": "8990"
 },
 "payerPaymentMethod": {
  "paymentMethodType": "WALLET",
  "walletDetail": {
   "customerId": "12****",
```

```
  "walletName": "HK_WALLET",
  "customerName": {
   "firstName": "William",
   "middleName": "Jefferson",
   "lastName": "Clinton",
   "fullName": "William Jefferson Clinton"
  }
 }
},
"transferToAmount": {
 "currency": "PHP",
 "value": "60000"
},
"beneficiaryReceiptMethod": {
 "paymentMethodType": "WALLET",
 "walletDetail": {
  "customerId": "927111****",
  "walletName": "PH_WALLET",
  "customerName": {
   "firstName": "FREDY",
   "middleName": "MANALO",
   "lastName": "SANTOS",
   "fullName": "FREDY MANALO SANTOS"
  }
 }
},
"instructedAmountType": "TRANSFER_TO",
"transferQuote": {
 "quoteId": "CDEI1NWPAQ196****"
},
"additionalTransferDetails": {
 "payer": {
  "birthDate": "1911****",
  "certificate": {
   "certificateNo": "G000000****",
   "certificateType": "PASSPORT"
  },
  "nationality": "US",
  "userAddress": {
   "address1": "Room 1xx1,Block x,xxx Castle Peak Road",
   "address2": "Tsuen Wan",
   "city": "HK",
   "state": "HK",
   "region": "HK",
   "zipCode": "999077"
  },
  "customerId": "12****",
  "userName": {
   "firstName": "William",
   "middleName": "Jefferson",
   "lastName": "Clinton",
   "fullName": "William Jefferson Clinton"
  },
  "userPhoneNo": "+8521111****"
 },
```

```
  "beneficiary": {
   "birthDate": "1911****",
   "certificate": {
    "certificateNo": "G000000****",
    "certificateType": "PASSPORT"
   },
   "nationality": "PH",
   "userAddress": {
    "address1": "Dasmarinas Village",
    "address2": "1xxx Pasay Road",
    "city": "Makati",
    "state": "Metro Manila",
    "region": "PH",
    "zipCode": "999005"
   },
   "userName": {
    "firstName": "FREDY",
    "middleName": "MANALO",
    "lastName": "SANTOS",
    "fullName": "FREDY MANALO SANTOS"
   }
  },
  "transferFromRegion": "HK",
  "transferToRegion": "PH",
  "transferReference": "transfer reference",
  "transferPurpose": "FAMILY_SUPPORT"
 }
}'
```

Receiving the response

Sample response

Take the Transfer interface for example, the response message consists of response header and response body:

Sample response

```
{
 "result": {
   "resultCode":"SUCCESS",
   "resultStatus":"S",
   "resultMessage":"success"
 }
}
```

Validating the signature

Complete the following steps to validate a signature:

1.Construct the content to be verified

The content to be verified is:

```
<HTTP-method> <HTTP-URI-with-query-string>
<Client-Id>.<Request-Time|Response-Time>.<HTTP BODY>
```

For example:

```
POST /transfer/v1/transfer
TEST_5X00000000000000.2019-05-28T12:12:14+08:00.{
 "result": {
   "resultCode":"SUCCESS",
   "resultStatus":"S",
   "resultMessage":"success"
 }
}
```

2. Validating the signature

Use the following code to validate the signature:

```
sha256withrsa_verify(base64UrlDecode(<signature>), <content_to_be_verified>, <serverPublicKey>)
```

Demo code

The following demo code describes signature signing and validation as a whole.

```
import org.apache.commons.codec.binary.Base64;
import org.apache.commons.io.IOUtils;

import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.net.URLEncoder;
import java.security.KeyFactory;
import java.security.NoSuchAlgorithmException;
import java.security.PrivateKey;
import java.security.PublicKey;
import java.security.Signature;
import java.security.SignatureException;
import java.security.spec.InvalidKeySpecException;
import java.security.spec.PKCS8EncodedKeySpec;
import java.security.spec.X509EncodedKeySpec;


public class OpenApiV2 {

   private static String privateKey
         =
"MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQDEmDOUHeEiwxpsxd+29b8Dvf4bdkT5iAgjpeEOwl/T+g
JVaEUUnkEBSvoakULobEZYwTnvioip/IdOA71OfFTii3ilGG+IU1l0sWly7CsW/u+NybBfNJ3kAFsxpxjdr3h9/6gZlGZdW/3Qzm
Wk3YBgGwi7GcFIFSyyXSE3aoWoQNsJ0hk1eULMW77D9DXdEvVqRqFRx61O/aAzTccPeN+QmPLDMVNimgsyGV+PePU
a8XDEaYx7G4XBs4pdY6TmiEHxnPXKTW2U3Rsy2rbjWq1GmbE/yz+fEBJ8ePDnrrB8Yx8+FB0n6QT9rxZx5Z48tQPphaMruA
YLJWtu4rs16oIhAgMBAAECggEALrP9pNVIU6rHz0lesSdGPoxXHo0FK8ZuMJHKmmgAvenpvgJsTs6doMIAEmmp3G9Ecvd
MXSSq/L76IcVvpoKQ49TAT3Azps4sfEodQ+jo/6iGnkdot+VXEQAoqe4JUTjLRhuDArinbMxays0NhpH8WlhxxYkDyq0FFkZbir
+qiuKaz27k7XXBxylk6xKUkppoVk1XVQAAb/x2PafQXzUZvPeACy4B7EA4mat6Yzwz8fsRpzdzwzvYItt183lEDCDG9y0iK53J1
Mtt58KN03YEsqb7VaG+okKwkhckGPWXETTITIJK+RFYbYREl18zQ4MhRexIF+HBbZhGqkgdKncIVQKBgQDgwO9SM6Ni7
fYsYG+MUFVH/ehNC6nsbxOZVCCZGPDqk26FMPPyXiQcFH7jBdqd+hTHlkiBW6nSam1NKd/bOuItYGpNqx18t6L6b7ylIMC
uSGqgx+rsP4bVHOEspkXZQahOP45Htrt0tsEmEROvgfBuTnuZhMT8ueXs4id8KpvSgwKBgQDf7RLI6Kl09lW4Mya/ATkVJm
zvwK0lIJ5m7fD7ROPsVr4vHBznCh1bwYFcWNbNanGflto1J/fRyBzF7N9VXPjiqeS+kg7qhihPC8E5EzDfsOQRfnwq25OE8/66
quma2rQh2poIwSroeq4r6FRlkT43Xsy0jqpnIo4SId9Y+0nniwKBgQCB/fUtdSJt6LY2SnyaCiiZH/kI/tMEJeaPC0JjajKFakpFDO
A7cUmvWAdfpRYhwu/b1EcRUoPxaI7ZT3vjH7aLijVRwMOoS8oe0Rup4jyvUMEjRx3QEJoajr6KBY1mr/v5bfcL8VuQkNc7/iqF
m0EFe60vSMRqa9bZYKB3x2kiowKBgC3Ds/0fnlcYqTg2LAsJYvMxoT32sOxPmFAM/R09XR+mMsRKCJYneQyRpbCL74qX
4JTvLbWDb6CON2M0IAjAMZWRh1BzZV0FAiP1kQ3rICf3bQUuRo4zxhh8DWI1GL0vSXVJygVUBULyH7qVhvojDwYd99RG/
AiMBWErTwxOfzI1AoGAb9lvSL8IKubxojlcXcrMj95+bXPS4sXqXRGQ4PyZ01tDV7NrGNMLvIViSnMJxXbzjPtaC0VKof0dncL
```

```
CCF0XKjlwHJWZ5/nliYot0azWMuhPL7z09ePqhEsIQfia8Tiy0nJL/9PU+oFwzVOhNwXfCuJrc3GprpVj5wboHts5LSw=";
    private static String publicKey

="MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAxJgzlB3hlsMabMXftvW/A73+G3ZE+YgII6XhDsJf0/oCVWhFFJ
5BAUr6GpFC6GxGWME574qIqfyHTgO9TnxU4ot4pRhviFNSNLFpcuwrFv7vjcmwXzSd5ABbMacY3a94ff+oGZRmXVv90M5l
pN2AYBsIuxnBSBUssl0hN2qFqEDbCdIZNXlCzFu+w/Q13RL1akahUcetTv2gM03HD3jfkJjywzFTYpoLMhlfj3j1GvFwxGmMex
uFwbOKXWOk5ohB8Zz1yk1tlN0bMtq241qtRpmxP8s/nxASfHjw566wfGMfPhQdJ+kE/a8WceWePLUD6YWjK7gGCyVrbuK7
NeqCIQIDAQAB";

    public static void main(String[] args) throws SignatureException {
        String httpMethod = "POST";
        String uriWithQueryString = "/aps/api/transfer/v1/validateTransfer";
        String clientId = "TEST00000****";
        String timeString = "2019-11-26T23:32:32-08:12";

        String content = "{\"beneficiary\": {\"userName\": {\"firstName\": \"GUO\",\"lastName\": \"JU\",\"middleName\":
\"\",\"fullName\": \"GUOJU\"}},\"transferToAmount\": {\"currency\": \"CNY\",\"value\": \"10000\"},\"beneficiaryReceiptMethod\":
{\"paymentMethodType\": \"WALLET\",\"walletDetail\": {\"walletName\": \"WALLET_ALIPAY_CN\",\"customerId\":
\"1382063****\",\"customerName\": {\"firstName\": \"GUO\",\"lastName\": \"JU\",\"middleName\": \"\",\"fullName\":
\"GUOJU\"}}},\"transferFromRegion\": \"KR\",\"transferToRegion\": \"CN\", \"instructedAmountType\": \"TRANSFER_TO\"}";

        String payload = httpMethod + " " + uriWithQueryString + "\n" + clientId + "." + timeString
                + "." + content;

        OpenApiV2 openApiV2 = new OpenApiV2();
        String sign = openApiV2.doSign(payload, privateKey, "UTF-8");

        System.out.println(urlEncode(sign));
        System.out.println(" ");
        System.out.println("Verify Signature result："+openApiV2.doCheck(payload, sign, publicKey, "UTF-8"));

    }

    /**
     * URL encode
     *
     * @param s
     * @return
     */
    private static String urlEncode(String s) {
        if (s == null) {
            return null;
        }
        try {
            return URLEncoder.encode(s, "UTF-8");
        } catch (Exception e) {
        }

        return null;
    }

    private boolean doCheck(String content, String sign, String publicKey, String charset) throws SignatureException {
        try {
            PublicKey pubKey = getPublicKeyFromX509("RSA", new ByteArrayInputStream(publicKey.getBytes()));

            Signature signature = Signature
```

```
            .getInstance("SHA256withRSA");

        signature.initVerify(pubKey);
        signature.update(content.getBytes(charset));

        return signature.verify(Base64.decodeBase64(sign.getBytes()));
    } catch (Exception e) {
        throw new SignatureException("RSA Signature[content = " + content + "; charset = " + charset
                + "; signature = " + sign + "]exception!", e);
    }
}

private String doSign(String content, String privateKey, String charset) throws SignatureException {
    try {
        PrivateKey priKey = getPrivateKeyFromPKCS8("RSA", new ByteArrayInputStream(privateKey.getBytes()));

        Signature signature = Signature.getInstance("SHA256withRSA");

        signature.initSign(priKey);
        signature.update(content.getBytes(charset));

        byte[] signed = signature.sign();

        return new String(Base64.encodeBase64(signed));
    } catch (Exception e) {
        throw new SignatureException("RSA Signature[content = " + content + "; charset = " + charset
                + "] exception!", e);
    }
}

private PrivateKey getPrivateKeyFromPKCS8(String algorithm, ByteArrayInputStream ins) {
    try {
        KeyFactory keyFactory = KeyFactory.getInstance(algorithm);
        byte[] encodedKey = IOUtils.toByteArray(ins);
        encodedKey = Base64.decodeBase64(encodedKey);
        return keyFactory.generatePrivate(new PKCS8EncodedKeySpec(encodedKey));
    } catch (IOException var4) {
    } catch (InvalidKeySpecException var5) {
    } catch (NoSuchAlgorithmException e) {
        e.printStackTrace();
    }

    return null;
}

private PublicKey getPublicKeyFromX509(String algorithm, ByteArrayInputStream ins) {
    try {
        KeyFactory keyFactory = KeyFactory.getInstance(algorithm);
        byte[] encodedKey = IOUtils.toByteArray(ins);
        encodedKey = Base64.decodeBase64(encodedKey);
        return keyFactory.generatePublic(new X509EncodedKeySpec(encodedKey));
    } catch (IOException var5) {
        var5.printStackTrace();
    } catch (InvalidKeySpecException var6) {
        var6.printStackTrace();
```

```
        } catch (NoSuchAlgorithmException e) {
            e.printStackTrace();
        }

        return null;
    }

}
```

General result processing suggestions

The resultStatus can be S, F, or U. While S means the request is successfully processed, F and U indicates that something might go wrong. You can refer to F & U Suggestion for general result processing methods and then take actions accordingly.

Receiving notifications

Contact Alipay technical support team and send your endpoints that are used to receive notifications.

You can use notifications to automate business processes. To process notifications, you must:

1. Expose an endpoint on your server.
2. Accept notifications and acknowledge the notification with required response
3. Apply your business logic

Response to notifications

After receiving a notification, send back a response to Alipay+. Like request messages, such a response also contains a header and a body, and must be properly constructed.

A response header mainly contains fields such as `client-Id`, `signature`, `content-type`, `response-time`, and `encrypt` (optional). For more information, see Technical specification for messaging interface.

For more information about how to create the signature or encrypt the message, see Technical specification for messaging interface.

Sample response:

```
curl -X POST \
  https://xxx/openapi/alipay/transfer/result/notify \
  -H 'content-Type: application/json' \
  -H 'client-Id: TEST_5X0000000000****' \
  -H 'response-time: 2020-09-09T11:58:02+08:00' \
  -H 'signature:
algorithm=RSA256,keyVersion=2,signature=Ytk4sPyVfoXtKW5y0A0R7uCRgcRBS3dA0huWIFZinZFktLbkosbO3fOtHZGHort
o7KK9rTdaukCrDMUk9cWYAMkb9Yzl26s6QECn5Gpr%2BT8PmGmdTlF1qixNmlHuIlutMh4PiFQnLMqZ%2FGtgMstFPPuaf
%2Fef84HAYkkm2wQZf2VmZiv0BhwuTIaZAqoOZs6FR4L3eSRCJXDkTZu973j01cTnb%2FJyDg6RxeACzTS28G2stDtFTJj
L4Ini9vZaXij8hQPoXqmV5dJZVCPxJ2Q4uhj3EkgWfuAaMrZ3MWXyeBrXhyWcOyr0vuBZ648dn8Bh5ov2nSeRoMmgKda%2
BSBee%2Bxw%3D%3D' \
  -d '{
  "result": {
    "resultCode": "SUCCESS",
    "resultMessage": "success",
    "resultStatus": "S"
  }
}'
```

Notification sending retry strategy

To ensure that your server is properly accepting notifications, Alipay requires you to acknowledge every notification with a `success` response.

If Alipay doesn't receive the success response within **10** seconds, for example because your server is down, the notification to your endpoint will be retried. Retry attempts happen regularly for **7** times at increasing time intervals:

- 2 minutes
- 10 minutes
- 10 minutes
- 1 hour
- 2 hours
- 6 hours
- 15 hours

Handling errors

As you integrate, you might receive error messages for many reasons, such as a failed transfer, invalid parameters, authentication errors, and network unavailability. It's recommended that your code can gracefully handle all possible API exceptions. For information on these errors and how to resolve them, see API result code.

Best practices

To be in good standing as an Alipay Partner, refer to the following sections for accurate, up-to-date information and best practices to help you process transactions.

Compatibility

You must ensure the compatibility of optional parameters. For example, if an API is updated and new optional parameters are added, you must be able to properly handle the response and validate the signature.

Retry strategy

Notification

Normally the notification will first time reach in **10 seconds**. If the notification was not responded as "**S**", but **"U" or "F"**, the notification **will retry**. Retry attempts happen regularly for 7 times at increasing time intervals:

- 2 minutes
- 10 minutes
- 10 minutes
- 1 hour
- 2 hours
- 6 hours
- 15 hours

Query

Query can be called any time when your business is under unknown status or any other needs.

Our suggested query strategy is retrying **7 times** with an interval of **{5m, 10m, 20m, 40m, 80m, 160m, 320m}.** If all queries are not successful, though very unlikely, something tricky might happen, contact Alipay technical support team for help.

Idempotence

Idempotence is supported by Alipay interfaces, allowing you to retry a request multiple times while only performing the action once. This helps avoid unwanted duplication in case of failures and retries. For example, in the case of a timeout error, it is possible to safely retry sending the same API payment call multiple times with the guarantee that the transfer will only be performed once.

Scenario 1: Two requests, connectivity lost

Scenario:

- T0: Sender sends a Transfer request to the Alipay with request ID `ABC12****`.
- T1: Alipay receives this request and processes it successfully.
- T2: Sender's server loses power prior to receiving the response in T1.
- T3: Sender's server power is restored and the same Transfer request (ID `ABC12****` is sent with all the same parameters (especially the body of request) to Alipay.

Outcome:

In this case Alipay must reply with the same result given at T1 since all the body parameters of the request are the same. The balance is only redeemed once. T3 has no monetary impact to Sender.

Scenario 2: Two requests, first request during maintenance

Scenario:

- T0: Alipay is down for maintenance.
- T1: Sender sends a Transfer request to Alipay with request ID `ABC1234****`.
- T2: Alipay correctly returns `UNKNOWN` status code.
- T3: Sender's server receives response. Schedules a retry.
- T4: Alipay is back online.
- T5: Sender sends same request from T1 (especially the body of request).
- T6: Alipay receives request and returns an OK status code along with full response.

Outcome:

In this case Alipay must process the request in T6 and do not return `UNKNOWN`. Instead, Alipay will fully process the request and return OK with appropriate result message. Note that while the system is `UNKNOWN`, Sender might make repeated requests similar to T1 and receive messages appeared in T2.

Scenario 3: Two requests, Sender error in request

Scenario:

- T0: Sender sends a Transfer request to Alipay with request ID `ABC123****`.
- T1: The Alipay server receives this request and processes it successfully.
- T2: Sender's server loses power prior to receiving the response in T1.
- T3: Sender's server power is restored and the same request (ID `ABC123****`) is sent. Unfortunately Sender has an error and some of the parameters are different.

Outcome:

In this case Alipay must reply with an `REPEAT_REQ_INCONSISTENT` error code which indicates that errors might exist in Sender system.

API fundamentals

Alipay Remittance offers a set of Application Programming Interfaces (APIs) that provides the ability to perform transfer related interactions such as validate transfers, make transfers, and query results. You can use POST method to send HTTPS requests and receive response accordingly.

Before you make any transfers, it is important to understand how Alipay API works and how requests and responses are structured. This section presents general information, such as message structure, message fields, and message transmission, of online message between your system and Alipay.

Message structure

Before you construct any request, it is important to understand how API works and how requests and responses are structured.

Request structure

The following figure illustrates the request structure.

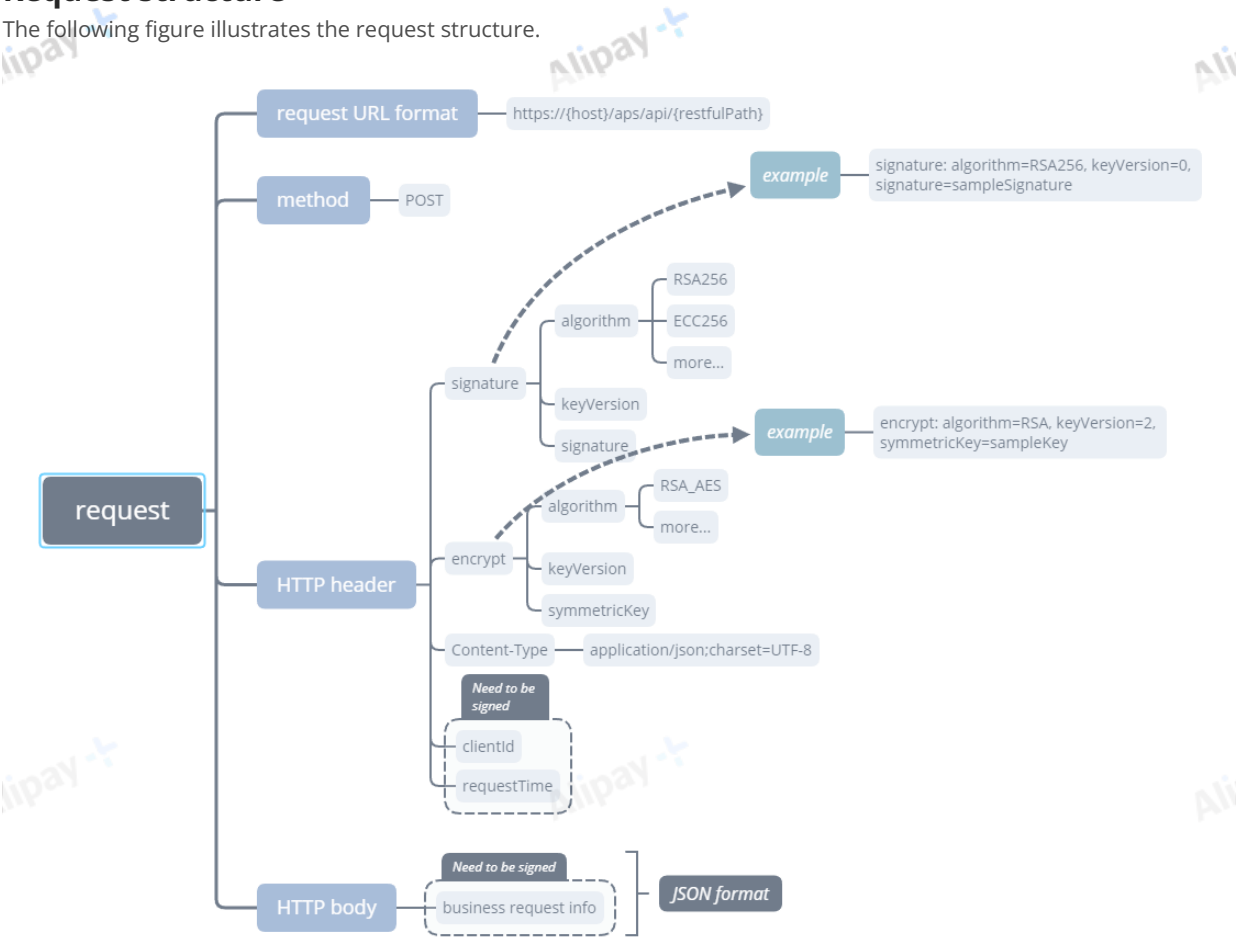


Figure 1. Request structure

Request URL

The request URL is https://{host}/api/{resful_path},

where,

- #1 is fixed as **https://**
- #2 is the **domain name of the endpoint**.
- #3 is fixed as **aps/api/**
- #4 is the restful path of the interface and it is case sensitive.

Request method

POST method is used to make an HTTP request.

Request header

The request header mainly contains fields such as:

- **Client-Id**
- **Signature**
- **Content-Type**
- **Request-Time**
- **Encrypt** (optional)

For more information about these fields, see Technical specification for messaging interface - Request header. For more information about **Encrypt**, see Encrypt and decrypt a message.

Request body

The request body contains the detailed request information in a JSON format. Fields enclosed in the request body vary depending on services. For more information, see the specific API specification.

Response structure

The following figure illustrates the response structure.

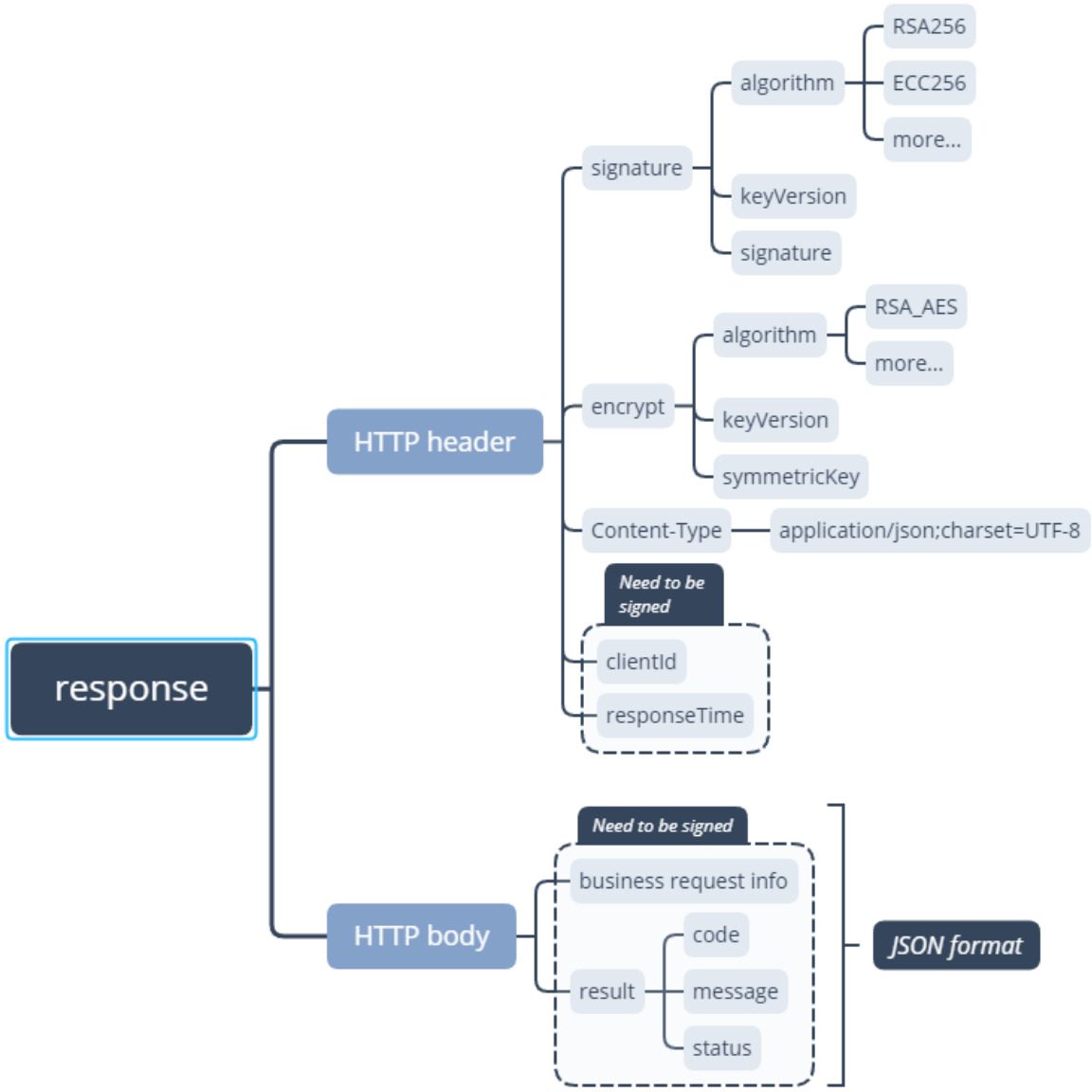


Figure 2. Request structure

Response header

The response header mainly contains fields such as:

- **Client-Id**
- **Signature**
- **Content-Type**
- **Response-Time**
- **Encrypt** (optional)

For more information about these fields, see Technical specification for messaging interface - Response header. For more information about **Encrypt**, see Encrypt and decrypt a message.

Response body

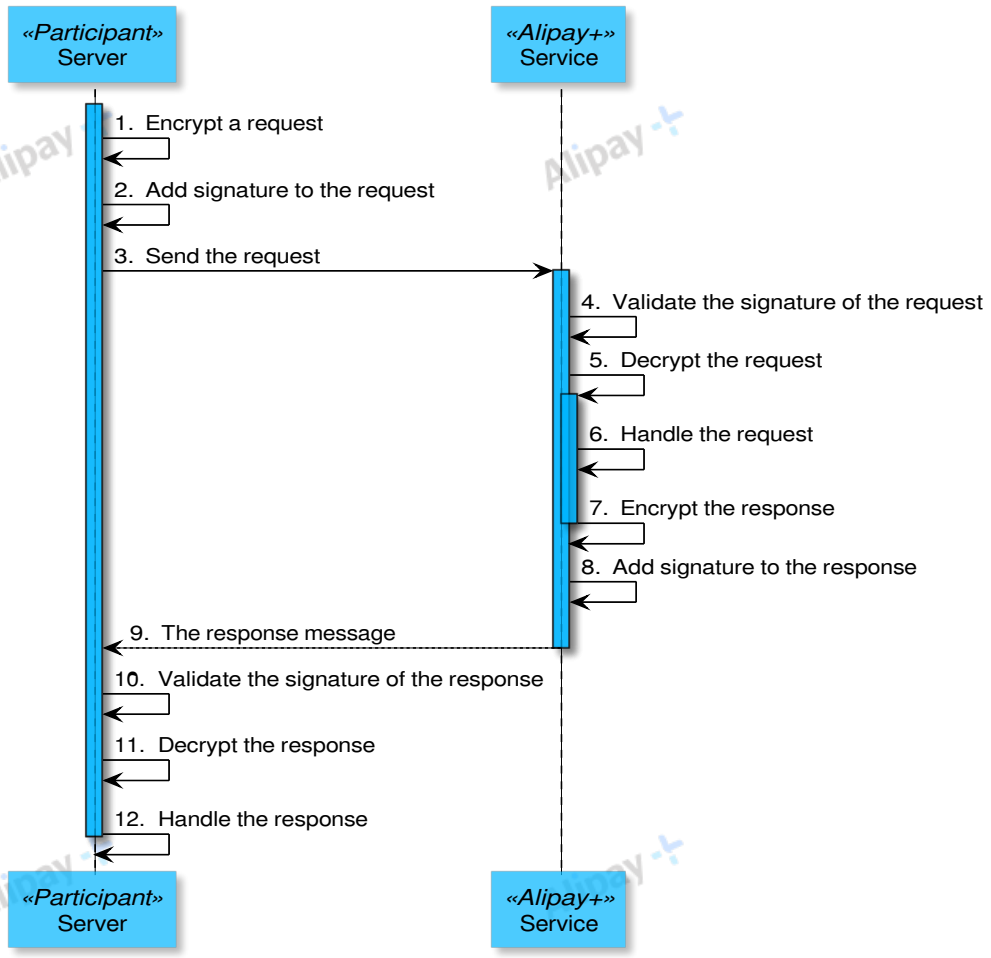
Response body contains the information responding to the client. Fields in this section vary depending on services. However, the `result` field, which indicates the result of an API call, is always contained. result field contains the following subparameters:

Field	Description
resultCode	**Required** String max(16) Result code
resultStatus	**Required** ResultStatusType Result status. Valid values are: • `S` : Successful • `F` : Failed • `U` : Unknown While `S` means the request is successfully processed, `F` and `U` indicates that something might go wrong. You can refer to F & U Suggestion for general result processing methods and then take actions accordingly.
resultMessage	**Optional** String max(64) Result message that describes *resultCode* in detail.

Each interface can return a set of `resultCode` and `resultMessage`, see API result code for details.

Message transmission workflow

The following figure illsutrates the API message transaction workflow:



Overall procedure

Follow the overall procedure to call an API.

Preparations

Authentication

Authenticate the request before sending the request. To authenticate your requests，you must use an API key. You must include the <API key> in the header of each API request. For more information about how to obtain and configure the key, see Technical specification for keys and certificates.

API idempontency

To prevent some potential errors that you might get in the response, consider API idempontence.

Processing special characters

The following rules are about how to process special characters enclosed in the message.

base64

For byte data, such as the signature and the encrypted content, encode the data with the base64 algorithm before transmitting.

urlencode

For URL data, perform URL encoding first before transmitting. For example:

Original URL: `https://www.merchant.com/authorizationResult`

Converted URL: `https%3A%2F%2Fwww.merchant.com%2FauthorizationResult`

1. Construct a request

Construct a request by complying with the request structure, for example, by adding the `Client-Id`, `Request-Time`, `Signature`, and other fields to the request header.

To ensure the message transmission security, perform the following security measures when constructing a request.

1. Encrypt a request when the request contains sensitive information. If encryption is required, the message body should be encrypted before it is signed.
2. Must sign a message. Message signing and signature validation are required for all requests and responses.
3. If data encryption is used, encrypt the message first BEFORE adding a digital signature.

For details, see the Making a request.

2. Send a request

You can send a request for example via Postman or cURL command.

3. Check the response

The response is returned usually in JSON or XML format. For details about the response, see the Response structure section.

After you receive the response, perform the following actions:

1. Validate the signature.
2. Decrypt the response if the request is encrypted.

4. Check the status code

The response data vary depending on services. However, the `result` field, which indicates the result of an API call, is always contained. If an error occurs, an error response is returned, where the `result` object indicates the error code and error message for you to troubleshoot issues. See API result code for details.

API result code

The following list shows all the resultCode returned from Beneficiary Agent. You can press Ctrl-F and then type in the resultCode to quickly find a specific result.

No.	resultCode	resultStatus	resultMessage	Action
1	SUCCESS	S	Success	/
2	COMPLIANCE_PENDING	U	The order is pending because of compliance check.	
3	TRANSFER_IN_PROCESS	U	The transfer is in process.	
4	UNKNOWN_EXCEPTION	U	The API call is failed, which is caused by unknown reasons.	
5	BALANCE_NOT_ENOUGH	F	The balance is not enough.	
6	BALANCE_NOT_EXIST	F	Balance not exist.	
7	CURRENCY_NOT_SUPPORT	F	The currency is not supported.	Check whether the currency is correct.
8	EXPIRED_REFRESH_TOKEN	F	The refresh token is expired.	
9	INVALID_AUTHCODE	F	The authorization code is invalid.	
10	INVALID_REFRESH_TOKEN	F	The refresh token is invalid.	
11	INVALID_ORDER_STATUS	F	The order status is invalid for this operation.	
12	ORDER_NOT_EXIST	F	The order does not exist.	
13	ORDER_IS_CANCELED	F	The order is canceled.	
14	ORDER_IS_CLOSED	F	The order is closed.	
15	ORDER_NOT_EXIST	F	The order does not exist.	
16	PARAM_ILLEGAL	F	Illegal parameters exist. For example, a non-numeric input, or an invalid date.	Check whether the specific parameter and value is correct.
17	PROCESS_FAIL	F	A general business failure occurred. Transfer validation failed.	Transfer validation failed. Do not retry. Change request content and then send a new request instead.
18	REPEAT_REQ_INCONSISTENT	F	Repeat request inconsistent.	
19	RISK_REJECT	F	Reject by risk control.	
20	TRANSFER_NOT_EXIST	F	Transfer does not exist.	
21	AMOUNT_EXCEED_LIMIT	F	The payment amount exceeds the user's amount limit.	
22	USER_ACCOUNT_ABNORMAL	F	User account status abnormal.	Check whether the user information is correct.
23	USER_INFO_NOT_MATCH	F	The user information does not match.	
24	USER_NO_KYC	F	This user has not completed KYC verification.	
25	USER_NOT_EXIST	F	The user does not exist.	
26	USER_STATUS_ABNORMAL	F	The user status is not normal.	

27	CANCEL_WINDOW_EXCEED	F	The agreed cancellable period is exceeded.	
28	REVERSE_WINDOW_EXCEED	F	The agreed reversal period is exceeded.	
29	WAIT_BENEFICIARY_CONFIRM	U	The order is pending on beneficiary's confirmation.	

Data dictionary

Complex data objects

This section introduces the common data objects used across all interfaces.

AdditionalBeneficiaryDetails

No.	Item	Description
1	beneficiaryReceiptMethod	**Required** TransferMethod The method that Beneficiary uses to receive the transfer. See TransferMethod for details.

AdditionalTransferDetails

No.	Item	Description
1	payer	**Required** User Payer information. See User for details.
2	beneficiary	**Required** User Beneficiary information. See User for details.
3	transferFromRegion	**Required** String Payer's country or region as defined by the two-character ISO 3166 country/region code. Length: 2.
4	transferToRegion	**Required** String Beneficiary's country or region as defined by the two-character ISO 3166 country/region code. Length: 2.
5	transferReference	**Optional** String Transfer reference. Max length: 256.
6	transferPurpose	**Optional** TransferPurposeType Transfer purpose. Enum values that are listed in TransferPurposeType.

Address

No.	Item	Description
1	region	**Required** String The two-character ISO 3166 country/region code. Length: 2.
2	state	**Optional** String State, country, or province. Max length: 64.
3	city	**Optional** String City, district, suburb, town, or village. Max length: 64.
4	address1	**Optional** String Address line 1 (such as street, PO Box, or company name). Max length: 256.
5	address2	**Optional** String Address line 2 (such as apartment, suite, unit, or building). Max length: 64.
6	zipCode	**Optional** String ZIP or postal code. Max length: 32.

Amount

No.	Item	Description
1	currency	**Required** String The three-character ISO-4217 currency code. Length: 3.
2	value	**Required** String The amount as a positive integer in the smallest currency unit (for example, if the value is 100, the amount is $1.00 when currency is USD, or ₩100 when currency is KRW). Max length: 16.

BankAccountDetail

No.	Item	Description
1	bankAccountNo	**Required** String Bank account number. Max length: 64.
2	holderAccountName	**Optional** UserName Holder account name
3	holderAccountType	**Optional** HolderAccountType Holder account type. Enum values that are listed in HolderAccountType.
4	holderAddress	**Optional** Address Holder address
5	bankName	**Optional** String Bank name. Max length: 128.
6	bankBIC	**Optional** String Bank BIC. Max length: 64.
7	routingNumber	**Optional** String Routing number code that is used to identify a specific financial institution, such as American ABA code, Australia BSB code, and British Sort code. Max length: 64.

Certificate

No.	Item	Description
1	certificateType	**Optional** CertificateType Certificate type. Enum values that are listed in CertificateType.
2	certificateNo	**Required** String Certificate number. Max length: 64.
3	holderName	**Optional** UserName Certificate holder name
4	effecticveDate	**Optional** Date Certificate effective date
5	expireDate	**Optional** Date Certificate expiry date

ParticipantQuota

No.	Item	Description
1	totalAmount	**Required** Amount Total amount
2	availableAmount	**Required** Amount Available amount
3	frozenAmount	**Optional** Amount Frozen amount

Quote

No.	Item	Description
1	quoteId	**Required** String The unique ID that is assigned by Alipay+ to identify a quote. Max length: 64.
2	quoteCurrencyPair	**Required** String Exchange rate between a pair of currencies. Two currencies are separated with a slash and use the three-letter ISO-4217 currency code. Length: 7.
3	quotePrice	**Required** Decimal The exchange rate used when a currency conversion occurs. Max length: 20.
4	quoteStartTime	**Optional** Datetime The date and time when quotePrice is effective
5	quoteExpiryTime	**Required** Datetime The date and time when quotePrice expires

Result

No.	Item	Description
1	resultCode	**Required** String Result code. Max length: 16.
2	resultStatus	**Required** ResultStatusType Result status. Valid values are: • S: Successful • F: Failed • U: Unknown
3	resultMessage	**Optional** String Result message that describes resultCode in detail. Max length: 64.

TransferMethod

No.	Item	Description
1	paymentMethodType	**Required** PaymentMethodType Payment method type. Max length: 32.
2	paymentMethodProvider	**Optional** String A unique ID that is assigned by Alipay+ to identify the payment method provider. Max length: 128.
3	walletDetail	**Optional** WalletDetail Wallet details
4	bankCardDetail	**Optional** BankCardDetail Bank card details
5	bankAccountDetail	**Optional** BankAccountDetail Bank account details
6	CashPickupDetail	**Optional** CashPickupDetail Cash pickup details

User

No.	Item	Description
1	userId	**Optional** String A unique identifier of the user. Max length: 64.
2	userName	**Optional** UserName User name
3	certificate	**Optional** Certificate Certificate information
4	birthDate	**Optional** Date Date of birth
5	nationality	**Optional** String The two-character ISO 3166 country/region code. Length: 2.
6	userEmail	**Optional** String User's email. Max length: 67.
7	userPhoneNo	**Optional** String User's contact number. Max length: 31.
8	userAddress	**Optional** Address User's address

UserName

No.	Item	Description
1	firstName	**Required** String User's first name. Max length: 64.
2	middleName	**Optional** String User's middle name. Max length: 64.
3	lastName	**Optional** String User's last name. Max length: 64.
4	fullName	**Required** String User's full name. Max length: 256.
5	nativeName	**Optional** String User's native name. Max length: 128.

WalletDetail

No.	Item	Description
1	customerId	**Required** String KKP wallet unique userId or A+ wallet unique userId. Max length: 64.
2	walletName	**Required** String The name of the wallet. Max length: 128.
3	customerName	**Required** UserName Customer name

MarketingDetails

No.	Item	Description
1	promoId	**Optional** String The promotion ID that is assigned by Alipay+. This can be used to identify the user. Max length: 256.
2	couponId	**Optional** String The coupon ID that is assigned by Alipay+. Max length: 256.
3	externalCouponInfos	**Optional** Array<ExternalCouponInfos> List of coupon information. See ExternalCouponInfos for details.
4	marketingTags	**Optional** Map<String, String> <marketingTags> Promtion marketing tags.One of the key values is *newSender*.

marketingTags

No.	Item	Description
1	newSender	**Required** String The tag indicates if the sender is a new sender for the remittance service from the payer partner to Alipay+. • false • true *True* indicates the sender is a new sender.

ExternalCouponInfos

No.	Item	Description
1	externalCouponId	**Required** String The coupon ID that is assigned by Alipay. Max length: 256.
2	externalCouponType	**Required** String <externalCouponType> The coupon type that is assigned by Alipay. Max length: 256. Check the enums of externalCouponType.
3	couponConsumeTime	**Required** Datetime The timeslot when the coupon is consumed.
4	couponClaimTime	**Optional** Datetime The timeslot when the coupon is claimed.

Enum

The enumeration values set out below are used across all interfaces.

BalanceFundType

Key	Description
BALANCE	Balance type
QUOTA	Quota type

BizSceneType

Key	Description
TRANSFER	Transfer
TUITION	tuition remittance
B2C_TRANSFER	B2C remittance
HOUSE_RENTAL	house rent remittance

QuoteBizType

Key	Description
PREFUND	The quote is used for prefunding.
C_TRANSFER	The quote is used for transfer.

FxTradeSide

Key	Description
SELL	The transaction amount uses the sell currency.
BUY	The transaction amount uses the buy currency.

CardBrandType

Key	Description
VISA	Visa Credit Card
MASTERCARD	Master Credit Card
MAESTRO	Master Debit Card
AMEX	American Express Credit Card
JCB	Japan Credit Bureau Card
DINERS	Diners Card
DISCOVER	Discover Card
CUP	China Union Pay
MIR	Russia MIR Card
ELO	Brazil ELo Card
HIPERCARD	Brazil Hipercard
TROY	Turkey's payment method

CertificateType

Key	Description
ID_CARD	Identification card
PASSPORT	Passport
DRIVING_LICENSE	Driving license
RESIDENCE_PERMIT_HK_MC	Residence Permit for Hong Kong and Macao Residents
MAINLAND_TRAVEL_PERMIT_HK_MC	Mainland Travel Permit for Hong Kong and Macao Residents
RESIDENCE_PERMIT_TAIWAN	Residence Permit for Taiwan Residents
MAINLAND_TRAVEL_PERMIT_TAIWAN	Mainland Travel Permit for Taiwan Residents

CustomerIdType

Key	Description
EMAIL	Email
USER_ID	User ID
MOBILE_NO	Mobile No.

FundDirectionType

Key	Description
C	Credit
D	Debit

HolderAccountType

Key	Description
INDIVIDUAL	Individual
COMPANY	Company

PaymentMethodType

Key	Description
WALLET	e-wallet
BANK_ACCOUNT	Bank account

ReverseReasonType

Key	Description
USER_ACCOUNT_ABNORMAL	User account abnormal.
COLLECTION_WINDOW_EXCEED	The transfer is not collected in time.
AMOUNT_EXCEED_LIMIT	Amount exceed limit.
RISK_REJECT	Risk reject.
CODE_IS_EXPIRED	The cash pickup code is expired

TransferParticipantRole

Key	Description
PAYER_AGENT	The Payer Agent
BENEFICIARY_AGENT	The Beneficiary Agent

TransferPurposeType

Key	Description
FAMILY_SUPPORT	Family support
SALARY	Salary
GOODS_PAYMENT	Good payment
SERVICE_PAYMENT	Service payment
SELF_SEND	Self sending
EDUCATION	Education
OTHER	Other reasons

TransferTransactionType

Key	Description
TRANSFER	Transfer
REVERSE	Reversal
PREFUND	Prefund
FX_TRADE	Forex transaction
WITHDRAW	Withdrawal

ExternalCouponType

Key	Description
QUOTE_DISCOUNT_COUPON	The coupon type is the quote discount
LUCKY_MONEY_COUPON	The coupon type is the lucky money promotion
FREE_FEE_COUPON	The coupon type is the fee-free

API list

The following APIs are used in this solution and API calls are initiated by Beneficiary Agent:

Interface name	RESTful Path	Interface direction
notifyTransfer	/transfer/notifyTransfer	Beneficiary Agent->Alipay+. Interface timeout configuration: 5s.

The following APIs are used in this solution and API calls are forwarded to Beneficiary Agent by Alipay+:

Interface name	RESTful Path	Description
validateTransfer	/transfer/validateTransfer	Alipay+ -> Beneficiary Agent. Interface timeout configuration: 9s.
transfer	/transfer/transfer	Alipay+ -> Beneficiary Agent. Interface timeout configuration: 9s.
inquiryTransfer	/transfer/inquiryTransfer	Alipay+ -> Beneficiary Agent. Interface timeout configuration: 6s.
cancelTransfer	/transfer/cancelTransfer	Alipay+->Beneficiary Agent. Interface timeout configuration: 6s.
notifyReverse	/transfer/notifyReverse	Alipay+ ->Beneficiary Agent. Interface timeout configuration: 5s.
inquiryBalance	/balance/inquiryBalance	Alipay+->Beneficiary Agent Interface timeout configuration: 6s.
inquiryQuote	/transfer/inquiryQuote	Alipay+->Beneficiary Agent Interface timeout configuration: 6s.

inquiryQuote

Before forex transactions, if Alipay+decides to provide the exchange rate, Alipay+ can use the **inquiryQuote** interface to obtain the exchange rate.

Structure

Request parameters

No.	Field	Description
1	quoteBizType	**Required** QuoteBizType The type of transaction that uses the quote. See QuoteBizType for details. Max length: 64.
2	sellCurrency	**Required** String The currency used by the Payer. The 3-character ISO-4217 currency code. Length: 3
3	buyCurrency	**Required** String The currency used by the Beneficiary. The 3-character ISO-4217 currency code. Length: 3
4	sellAmount	**Optional** Amount The amount in *sellCurrency*
5	buyAmount	**Optional** Amount The amount in *buyCurrency*
6	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Response parameters

No.	Field	Description
1	result	**Required** Result Request result, which contains information such as status and error codes. See Result for details.
2	foreignExchangeQuote	**Required** Quote The quote information. See Quote for details.
3	sellAmount	**Optional** Amount The amount in *sellCurrency*
4	buyAmount	**Optional** Amount The amount in *buyCurrency*
5	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Result code

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
2	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
3	PARAM_ILLEGAL	F	Illegal parameters exist. For example, a non-numeric input, or an invalid date.
~~4~~	~~REPEAT_REQ_INCONSISTENT~~	~~F~~	~~Repeated requests are inconsistent.~~
5	UNKNOWN_EXCEPTION	U	The API call is failed, which is caused by unknown reasons.

Samples

The following example assumes that the Alipay+queries the quote for a forex transaction where **10000 USD** is sold.

1. Alipay+ sends the quote inquiry request to Beneficiary Agent.

```
{
  "quoteBizType": "C_TRANSFER",
  "sellCurrency": "USD",
  "buyCurrency": "KRW",
  "sellAmount": {
    "value": "1000000",
    "currency": "USD"
  }
}
```

2. Beneficiary Agent returns the quote information to the Alipay+.

```
{
  "result": {
    "resultCode": "SUCCESS",
    "resultMessage": "success",
    "resultStatus": "S"
  },
  "foreignExchangeQuote": {
    "quoteId": "202001230901000012****",
    "quoteCurrencyPair": "USD/KRW",
    "quotePrice": "1439.65322312",
    "baseCurrency": "USD",
    "quoteUnit": "1",
    "quoteExpiryTime": "2025-06-06T12:01:01+08:00"
  },
  "sellAmount": {
    "value": "1000000",
    "currency": "USD"
  },
  "buyAmount": {
    "value": "14396532",
    "currency": "KRW"
  }
}
```


The following example assumes that the Alipay+queries the quote for a forex transaction where **14396532 KRW** is bought.

1. Alipay+ sends the quote inquiry request to Beneficiary Agent.


```
{
  "quoteBizType": "C_TRANSFER",
  "sellCurrency": "USD",
  "buyCurrency": "KRW",
  "buyAmount": {
    "value": "14396532",
    "currency": "KRW"
  }
}
```


2. Beneficiary Agent returns the quote information to the Alipay+.

```
{
  "result": {
    "resultCode": "SUCCESS",
    "resultMessage": "success",
    "resultStatus": "S"
  },
  "foreignExchangeQuote": {
    "quoteId": "202001230901000012****",
    "quoteCurrencyPair": "USD/KRW",
    "quotePrice": "1439.65322312",
    "baseCurrency": "USD",
    "quoteUnit": "1",
    "quoteExpiryTime": "2025-06-06T12:01:01+08:00"
  },
  "sellAmount": {
    "value": "1000000",
    "currency": "USD"
  },
  "buyAmount": {
    "value": "14396532",
    "currency": "KRW"
  }
}
```

validateTransfer

Alipay+ use **validateTransfer** interface to send a transfer validation request to validate information such as:

- Legality of the beneficiary account.
- Whether the Beneficiary and the beneficiary account match.
- Whether Beneficiary meets KYC requirements.
- Beneficiary collection quotas (if available).
- Legality of the payer (if available).
- Currency check of incoming and outgoing payments
- Purpose of the transfer. The valid purposes vary according to the requirements in different countries. Beneficiary Agent must complete the transfer validation and then send the result to Beneficiary Agent.

Structure

Request parameters

No.	Field	Description
1	payerAgentId	**Optional** String The unique ID that is assigned by Alipay+ to identify a Payer Agent. Max length: 64.
2	beneficiaryAgentId	**Optional** String The unique ID that is assigned by Alipay+to identify a Beneficiary Agent. Max length: 64.
3	payer	**Optional** User Payer information. See User for details.
4	beneficiary	**Optional** User Beneficiary information. See User for details.
5	instructedAmountType	**Required** String The identifier indicating the exchange rate calculation direction. To avoid reconciliation issues, both parties need to agree on the exchange rate calculation direction before cooperation. For KKP Wallet, it is recommended to use `TRANSFER_TO`. `TRANSFER_TO`: The transfer base currency is transferToAmount; `TRANSFER_FROM`: The transfer base currency is transferFromAmount;
6	transferFromAmount	**Conditional** Amount Amount paid by payer, also known as the payment amount. transferFromAmount is **Required,** when 1. instructedAmountType is `TRANSFER_FROM`; 2. transferQuote is not null**.**
7	transferToAmount	**Required** Amount Amount collected by beneficiary, also known as collection amount.
8	beneficiaryReceiptMethod	**Required** TransferMethod The method that Beneficiary uses to receive the transfer. See TransferMethod for details.
9	transferFromRegion	**Optional** String Payer's country or region as defined by the two-character ISO 3166 country/region code. Length: 2.
10	transferToRegion	**Optional** String Beneficiary's country or region as defined by the two-character ISO 3166 country/region code. Length: 2.
11	bizSceneType	**Required** BizSceneType Business scene type.
12	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Response parameters

No.	Field	Description
1	result	**Required** Result Request result, which contains information such as status and error codes. See Result for details.
2	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.
3	beneficiaryReceiptMethod	**Optional** TransferMethod The method that Beneficiary uses to receive the transfer. See TransferMethod for details.

Result code

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
3	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
4	PARAM_ILLEGAL	F	Illegal parameters exist. For example, a non-numeric input, or an invalid date.
5	UNKNOWN_EXCEPTION	U	The API call is failed, which is caused by unknown reasons.
6	USER_NOT_EXIST	F	The user does not exist.
7	CURRENCY_NOT_SUPPORT	F	The currency is not supported.
8	USER_INFO_NOT_MATCH	F	The user information does not match.
9	USER_NO_KYC	F	This user has not completed KYC verification.
10	USER_ACCOUNT_ABNORMAL	F	User account status abnormal.
11	AMOUNT_EXCEED_LIMIT	F	The amount exceeds the limit.

Rules

The following rules apply to **validateTransfer** interface.

Request conditionality

- *beneficiaryReceiptMethod*
  - *walletDetail*: Required when the value of *paymentMethodType* is `WALLET`.

Result processing logic

For different request results, different actions are to be performed. See the following list for details:

- If *result.resultStatus* is `S`, Beneficiary Agent has successfully validated the transfer information, and Alipay+can proceed with the transfer.
- If *result.resultStatus* is `F`, transfer validation fails. Alipay+can terminate the transfer.
- If *result.resultStatus* is `U`, processing of the transfer validation request might have failed, which can be caused by system or network issues. Alipay+ can retry.

Samples

The following examples assume that a Payer William Jefferson Clinton who lives in ShangHai transfer 10000 KRW to KKP

wallet user LI LEI, and

Alipay+sends the transfer information to Beneficiary Agent to verify.

1. Alipay+sends transfer validation request to Beneficiary Agent.


```
{
 "beneficiaryReceiptMethod": {
   "paymentMethodType": "WALLET",
   "walletDetail": {
     "walletId":"3456786543",
     "walletName": "KKP_WALLET",
     "customerId": "86-13721473389",
     "customerName": {
       "firstName": "LI",
       "lastName": "LEI",
       "fullName": "LI LEI"
     }
   }
 },
 "beneficiary": {
   "userName": {
     "firstName": "LI",
     "lastName": "LEI",
     "fullName": "LI LEI"
   }
 },
 "bizSceneType": "TRANSFER",
 "transferToAmount": {
   "currency": "KRW",
   "value": "10000"
 },
 "transferFromRegion": "CN",
 "transferToRegion": "KR"
}
```


2. Beneficiary Agent returns the validation result to Alipay+.

```
{
  "result": {
    "resultCode": "SUCCESS",
    "resultStatus": "S",
    "resultMessage": "success"
  },
  "beneficiaryReceiptMethod": {
    "paymentMethodType": "WALLET",
    "walletDetail": {
      "walletId":"3456786543",
      "walletName": "KKP_WALLET",
      "customerId": "86-13721473389",
      "customerName": {
        "firstName": "LI",
        "lastName": "LEI",
        "fullName": "LI LEI"
      }
    }
  }
}
```

transfer

Alipay+use **transfer** interface to initiate a transfer request. Beneficiary Agent processes transfer, including the KYC and compliance check, and then returns the acceptance result to Alipay+. The transfer result, however, is returned by notifyTransfer or inquiryTransfer.

Structure

Request parameters

No.	Field	Description	
1	payerAgentId	**Optional** String The unique ID that is assigned by Alipay+ to identify a Payer Agent. Max length: 64.	
2	beneficiaryAgentId	**Optional** String The unique ID that is assigned by Alipay+ to identify a Beneficiary Agent. Max length: 64.	
3	additionalTransferDetails	**Required** AdditionalTransferDetails Additional information about the transfer. See Data dictionary AdditionalTransferDetails for details.	
4	transferRequestId	**Required** String The unique ID that is assigned by the Alipay+to identify a transfer request. Max length: 64.	
5	instructedAmountType	**Required** String The identifier indicating the exchange rate calculation direction. To avoid reconciliation issues, both parties need to agree on the exchange rate calculation direction before cooperation. For KKP Wallet, it is recommended to use `TRANSFER_TO`. `TRANSFER_TO`: The transfer base currency is transferToAmount; `TRANSFER_FROM`: The transfer base currency is transferFromAmount;	
6	transferFromAmount	**Conditional** Amount Amount paid by payer, also known as the payment amount. transferFromAmount is **Required**, when 1. instructedAmountType is `TRANSFER_FROM`; 2. transferQuote is not null.	
7	transferToAmount	**Required** Amount Amount collected by beneficiary, also known as collection amount.	
8	payerPaymentMethod	**Optional** TransferMethod The payment method that the payer uses to make the transfer. For more information, See Data dictionary TransferMethod.	
9	beneficiaryReceiptMethod	**Required** TransferMethod The method that Beneficiary uses to receive the transfer. See Data dictionary TransferMethod for	

		transfer. See Data dictionary TransferMethod for details.	
10	bizSceneType	**Required** BizSceneType Business scene type.	
11	transferQuote	**Optional** Quote Transfer quote information. See Data dictionary Quote for details. Required for post funding mode.	
12	transferNotifyUrl	**Optional** URL URL that is used to receive the transfer result notification	
13	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.	
14	marketingDetails	**Optional** MarketingDetails Information about the user and the promotion. Required if promotions are used. See Data dictionary MarketingDetails for details.	

Response parameters

No.	Field	Description
1	result	**Required** Result Request result, which contains information such as status and error codes. See Data dictionary Result for details.
2	transferId	**Optional** String A unique ID that is assigned by the Beneficiary Agent to identify a transfer. Max length: 64.
3	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Result code

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
2	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
3	PARAM_ILLEGAL	F	Illegal parameters exist. For example, a non-numeric input, or an invalid date.
4	REPEAT_REQ_INCONSISTENT	F	Requests with the same request ID are sent but inconsistent parameter values exist.
5	UNKNOWN_EXCEPTION	U	The API call is failed, which is caused by unknown reasons.
6	USER_NOT_EXIST	F	The user does not exist.
7	USER_ACCOUNT_ABNORMAL	F	The user account status is abnormal
8	AMOUNT_EXCEED_LIMIT	F	The amount exceeds the limit.
9	RISK_REJECT	F	Reject by risk control.
10	ORDER_IS_CANCELED	F	The order is canceled.
11	ORDER_IS_CLOSED	F	The order is closed.
12	BALANCE_NOT_ENOUGH	F	The balance is not enough.
13	CURRENCY_NOT_SUPPORT	F	The currency is not supported.
14	USER_INFO_NOT_MATCH	F	The user information does not match.
15	USER_NO_KYC	F	The user has not completed KYC verification.
16	TRANSFER_IN_PROCESS	U	The transfer is in process.
17	COMPLIANCE_PENDING	U	The order is pending because of compliance check.
18	WAIT_BENEFICIARY_CONFIRM	U	The order is pending on beneficiary's confirmation.

Rules

The following rules apply to **transfer** interface.

Idempotence

transferRequestId: Beneficiary Agent can use this field for idempotent control.

- A *transferRequestId* is unique and identifies one transfer. A new *transferRequestId* indicates a new transfer. If multiple requests with the same *transferRequestId* are received, the Beneficiary Agent returns the result of the first transfer request that is successfully processed. For example, if a transfer request with *transferRequestId* `20111111111111111111111111****` has been processed successfully, when Alipay+ submits transfer requests with the same *transferRequestId*, Beneficiary Agent responds with the same result.
- If Alipay+ sends multiple requests by using the same *transferRequestId*, but the data in the subsequent requests vary from the first one, Beneficiary Agent returns `REPEAT_REQ_INCONSISTENT` for the subsequent requests.

Request conditionality

- *payerPaymentMethod*
  - *walletDetail* : Required when the value of *paymentMethodType* is `WALLET` .
- *beneficiaryReceiptMethod*

    - *walletDetail* : Required when the value of *paymentMethodType* is `WALLET`.
- *transferNotifyUrl* : Required when the transfer result is to be sent to a customized URL.

Result processing logic

For different request results, different actions are to be performed. See the following list for details:

- If *result.resultStatus* is `S`, the transfer request is successfully processed by Beneficiary Agent.
- If *result.resultStatus* is `F`, the request fails. Proceed according to the result code.
- If *result.resultStatus* is `U`, the transfer result is unknown. Alipay+ can use inquiryTransfer to query the result.

For Beneficiary Agent

- All the information of a transfer must be authentic, complete, trackable, and consistent during the transfer process. Do not tamper or hide the information.
- Beneficiary Agent must use idempotence control on all transfer requests.

Samples

The following examples assume that a Payer William Jefferson Clinton who lives in ShangHai transfer 10000 KRW to KKP wallet user LI LEI.

1. Alipay+sends the transfer request to Beneficiary Agent.


```
{
 "transferRequestId": "20111111111111111111111111****",
 "beneficiaryReceiptMethod": {
  "paymentMethodType": "WALLET",
  "walletDetail": {
   "walletId":"3456786543",
   "walletName": "KKP_WALLET",
   "customerId": "86-13721473389",
   "customerName": {
    "firstName": "LI",
    "lastName": "LEI",
    "fullName": "LI LEI"
   }
  }
 },
 "bizSceneType": "TRANSFER",
 "transferToAmount": {
  "currency": "KRW",
  "value": "10000"
 },
 "additionalTransferDetails": {
  "beneficiary": {
   "userName": {
    "firstName": "",
    "lastName": "",
    "fullName": "test wf EUR school02",
    "middleName": ""
   }
  },
  "transferPurpose": "FAMILY_SUPPORT",
  "transferFromRegion": "CN",
  "payer": {
   "userAddress": {
    "zipCode": "04072",
```

```
      "city": "SHANGHAI",
      "address2": "***-*. Hapjeong-dong",
      "address1": "Mapo-gu, SHANGHAI",
      "region": "CN"
    },
    "nationality": "CN",
    "userPhoneNo": "0101111****",
    "certificate": {
      "certificateNo": "1121111****",
      "certificateType": "PASSPORT"
    },
    "userName": {
      "firstName": "William",
      "lastName": "Clinton",
      "fullName": "William Jefferson Clinton",
      "middleName": "Jefferson"
    },
    "userId": "12111****",
    "birthDate": "20020604"
  },
  "transferToRegion": "KR"
 },
 "transferReference": "test transfer reference"
}
```


2. Beneficiary Agent returns the request result to Alipay+.


```
{
  "result": {
    "resultCode": "SUCCESS",
    "resultMessage": "success",
    "resultStatus": "S"
  },
  "transferId": "20111111111111111111111111****"
}
```

notifyTransfer

Beneficiary Agent uses **notifyTransfer** interface to send the transfer result to Alipay+.

Structure

Request parameters

No.	Item	Description
1	transferResult	**Required** Result The transfer result. See Result for details.
2	payerAgentId	**Optional** String The unique ID that is assigned by Alipay+ to identify a Payer Agent. Max length: 64.
3	beneficiaryAgentId	**Optional** String The unique ID that is assigned by Alipay+ to identify a Beneficiary Agent. Max length: 64.
4	transferRequestId	**Required** String The unique ID that is assigned by the Alipay+ to identify a transfer request. Max length: 64.
5	transferId	**Optional** String A unique ID that is assigned by the Beneficiary Agent to identify a transfer. Max length: 64.
6	transferFromAmount	**Optional** Amount Amount paid by payer, also known as payment amount. Required when *transferQuote* is passed.
7	transferToAmount	**Required** Amount Amount collected by beneficiary, also known as collection amount.
8	transferQuote	**Optional** Quote Transfer quote information. See Quote for details.
9	transferTime	The time of a successful transfer
10	additionalBeneficiary Details	**Optional** AdditionalBeneficiaryDetails Additional beneficiary details
11	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Response parameters

No.	Field	Description
1	result	**Required** Result The request result, which contains information related to the request result, such as status and error codes. See Result for details.
2	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Result code

Request:

transferResult

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
2	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
3	USER_NOT_EXIST	F	The user does not exist.
4	USER_ACCOUNT_ABNORMAL	F	User account status abnormal.
5	AMOUNT_EXCEED_LIMIT	F	The amount exceeds the limit.
6	RISK_REJECT	F	Reject by risk control.
7	ORDER_IS_CANCELED	F	The order is canceled.
8	ORDER_IS_CLOSED	F	The order is closed.
9	BALANCE_NOT_ENOUGH	F	The balance is not enough.
10	USER_INFO_NOT_MATCH	F	The user information does not match.
11	USER_NO_KYC	F	The user has not completed KYC verification.
12	CURRENCY_NOT_SUPPORT	F	The currency is not supported.

Response:

result

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
2	INVALID_AUTHCODE	F	The authorization code is invalid.
3	INVALID_REFRESH_TOKEN	F	The refresh token is invalid.
4	EXPIRED_REFRESH_TOKEN	F	The refresh token is expired.
5	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
6	PARAM_ILLEGAL	F	Illegal parameters exist. For example, a non-numeric input, or an invalid date.
7	UNKNOWN_EXCEPTION	U	The API call is failed, which is caused by unknown reasons.

Rules

The following rules apply to Transfer Result Notification interface.

Retry strategy

If Alipay+ does not response that the notification is successfully received, Beneficiary Agent will retry the result notification.

- Times: 7.
- Intervals: 2m, 10m, 10m, 1h, 2h, 6h, and 15h.

Request conditionality

- *tranferFromAmount*: Required if *transferFromAmount* is passed in Transfer interface and *transferResult.resultStatus* is

`S`.

- *transferToAmount*: Required if *transfserToAmount* is passed in Transfer interface and *tranferResult.resultStatus* is `S`.
- *transferQuote*: Required if *transferQuote* is passed in Transfer interface and *transferResult.resultStatus* is `S`.
- *transferTime*: Required if *transferResult.resultStatus* is `S`.
- *transferRequestId*: Required if *transferResult.resultStatus* is `S`.

Result processing logic

For different request results, different actions are to be performed. See the following list for details:

- If *result.resultStatus* is `S`, transfer result notification is processed successfully.
- If *result.resultStatus* is `F`, the transfer notification failed. Beneficiary Agent can monitor the situation and intervene if necessary.
- If *result.resultStatus* is `U`, transfer result notification might have failed because of system or network errors. Beneficiary Agent must retry. Suggested retry rules:
  - Use incremental retry interval, such as 5 seconds, 30 seconds, 1 minute, and 5 minutes.
  - The retry spans more than one day.

For Beneficiary Agent

If all the retrials are sent and Alipay+ does not response that the notification is successfully received, Beneficiary Agent must contact Alipay+ Technical Support.

Samples

The following examples assume that a Hong Kong e-wallet user William Jefferson Clinton has successfully transferred 600 PHP to a Philippine e-wallet user FREDY MANALO SANTOS.

1. Beneficiary sends the transfer result to Alipay+.


```
{
  "transferResult": {
    "resultStatus": "S",
    "resultCode": "SUCCESS",
    "resultMessage": "success"
  },
  "transferRequestId":"20111111111111111111111111****",
  "transferId":"20111111111111111111111111****",
  "transferFromAmount": {
    "currency": "HKD",
    "value": "8990"
  },
  "transferToAmount": {
    "currency": "PHP",
    "value": "60000"
  },
  "transferQuote": {
    "quoteId": "ALI1NWPAQ296****"
  },
  "transferTime": "2019-01-01T12:19:56+08:00"
}
```


2. Alipay+ processes the result and responses to Beneficiary Agent.

```
{
  "result":{
    "resultStatus":"S",
    "resultCode":"SUCCESS",
    "resultMessage":"success"
  }
}
```

inquiryTransfer

Alipay+ use **inquiryTransfer** interface to query the result of a transfer if the transfer result is not returned for a long period of time.

Structure

Request parameters

No.	Field	Description
1	transferRequestId	**Required** String The unique ID that is assigned by the transfer initiator to identify a transfer request. Max length: 64.
2	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Response parameters

No.	Field	Description
1	result	**Required** Result Request result, which contains information such as status and error codes. See Result for details.
2	transferResult	**Optional** Result The transfer result. See Result for details.
3	payerAgentId	**Optional** String The unique ID that is assigned by Alipay+ to identify a Payer Agent. Max length: 64.
4	beneficiaryAgentId	**Optional** String The unique ID that is assigned by Alipay+ to identify a Beneficiary Agent. Max length: 64.
5	transferId	**Optional** String A unique ID that is assigned by the Beneficiary Agent to identify a transfer. Max length: 64.
6	transferFromAmount	**Optional** Amount Amount paid by payer, also known as payment amount. Required when *transferQuote* is passed.
7	transferToAmount	**Required** Amount Amount collected by beneficiary, also known as collection amount.
9	transferTime	**Optional** Datetime The time of a successful transfer
10	transferQuote	**Optional** Quote Transfer quote information. See Quote for details.
11	additionalBeneficiaryDetails	**Optional** AdditionalBeneficiaryDetails Additional beneficiary details
12	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Reuslt code

result

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
2	INVALID_AUTHCODE	F	The authorization code is invalid.
3	INVALID_REFRESH_TOKEN	F	The refresh token is invalid.
4	EXPIRED_REFRESH_TOKEN	F	The refresh token is expired.
5	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
6	PARAM_ILLEGAL	F	Illegal parameters exist. For example, a non-numeric input, or an invalid date.
7	UNKNOWN_EXCEPTION	U	The API call is failed, which is caused by unknown reasons.
8	ORDER_NOT_EXIST	F	The order does not exist.

transferResult

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
2	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
3	USER_NOT_EXIST	F	The user does not exist.
4	USER_ACCOUNT_ABNORMAL	F	User account status abnormal.
5	AMOUNT_EXCEED_LIMIT	F	The amount exceeds the limit.
6	RISK_REJECT	F	Reject by risk control.
7	TRANSFER_IN_PROCESS	U	The transfer is in process.
8	COMPLIANCE_PENDING	U	The order is pending because of compliance check.
9	WAIT_BENEFICIARY_CONFIRM	U	The order is pending on beneficiary's confirmation.
10	ORDER_IS_CANCELED	F	The order is cancelled.
11	ORDER_IS_CLOSED	F	The order is closed.
12	BALANCE_NOT_ENOUGH	F	The balance is not enough.
13	USER_INFO_NOT_MATCH	F	The user information does not match.
14	USER_NO_KYC	F	The user has not completed KYC verification.
15	CURRENCY_NOT_SUPPORT	F	The currency is not supported.

Rules

The following rules apply to **inquiryTransfer** interface.

Response conditionality

- *transferResult*: Required if *result.resultStatus* is `S`.
- *transferId*: Required if *transferResult.resultStatus* is `S`.

- *tranfserFromAmount*: Required if *tranfserFromAmount* is passed in **transfer** interface and *transferResult.resultStatus* is `S`.
- *transferToAmount*: Required if *transferToAmount* is passed in **transfer** interface and *tranferResult.resultStatus* is `S`.
- *transferQuote*: Required if *transferQuote* is passed in **transfer** interface and *transferResult.resultStatus* is `S`.
- *transferTime*: Required if *transferResult.resultStatus* is `S`.

Result processing logic

For different request results, different actions are to be performed. See the following list for details:

- If *result.resultStatus* is `S`, the transfer result inquiry is successful,
  - If *transferResult.resultStatus* is `S`, the transfer is successful and the funds are sent to the Beneficiary.
  - If *transferResult.resultStatus* is `F`, the transfer fails. Proceed according to the error code. Beneficiary Agent might have to return the funds to the prefund account.
  - If *transferResult.resultStatus* is `U`, the transfer result is unknown. Alipay+ can retry the inquiry.

- If *result.resultStatus* is `F`, the transfer result inquiry failed. Proceed according to the error code. For example, if the value of *resultCode* is `ORDER_NOT_EXIST`, it means that the transfer is not yet accepted and can be treated as transfer failure. For other result codes (failure reasons), human intervention is recommended.

Samples

The following examples assume that a Hong Kong e-wallet user has successfully transferred 600.00 PHP to a Philippine e-wallet user by paying 89.90 HKD. The Hong Kong e-wallet then queries the transfer result.

1. Alipay+ sends the transfer result inquiry request to Beneficiary Agent.

```
{
  "transferRequestId":"2020032600000****"
}
```

2. Beneficiary Agent returns the transfer result to Alipay+.

```
{
  "result": {
    "resultStatus": "S",
    "resultCode": "SUCCESS",
    "resultMessage": "success"
  },
  "transferResult": {
    "resultStatus": "S",
    "resultCode": "SUCCESS",
    "resultMessage": "success"
  },
  "transferId": "20201209093209302930923****",
  "transferFromAmount": {
    "currency": "HKD",
    "value": "8990"
  },
  "transferToAmount": {
    "currency": "PHP",
    "value": "60000"
  },
  "transferQuote": {
    "quoteId": "CDEI1NWPAQ196****"
  },
  "transferTime": "2019-01-01T12:01:01+08:00"
}
```

cancelTransfer

Alipay+ uses **cancelTransfer** interface to cancel a transfer.transfers in `COMPLIANCE_PENDING` or `WAIT_BENEFICIARY_CONFIRM` status can be canceled. After Alipay+requests a transfer cancelation, Beneficiary Agent receives and processes the request and then returns the cancelation result to Alipay+.

Structure

Request parameters

No.	Item	Description
1	transferRequestId	**Required** String The unique ID that is assigned by the transfer initiator to identify a transfer request. Max length: 64.
2	cancelReason	**Optional** String Cancelation reason. Max length: 1024.
3	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Response parameters

No.	Field	Description
1	result	**Required** Result Request result, which contains information such as status and error codes. See Result for details.
2	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Result code

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
2	UNKNOWN_EXCEPTION	U	The API call is failed, which is caused by unknown reasons.
3	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
4	PARAM_ILLEGAL	F	Illegal parameters exist. For example, a non-numeric input, or an invalid date.
5	ORDER_NOT_EXIST	F	The order does not exist.
6	INVALID_ORDER_STATUS	F	The order status is invalid for this operation.
7	CANCEL_WINDOW_EXCEED	F	The agreed cancellable period is exceeded.

Rules

The following rules apply to **cancelTransfer** interface.

Idempotence

transferRequestId: Beneficiary Agent can uses this field for idempotent control.

- If multiple requests with the same *transferRequestId* are received, Beneficiary Agent returns the result of the first cancelation request that is successfully processed. For example, if a cancelation request with *transferRequestId* `2020032600000****` has been processed successfully, when Alipay+submits cancelation requests with the same *transferRequestId*, Beneficiary Agent responds with the same result.
- If Alipay+sends multiple requests by using the same *transferRequestId*, but the data in the subsequent requests vary from the first one, Beneficiary Agent returns `REPEAT_REQ_INCONSISTENT` for the subsequent requests.

Request conditionality

- *transferRequestId*: Beneficiary Agent uses this field for idempotent control. If multiple requests with the same *transferRequestId* are received, Beneficiary Agent returns the result of the first transfer request that is successfully processed. For example, if a transfer request with *transferRequestId* `2020032600000****` has been processed successfully, when Alipay+ submits transfer requests with the same *transferRequestId*, Beneficiary Agent responds success.
- *cancelReason*: The cancelation reason is provided by the Alipay+.

Result processing logic

For different request results, different actions are to be performed. See the following list for details:

If *result.resultStatus* is `S`, the cancelation request is successfully processed by Beneficiairy Agent.

If *result.resultStatus* is `F`, the cancelation request fails. Alipay+can proceed according to the result code.

If *result.resultStatus* is `U`, the cancelation result is unknown. Alipay+can retry the cancelation.

Samples

1. Alipay+ sends the cancelation request to Beneficiary Agent.

```
{
  "transferRequestId": "2020032600000****",
  "cancelReason": "per user requested"
}
```

2. Beneficiary Agent returns the request result toAlipay+.

```
{
  "result": {
    "resultCode": "SUCCESS",
    "resultMessage": "success",
    "resultStatus": "S"
  }
}
```

notifyReverse

Alipay+ uses **notifyReverse** interface to send the transfer reversal result to Beneficiary Agent.

Structure

Request parameters

No.	Field	Description
1	transferRequestId	**Required** String The unique ID that is assigned by the transfer initiator to identify a transfer request. Max length: 64.
2	reverseRequestId	**Required** String The unique ID that is generated by the Beneficiary Agent to identify a reversal request. Max length: 64.
3	reverseId	**Required** String A unique ID that is assigned by Alipay+ to identify a reversal. Max length: 64.
4	reverseResult	**Optional** Result The reversal result. See Result for details.
5	reverseTime	**Optional** Datetime The time of a successful reversal
6	reverseReason	**Optional** ReverseReasonType The reversal reason.
7	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Response parameters

No.	Field	Description
1	result	**Required** Result Request result, which contains information such as status and error codes. See Result for details.
2	passThroughInfo	**Optional** String Only used for pass-through. In JSON map format. Max length: 2048.

Reuslt code

Request

Reverse Result

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
2	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
3	BALANCE_NOT_ENOUGH	F	The Alipay+ Balance is not enough

Response

Result

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
2	INVALID_AUTHCODE	F	The authorization code is invalid.
3	INVALID_REFRESH_TOKEN	F	The refresh token is invalid.
4	EXPIRED_REFRESH_TOKEN	F	The refresh token is expired.
5	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
6	PARAM_ILLEGAL	F	Illegal parameters exist. For example, a non-numeric input, or an invalid date.
7	UNKNOWN_EXCEPTION	U	The API call is failed, which is caused by unknown reasons.

Rules

The following rules apply to **notifyReverse** interface.

Result processing logic

For different request results, different actions are to be performed. See the following list for details:

reverse.resultStatus

- If *reverse.resultStatus* is `S`, the reversal is successful.
- If *reverse.resultStatus* is `F`, the reversal fails. Beneficiary Agent can determine whether to resend the reversal request according to the specific error code.

result.resultStatus

If *result.resultStatus* is `S`, the Beneficiary Agent system handles the reversal result notification successfully.

If *result.resultStatus* is `F` or `U`, the Beneficiary system fails to handle the reversal result notification. Alipay+ re-sends the reversal result notification.

- Total retrials: 7.
- Intervals: 2m, 10m, 10m, 1h, 2h, 6h, and 15h.

If all the retrials are sent and the notification is still not successfully received, Alipay+ Technical Support will contact the Beneficiary Agent for help.

Samples

The following examples assume that a transfer is successfully reversed. Alipay+ then sends the reversal result to Beneficiary Agent.

1. Alipay+ sends reversal result notification to Beneficiary Agent.

```
{
  "reverseResult": {
    "resultStatus": "S",
    "resultCode": "SUCCESS",
    "resultMessage": "success"
  },
  "transferRequestId": "2020032600000****",
  "reverseRequestId": "20211111111111111111111111111****",
  "reverseId": "20211111111111111111111111111****",
  "reverseReason": "AMOUNT_EXCEED_LIMIT",
  "reverseTime": "2019-01-01T12:19:56+08:00"
}
```


2. Beneficiary Agent response to Alipay+.


```
{
  "result": {
    "resultStatus": "S",
    "resultCode": "SUCCESS",
    "resultMessage": "success"
  }
}
```

inquiryBalance

Alipay+ uses **inquiryBalance** interface to query the current AAlipay+'s Balance.

Structure

Request parameters

No.	Field	Description
1	currency	**Optional** String The currency of the Alipay+ Balance. A three-character ISO-4217 currency code. Length: 3. If this field is empty, balance information of all the currencies will be displayed. If this field is not empty, balance information of the currency passed will be displayed.
2	accountAlias	**Optional** String max(64) The alias of the Alipay+'s balance account
3	passThroughInfo	**Optional** String max(2048) Only used for pass-through. In JSON map format.

Response parameters

No.	Item	Description
1	result	**Required** Result Request result, which contains information such as status and error codes. See Result for details.
2	balances	**Optional** Array<Amount> List of balances information.
3	quotas	**Optional** Array<ParticipantQuota> List of quota information.
4	passThroughInfo	**Optional** String max(2048) Only used for pass-through. In JSON map format.

Result code

No.	resultCode	resultStatus	resultMessage
1	SUCCESS	S	Success
2	INVALID_AUTHCODE	F	The authorization code is invalid.
3	INVALID_REFRESH_TOKEN	F	The refresh token is invalid.
4	EXPIRED_REFRESH_TOKEN	F	The refresh token is expired.
5	PROCESS_FAIL	F	A general business failure occurred. Do not retry.
6	PARAM_ILLEGAL	F	Illegal parameters exist. For example, a non-numeric input, or an invalid date.
7	UNKNOWN_EXCEPTION	U	The API call is failed, which is caused by unknown reasons.
8	BALANCE_NOT_EXIST	F	Balance does not exist.

Rules

The following rules apply to **inquiryBalance** interface.

Request conditionality

accountAlias: If this field is not empty, Beneficiary Agent uses the passed-in value to query the balance information for Alipay+.

Response conditionality

If the Alipay+has transfer quotas, which is allocated by Beneficiary Agent, *quotas* is returned.

Result processing logic

For different request results, different actions are to be performed. See the following list for details:

- If *result.resultStatus* is `S`, Alipay+has successfully queried the Balance information.
- If *result.resultStatus* is `F`, the Alipay+fails to query the Balance information. Proceed according to the error code.
- If *result.resultStatus* is `U`, failure might have occurred because of system or network issues. Alipay+ can retry.

Samples

The following examples assume that Alipay+ queries the Balance.

1. The Alipay+ sends a balance inquiry request to Beneficiary Agent.

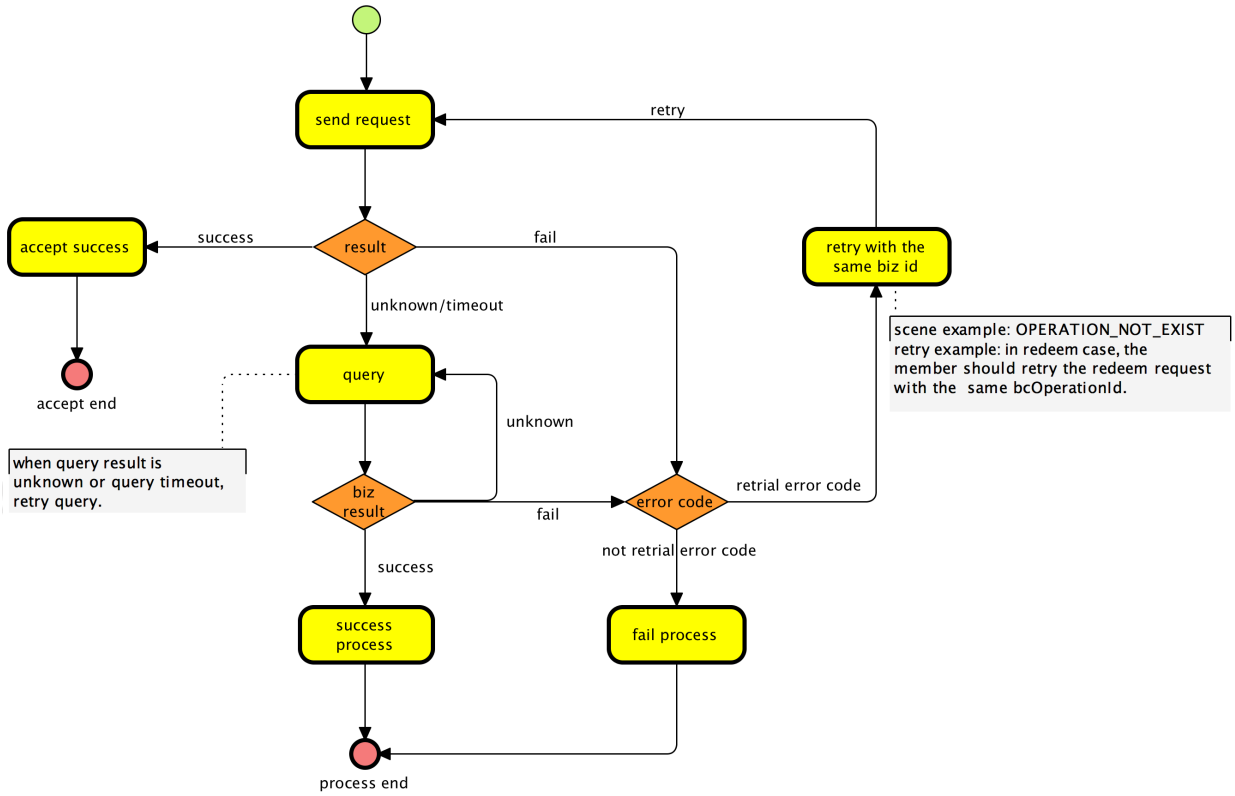
```
{}
```

2. Beneficiary Agent returns the Alipay+'s Balance information.

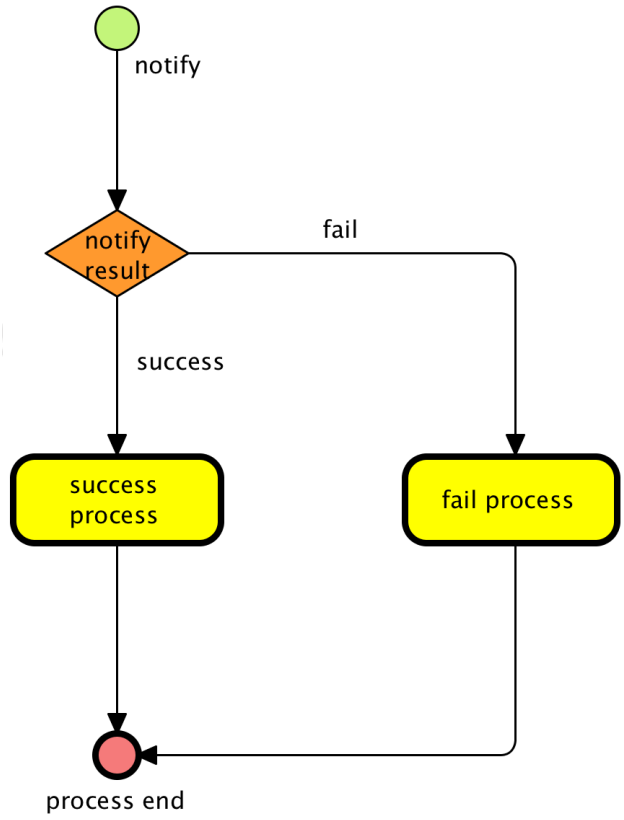
```
{
  "balances": [
    {
      "value": "10000000",
      "currency": "USD"
    },
    {
      "value": "10000000",
      "currency": "HKD"
    }
  ],
  "quotas": [
    {
      "totalAmount": {
        "value": "10000000",
        "currency": "USD"
      },
      "availableAmount": {
        "value": "10000000",
        "currency": "USD"
      }
    }
  ],
  "result": {
    "resultCode": "SUCCESS",
    "resultStatus": "S",
    "resultMessage": "success"
  }
}
```

F and U Suggestion

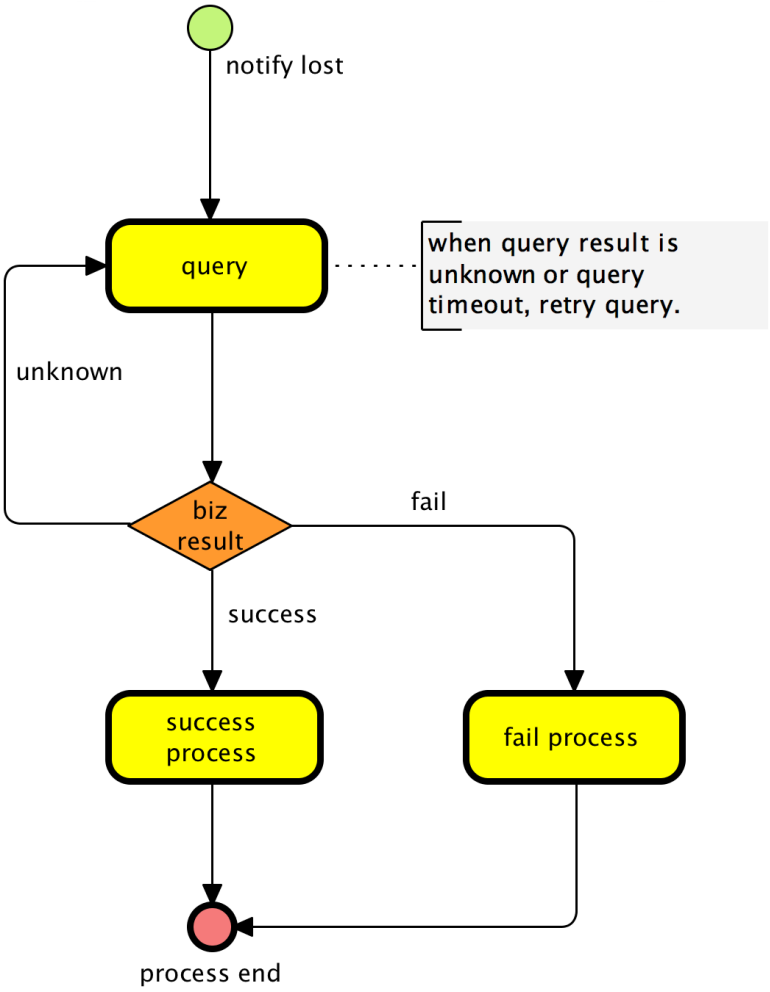
1. Main Process Flow



2. Notify Process Flow



3. Query Process Flow


notify lost
query
when query result is unknown or query timeout, retry query.
unknown
biz result
fail
success
success process
fail process
process end

Technical specification for messaging interface

1.Scope

Technical specification for messaging interface presents general information (such as message structure, message fields, and message transmission) of online message between Participants and Platform.

This specification is applicable to all participants.

2. Terms and acronyms

The following terms and definitions are used in this specification:

Decryption

Decryption is the reverse process of encryption.

Digital Signature

An irreversible cipher (signature) that is generated by the information sender, which is sent to the receiver together with the original information.

Encryption

The process of changing plaintext into ciphertext that is unreadable.

An entity that holds the key or certificate.

Issuer

An entity that generates the key or certificate.

key

A key is a unique, generated electronic string of bits used for encrypting, decrypting, creating digital signatures, or validating digital signatures.

Participant

An institution that joins Alipay Scheme and complies with the rights and obligations set out in the Scheme.

Private Key

The secret part of an asymmetric key pair that is used to digitally sign or decrypt data.

Public Key

The public part of an asymmetric key pair that is typically used to verify signatures or encrypt data.

Signature Validation

The information receiver verifies whether the original information and signature received are matched.

3. Message structure

A message refers to the request or response message. The following figures illustrate the request and response structures separately:

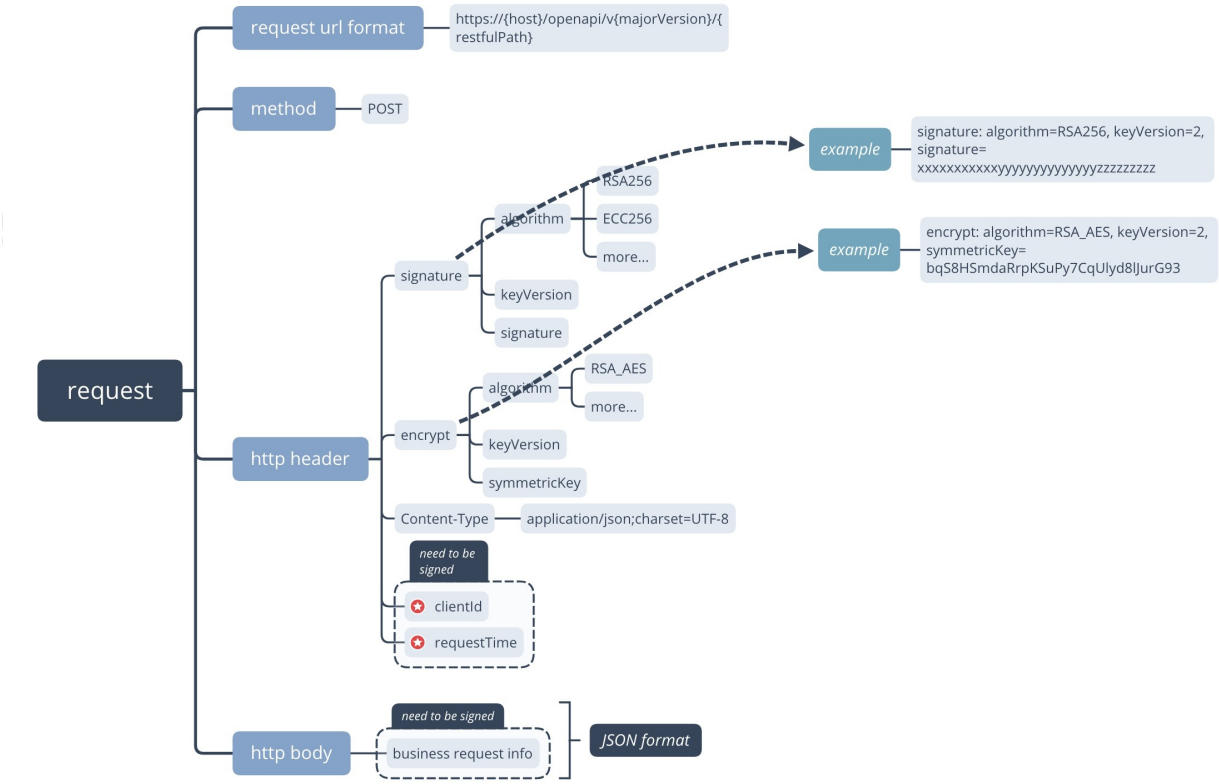


Figure 1. Request structure

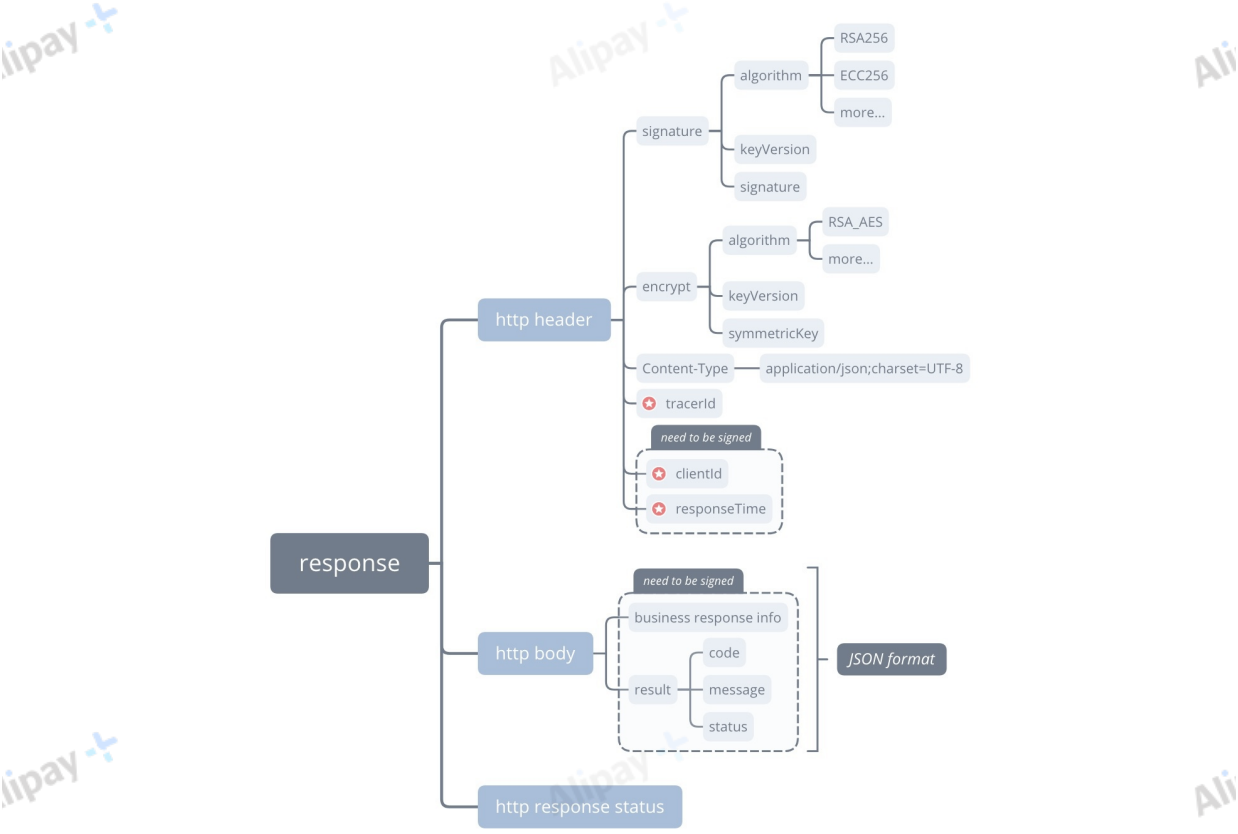


Figure 2. Response structure

The following curl example depicts the general structure of a message:

```
curl -X POST \
  https://open.alipayconnect.com/openapi/v1/payments/payment \
  -H 'Content-Type: application/json; charset=UTF-8' \
  -H 'Client-Id: 5X67656YHNHGF' \
  -H 'Request-Time: 2019-04-04T12:08:56+05:30' \
  -H 'Signature: algorithm=RSA256, keyVersion=2,
signature=KEhXthj4bJ801Hqw8kaLvEKc0Rii8KsNUazw7kZgjxyGSPuOZ48058UVJUkkR21iD9JkHBGRrWiHPae8ZRPuBag
h2H3qu7fxY5GxVDWayJUhUYkr9m%2FOW4UQVmXaQ9yn%2Fw2dCtzwAW0htPHYrKMyrTpMk%2BfDDmRflA%2FAMJh
Q71yeyhufIA2PCJV8%2FCMOa46303A0WHhH0YPJ9%2FI0UeLVMWlJ1XcBo3JrbRFvcowQwt0lP1XkoPmSLGpBevDE8%
2FQ9WnxjPNDfrHnKgV2fp0hpMKVXNM%2BrLHNyMv3MkHg9iTMOD%2FFYDAwSd%2B6%2FEOFo9UbdlKcmodJwjKlQox
ZZIzmF8w%3D%3D' \
  -d '{
 "order": "",
 "paymentId":"20181129190741010007000000011111",
 "paymentAmount":{
    "value":"100",
    "currency":"JPY"
 },
 "paymentMethod":{
      "paymentMethodType": "SAMPLE_WALLET",
      "paymentMethodSubType": "PAYMENT_CODE",
      "paymentMethodId": "2800160000000001"
  },
 "payToAmount":{
    "value":"100",
    "currency":"JPY"
 },
 "paymentFactor": {
     "isInStorePayment": "true"
 }
}'
```

3.1 Request method

POST method is used to make an HTTP request.

3.2 Request URL

The request URL is:

`https://{host}/openapi/v{majorVersion}/{restfulPath}`

where,

`host` is the standard domain name assigned by Platform.

`MajorVersion` is the main version number of the interface.

`restfulPath` is the path to the interface.

An interface can be uniquely identified by `restfulPath` and `majorVersion`.

3.3 Request header

The request header mainly contains fields such as `client-id`, `signature`, `encrypt`, `content-type`, and `request-time`.

3.3.1 Client-Id

Required. `client-id` is used to identify a client, and is associated with the keys that are used for signature and encryption.

3.3.2 Signature

Required. `signature` contains key-value pairs that are separated by comma (,). Each key-value pair is an equation, which is a key joined with its value with an equal sign (=).

The following keys can be configured:

- `algorithm`: Specifies the digital signature algorithm that is used to generate the signature. The value is not case-sensitive. RSA256 and ECC224 are supported, and RSA256 by default.
- `keyVersion`: Specifies the key version that is used to generate or validate the signature. By default, the value is the latest version of the key associated with `client-Id`.
- `signature`: Contains the signature value of the request.

Example:

```
signature: algorithm=RSA256, keyVersion=2,
signature=KEhXthj4bJ801Hqw8kaLvEKc0Rii8KsNUazw7kZgjxyGSPuOZ48058UVJUkkR21iD9JkHBGRrWiHPae8ZRPuBag
h2H3qu7fxY5GxVDWayJUhUYkr9m%2FOW4UQVmXaQ9yn%2Fw2dCtzwAW0htPHYrKMyrTpMk%2BfDDmRflA%2FAMJh
Q71yeyhufIA2PCJV8%2FCMOa46303A0WHhH0YPJ9%2FI0UeLVMWlJ1XcBo3JrbRFvcowQwt0lP1XkoPmSLGpBevDE8%
2FQ9WnxjPNDfrHnKgV2fp0hpMKVXNM%2BrLHNyMv3MkHg9iTMOD%2FFYDAwSd%2B6%2FEOFo9UbdlKcmodJwjKlQox
ZZIzmF8w%3D%3D
```

3.3.3 Encrypt

Required when a message need to be encrypted. encrypt contains key-value pairs that are separated by comma (,). Each key-value pair is an equation, which is a key joined with its value with an equal sign (=).

The following keys can be configured:

- `algorithm`: Specifies the symmetric key algorithm that is used to encrypt message. The value is not case-sensitive, and currently only RSA_AES is supported.
- `keyVersion`: Specifies the symmetric key version that is used to encrypt message. By default, the value is the latest version of the key associated with `clientId`.
- `symmetricKey`: Contains the encrypted symmetric key.

Example:

```
encrypt: algorithm=RSA, keyVersion=2, symmetricKey=bqS8HSmdaRrpKSuPy7CqUlyd8lJurG93
```

3.3.4 Content-Type

Optional. `content-Type` indicates the media type of the body of the request, as defined by RFC2616. In which, `charset` is used for generating/validating signature and encrypting/decrypting content.

Example：

```
content-type: application/json; charset=UTF-8
```

3.3.5 Request-Time

Required. request-Time specifies the time when the request is sent, as defined by RFC3339.

Example:

```
request-time: 2019-04-04T12:08:56+05:30
```

3.4 Request body

The request body contains the detailed request information in a JSON format. Fields enclosed in the request body section vary depending on services. For more information, see instructions of the specific message interface.

3.5 Response header

Response header carries additional information about the response, such as the signature and the encryption.

For more information about how to create the signature or encrypt the message, see Technical specification for secure data transmission.

The Response header mainly contains fields such as `client-Id`, `signature`, `encrypt`, `content-type`, and `response-time`.

3.5.1 Client-Id

Required. `client-id` is used to identify a client, and is associated with the keys that are used for signature and encryption.

3.5.2 Signature

Required. `signature` contains key-value pairs that are separated by comma (,). Each key-value pair is an equation, which is a key joined with its value with an equal sign (=).

The following keys can be configured:

- `algorithm`: Specifies the digital signature algorithm that is used to generate the signature. The value is not case-sensitive. RSA256 and ECC224 are supported, and RSA256 by default.
- `keyVersion`: Specifies the key version that is used to generate or validate the signature. By default, the value is the latest version of the key associated with `client-id`.
- `signature`: Contains the signature value of the request.

Example:

```
signature: algorithm=RSA256, keyVersion=2,
signature=KEhXthj4bJ801Hqw8kaLvEKc0Rii8KsNUazw7kZgjxyGSPuOZ48058UVJUkkR21iD9JkHBGRrWiHPae8ZRPuBag
h2H3qu7fxY5GxVDWayJUhUYkr9m%2FOW4UQVmXaQ9yn%2Fw2dCtzwAW0htPHYrKMyrTpMk%2BfDDmRflA%2FAMJh
Q71yeyhufIA2PCJV8%2FCMOa46303A0WHhH0YPJ9%2FI0UeLVMWlJ1XcBo3JrbRFvcowQwt0lP1XkoPmSLGpBevDE8%
2FQ9WnxjPNDfrHnKgV2fp0hpMKVXNM%2BrLHNyMv3MkHg9iTMOD%2FFYDAwSd%2B6%2FEOFo9UbdlKcmodJwjKlQox
ZZIzmF8w%3D%3D
```

3.5.3 Encrypt

Required when a message need to be encrypted. encrypt contains key-value pairs that are separated by comma (,). Each key-value pair is an equation, which is a key joined with its value with an equal sign (=).

The following keys can be configured:

- `algorithm`: Specifies the symmetric key algorithm that is used to encrypt message. The value is not case-sensitive, and currently only RSA_AES is supported.
- `keyVersion`: Specifies the symmetric key version that is used to encrypt message. By default, the value is the latest version of the key associated with `clientId`.
- `symmetricKey`: Contains the encrypted symmetric key.

Example:

```
encrypt: algorithm=RSA, keyVersion=2, symmetricKey=bqS8HSmdaRrpKSuPy7CqUlyd8lJurG93
```

3.5.4 Content-Type

Optional. `content-type` indicates the media type of the body of the request, as defined by RFC2616. In which, `charset` is used for generating/validating signature and encrypting/decrypting content.

Example:

```
content-Type: application/json; charset=UTF-8
```

3.5.5 Response-Time

Required. request-Time specifies the time when the request is sent, as defined by RFC3339.

Example:

```
response-time: 2019-04-04T12:08:56+05:30
```

3.6 Response body

Response body contains the information responding to the Client. Fields in this section vary depending on services. However, the `result` parameter, which indicates the result of an interface call, is always contained.

4. Message fields

Read the following chapter for general information of message fields, such as data type, common data structure, and processing rules of special characters.

4.1 Data type

This section describes the data types supported by Platform.

Data type	Description
String	A sequence of bytes.
Email	Email format. Platform conducts email format validation for this type.
URL	URL format. Platform conducts URL format validation for this type.
Datetime	Date time as defined by RFC3339. For example, 2020-01-01T23:59:59+08:00
Date	A three-part value (yyyyMMdd) designating a time point in time.
Boolean	The `boolean` value can be either `true` or `false`.
Integer	A numeric value without a decimal.
Decimal	A numeric value with a decimal.
Array	A homogeneous data structure (elements have same data type) that stores a sequence of consecutively numbered objects--allocated in contiguous memory.

4.2 Common data structure

The following sections introduce the common data structures that are used in messages.

4.2.1 Result

Field name	Data type	Description	Sample
resultCode	String	Result code	SUCCESS
resultMessage	String	Result message	success
resultStatus	String	Result status. Valid values: • S: Request is successfully processed. • U: Request result is unknown. Resend the request to try again. • F: Request failed. No retry needed. • A: Request is successfully accepted.	S

The following table lists the common `resultCode` that might be returned. For more information about the service-specific resultCode, see the specific interface doc.

resultCode	resultMessage	resultStatus
SUCCESS	success	S
PROCESS_FAIL	General business failure. No retry needed.	F
UNKNOWN_EXCEPTION	API call failed because of unknown reason.	U
PARAM_ILLEGAL	Illegal parameters. For example, non-numeric input, invalid date.	F
SIGNATURE_INVALID	Signature is invalid.	F
KEY_NOT_FOUND	Key is not found.	F
API_INVALID	API is not defined.	F
ACCESS_DENIED	Access denied.	F
CLIENT_INVALID	Invalid client.	F
REQUEST_TRAFFIC_EXCEED_LIMIT	Request traffic exceeds limit.	F
METHOD_NOT_SUPPORTED	The request method is not supported.	F
MEDIA_TYPE_NOT_ACCEPTABLE	The content-type is not acceptable.	F

4.2.2 Amount

Field name	Data type	Description	Sample
currency	String	3-letter currency code as defined in ISO 4217.	USD
value	Integer	Amount. Provide the value in the smallest currency unit. For example, when currency is USD, the unit of value is cents; when currency is JPY, the unit is yen.	1000

4.2.3 Country/Region

Each country or area is shown in one region only.

The region value is a two-letter code (alpha-2) as defined in ISO 3116, such as JP and CN.

4.2.5 MCC

The Merchant Category Code (MCC) is a four-digit number assigned by Platform to identify a merchant's primary business. Based on ISO 18245, MCC list contains custom codes that are added to meet market demands and industry-specific requirements. See MCC list for more information.

4.3 Processing rules of special characters

The following rules are about how to process special characters enclosed in the message.

4.3.1 base64

For byte data, such as the signature and the encrypted content, encode the data with the base64 algorithm before transmitting.

4.3.2 urlencode

For URL data, perform URL encoding first before transmitting. For example:

Original URL:

`https://www.merchant.com/authorizationResult`

Converted URL:

`https%3A%2F%2Fwww.merchant.com%2FauthorizationResult`

5. Message transmission

See Technical specification for secure data transmission for details.

6. Interface version

Interface version information can be found in the request URL, and the format is `v\d+`.

The version is incremented when **incompatible** changes are made. Backwards compatible upgrades might include the following items:

- New optional fields are added in a request
- New fields are added in a response
- New values are added to enumeration fields
- New resultCodes are added in a request or response
- The field length is increased
- The order of fields in a request or response is changed

Participants must be able to correctly handle backwards compatible upgrades.

Technical specification for secure data transmission

1. Scope

Technical specification for secure data transmission provides guidelines and methods to transfer data between Participant and Platform.

This specification is applicable to all participants.

2. Terms and definitions

The following terms and definitions are used in this specification:

Key

A key is a set of bits that determines the functional output of a cryptographic algorithm, specifies the transformation between plaintext and ciphertext.

Public Key

The public part of an asymmetric key pair that is typically used to validate signatures or encrypt data.

Private Key

The secret part of an asymmetric key pair that is used to digitally sign or decrypt data.

Signature

See also "Digital Signature". An irreversible cipher (signature) that is generated by the information sender, which is sent to the receiver together with the original information.

Validation

The information receiver validates whether the original information and signature received are matched.

3. Algorithm description

The following section introduces the signature algorithm and encryption algorithms that are used for data transmission.

3.1 Signature algorithm

RSA256 is used for creating or validating signatures.

3.1.1 RSA256

Signature algorithm sha256withrsa is used. When creating a signature, calculate a sha256 digest first, and then encrypt the digest by using RSA algorithm.

Recommended RSA key size: 2048 bits.

3.2 Encryption algorithm

Encryption algorithms such as RSA and PGP are supported.

3.2.1 RSA_AES

RSA_AES stands for AES symmetric encryption and RSA asymmetric encryption algorithm. RSA_AES is used in message encryption. Encrypt the message with AES symmetric encryption algorithm, and then encrypt the symmetric key with RSA asymmetric encryption algorithm.

Recommended AES key size: 256 bits.

Recommended RSA key size: 2048 bits.

3.2.2 PGP

PGP algorithm is used in file encryption.

PGP (Pretty Good Privacy) encryption uses a serial combination of hashing, data compression, symmetric-key cryptography, and finally public-key cryptography; each step uses one of several supported algorithms.

4. Process description

The following sections provide security specifications on messages and files transmission.

4.1 Message transmission

To guarantee that data have not been altered in transmission, digital signature and encryption mechanism can be adopted. For all messages, the digital signature is mandatory. In addition, data encryption is required if sensitive information is enclosed in the message,

When both digital signature and data encryption are required, encrypt the message first before adding a digital signature. The following figure illustrates the data encryption and digital signature workflow:

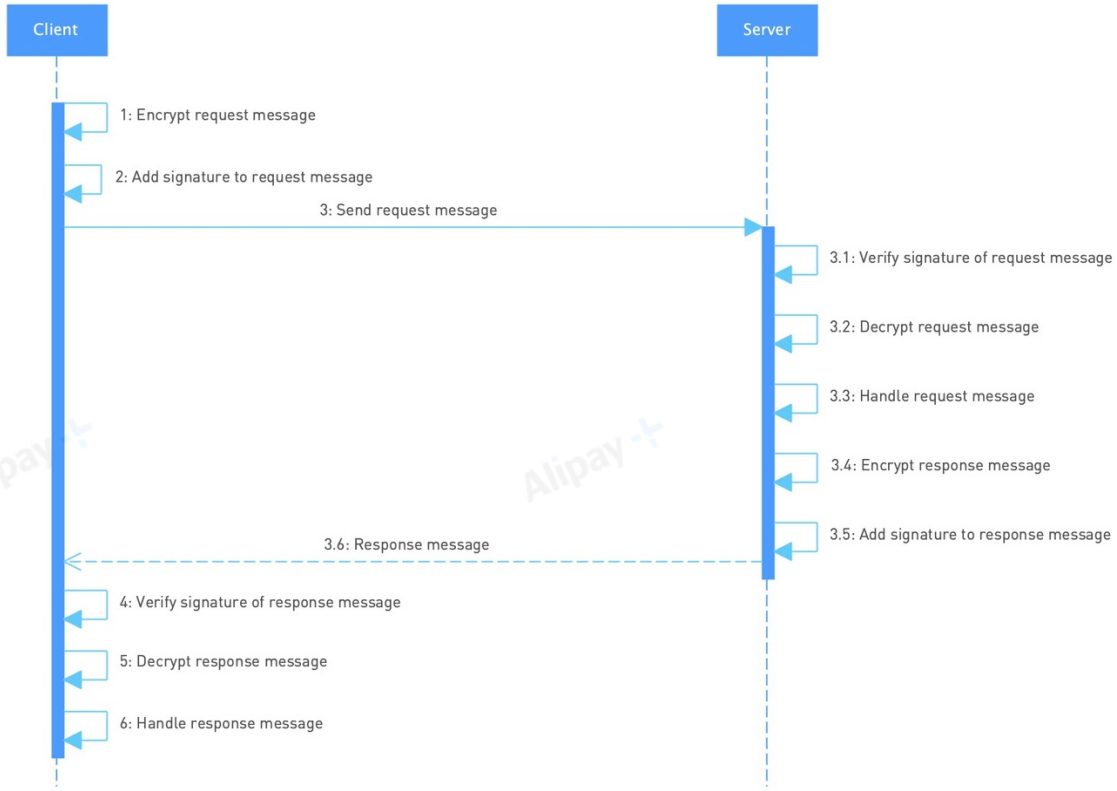


Figure 1. Message transmission workflow

4.1.1 Creating and validating a signature

The following sections provide information about creating and validating digital signatures based on asymmetric keys. A digital signature is created using the private key portion of an asymmetric key, and is validated using the public key portion of the same asymmetric key.

4.1.1.1 Creating a signature

The following figure illustrates the process to create a signature:

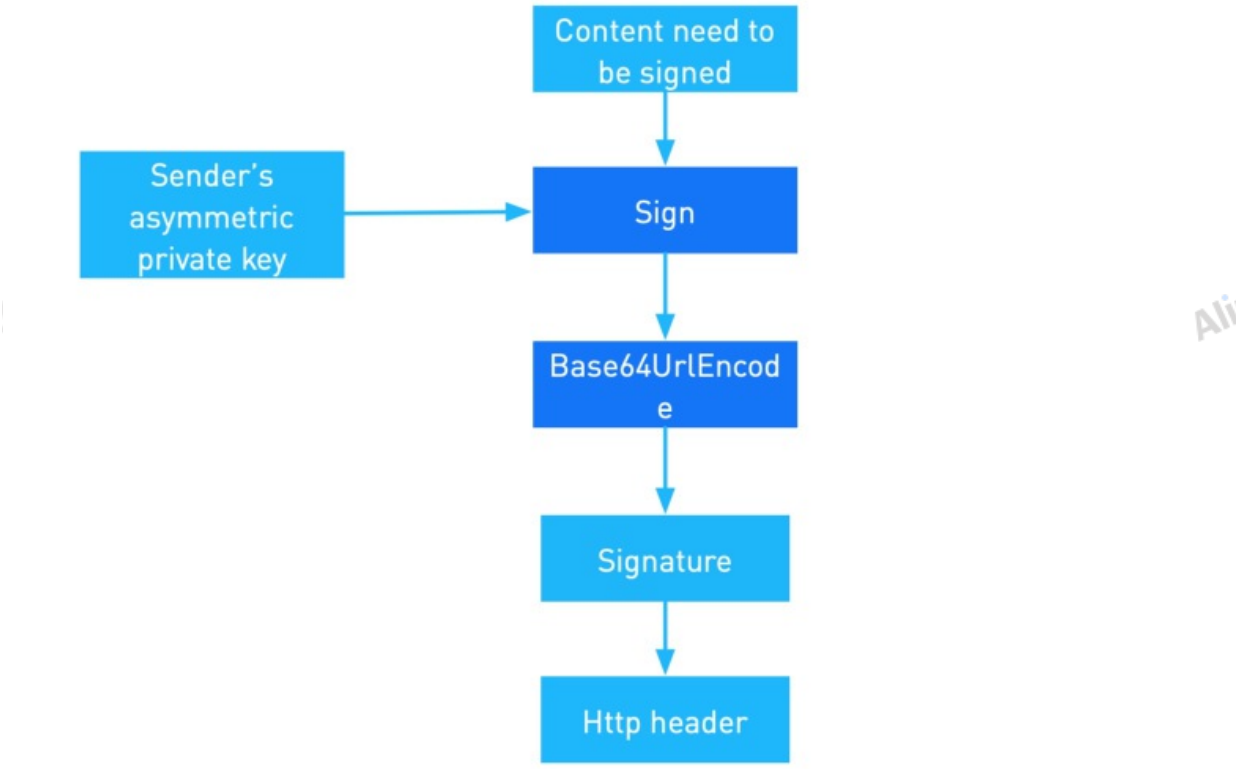


Figure 2. Signature creation process

Complete the following steps to create a signature:

1. Obtain the private key. The private key is associated with `Client-Id`. Different key versions might exist. You can use `keyVersion` of `Signature` to specify the version number. If key version is not specified, the latest version is used by default.
2. Create the string to sign. The content to be signed is:

```
<HTTP Method> <HTTP-URI-with-query-string>
<Client-Id>.<Request-Time|Response-Time>.<HTTP body>
```

3. Generate the signature. Use the algorithm to calculate a digest, and then sign the digest with the private key obtained in step1.

The following example assumes that RSA256 algorithm is used to generate the signature:

```
signature=base64UrlEncode(sha256withrsa(string-to-sign))
```

4. Add the signature to header. Assemble the signature algorithm, the key version used for the signature, and the signature into `Signature` header.

The following example shows a finished `Signature` header:

```
key: Signature；
value：algorithm=<algorithm>，keyVersion=<key-version>，signature=<signature>
```

4.1.1.2 Validating a signature

See the following figure for an overview of the signature validation process:

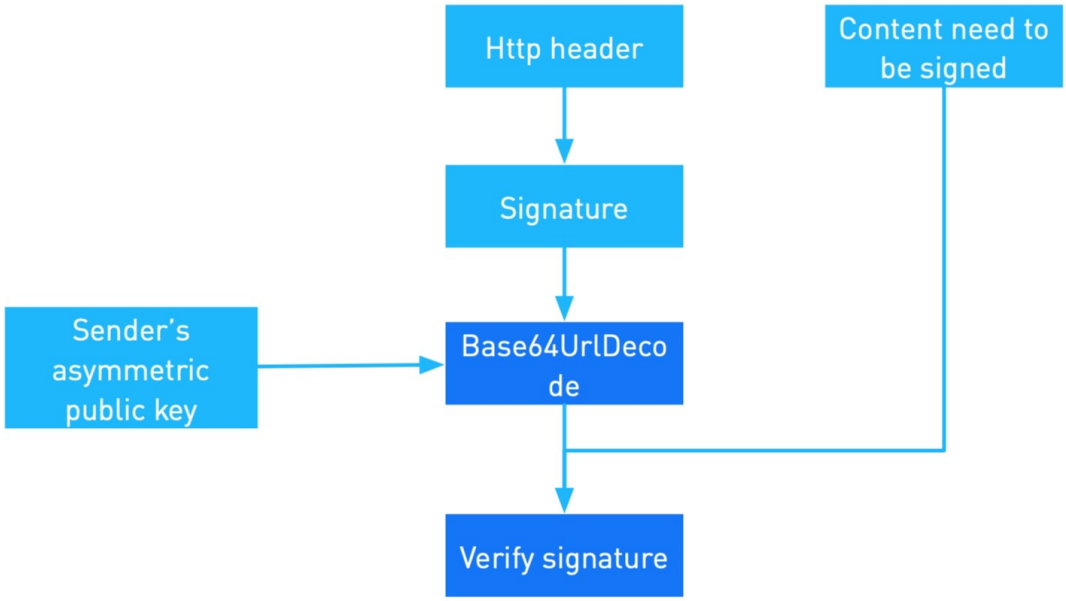


Figure 3. Signature validation process

The signature verification process goes as follows:

1、Retrieve the public key. Obtain `Client-Id`, `keyVersion`, and `algorithm` from header. If key version is not specified, the latest version is used by default. With `Client-Id` and `keyVersion`, you can then retrieve the public key for validating the signature.

2、Create the string to sign. See 4.1.1.1 step 2 for details.

3、Use the algorithm obtained in step 1 to calculate a digest of the string you created in step 2. Then, decrypt the signature by using the public key to get a digest. Compare two digests, if the digests match the signature is verified.

For example, assume RSA256 algorithm is used, base64url decode the signature content to obtain the original signature, and then validate the signature by using the sender's public key and sha256withrsa algorithm.

4.1.2 Encrypting and decrypting a message

When a message contains sensitive information, the entire message needs to be encrypted for transmission. For more information about sensitive information, see Appendix A Sensitive information.

4.1.2.1 Encrypting a message

The following figure illustrates the message encryption process:

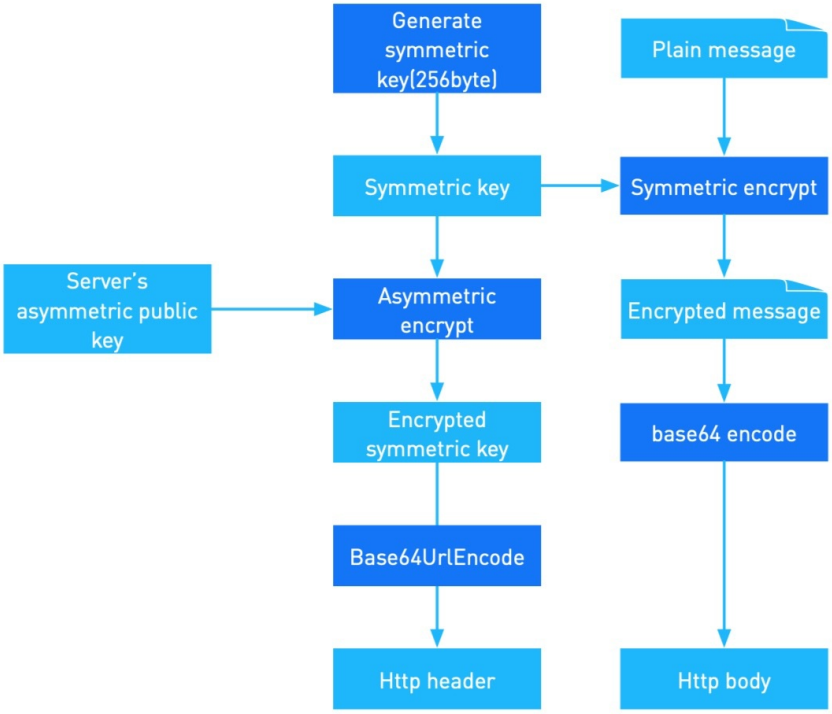


Figure 4. Message encryption process

Complete the following steps to encrypt a message:

1. Generate a one-time symmetric key. The symmetric key format is selected according to the symmetric algorithm. If AES algorithm is used, it is recommended to use a 256-bit length key. For a single request and response, the symmetric key can be reused.
2. Obtain the content to be encrypted. The entire HTTP body needs to be encrypted.
3. Encrypt the message. Use the one-time symmetric key that you obtained in step 1 to encrypt the message.
4. Obtain the public key of the message receiver. The public key is associated with `Client-Id`. Key rotation is supported, therefore, different key versions might exist. You can use `keyVersion` to specify the version number. If key version is not specified, the latest version is used by default.
5. Encrypt the symmetric key. Use the public key obtained in step 4 to encrypt the one-time symmetric key.
6. Add algorithm, key version, and encrypted symmetric key to Encrypt header, and add the encrypted message to http body.

The following example shows a finished encrypt header:

```
key: Encrypt
value：algorithm=<encryption-algorithm>，keyVersion=<key-version>,
symmetricKey=bqS8HSmdaRrpKSuPy7CqUlyd8lJurG93
```

4.1.2.2 Decrypting a message

The following figure illustrates the message decryption process:

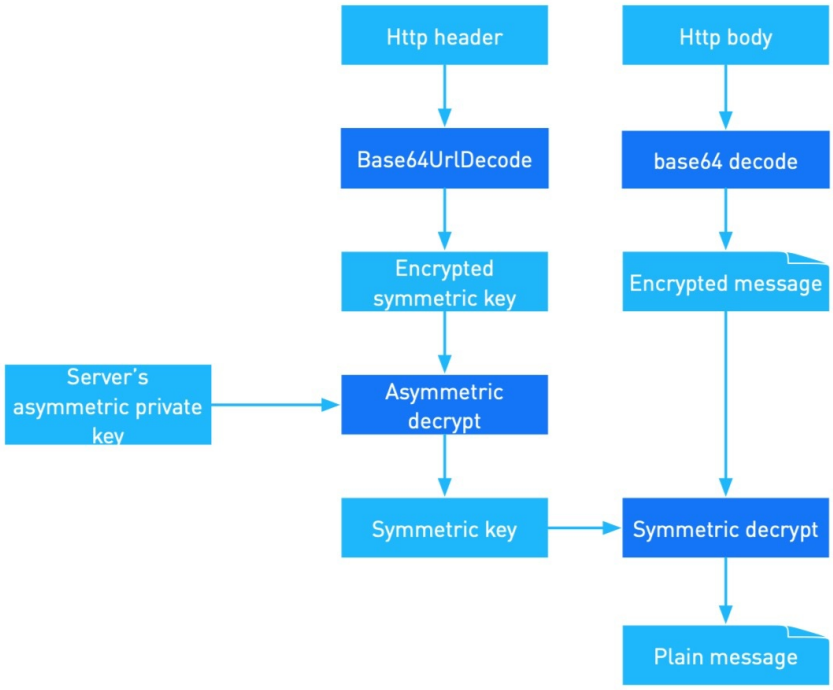


Figure 5. Message decryption flow

Complete the following steps to decrypt a message:

1. Retrieve the private key. Obtain `Client-Id`, `keyVersion`, and `algorithm` from header. With `Client-Id` and `keyVersion`, you can then retrieve the private key for decryption. If key version is not specified, the latest version is used by default.
2. Retrieve the symmetric key. Obtain the encrypted symmetric key from header, and then use the private key you retrieved in step1 to decrypt to get the symmetric key.
3. Obtain the ciphertext. The entire body part needs to be decrypted.
4. Decrypt the message. Use the symmetric key obtained in step2 to decrypt the ciphertext.

4.1.3 One-way encryption

A one-way encryption, or the hash function, is used when the transmitted information is sensitive and the receiver is not expected to learn the clear text information. The recommended hashing algorithm is sha256, and the encrypted byte array must be base64 encoded before transmission.

For example: In the blockchain remittance scenario, the user information cannot be directly stored on blockchain. Instead, hash the user information first by using the sha256 algorithm and then send it to Blockchain.

4.1.4 Transport layer security

To provide privacy and data integrity for communication over network, the following requirements must be met:

- Hypertext Transfer Protocol Secure (HTTPS) must be used for secure communication over network
- Transport Layer Security (TLS) v1.2 must be used
- Use the one-way authentication method

4.2 File transmission

Use PGP to provide cryptographic privacy and authentication for data communication. It is recommended to use SFTP mode to access files.

The following figure illustrates the file transmission flow:

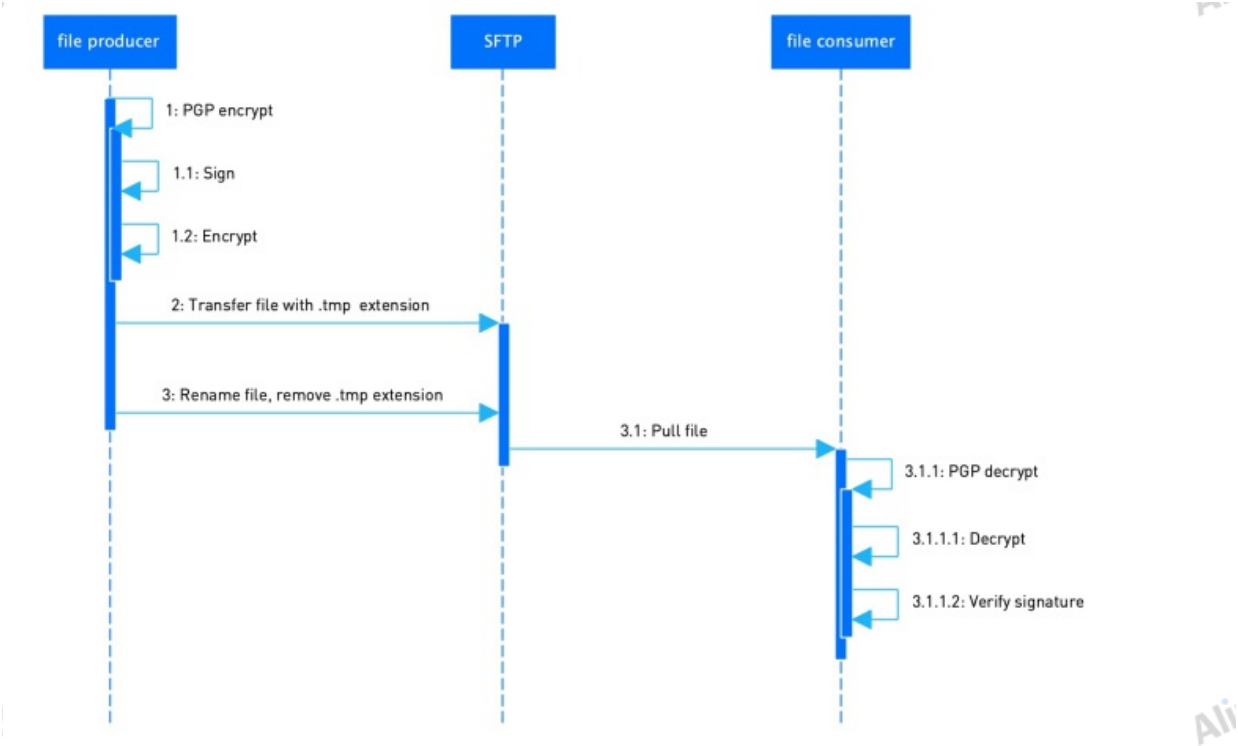


Figure 6. File transmission flow

For more information, see Technical specification for secure file transmission.

Appendix A Sensitive information

The sensitive information contains the following items:

Data Type – Level 1	Data Type – Level 2	Transmission
Sensitive personally identifiable information (PII)	Security code, access code, password or certificate	Encryption required
	Biometric records	Encryption required

Technical specification for secure file transmission

1. Scope

Technical specification for secure file transmission defines the security requirements for file exchange between Participant and Platform.

This specification is applicable to all participants.

2. Terms and definitions

The following terms and definitions apply:

PGP

Pretty Good Privacy, an encryption that uses a serial combination of hashing, data compression, symmetric-key cryptography, and finally public-key cryptography; each step uses one of several supported algorithms.

Platform

Platform refers to Connect or Alipay Acquiring Center (AAC).

3. File directory

If SFTP server of Platform is used, the directory structure can be customized according to specific requirements. For example, *Participant code* + *Date*.

If SFTP server of Platform is used, the directory structure is *Participant code* + *File type* + *Date*.

4. SFTP account management

If SFTP server of Platform is used, contact Technical support to obtain the credentials to login to the SFTP server.

If SFTP server of Participant or Merchant is used, Participant or Merchant must complete the SFTP server system configuration on Platform first.

5. File transfer process

File exchange happens between Platform and Participants.

For file sender, the file uploading process is as follows:

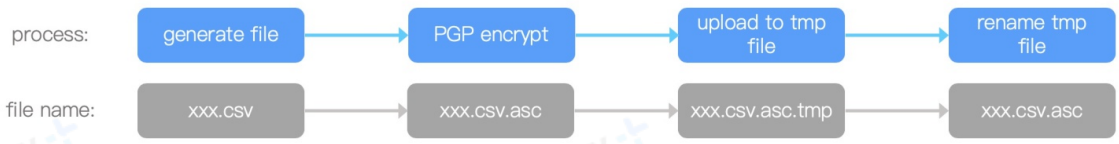


Figure 1. File uploading process

For file receiver, the file downloading process is as follows:

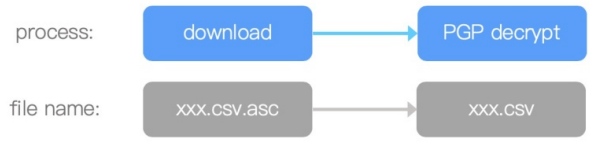


Figure 2. File downloading process

5.1 Pre-upload processing

Before the transmission, encrypt files by using PGP algorithm to enhance the transmission security. A .asc suffix is added to the encrypted file to differentiate from the unencrypted file.

5.2 Uploading files

To prevent temporary files from being wrongly downloaded, rename the file by appending a .tmp suffix to the file name before uploading. After the transmission is completed, rename the file back to the original name by removing the .tmp suffix.

5.3 Downloading files

Do not download the temporary file, which is ended with a .tmp suffix.

After the file is downloaded, decrypt the file by using PGP algorithm.

5.4 File retention

Files on SFTP server can be kept for 7 days. Files older than 7 days are automatically removed from the server.

6. Security controls

For more information about the security specification for file transmission, see Technical specification for secure data transmission for details.

Technical specification for keys and certificates

1. Scope

This document introduces the specifications and operational requirements of the keys and certificates used by Platform and Participants (merchants). The specifications and operational requirements in this document apply to the messages keys, SFTP SSH keys, TLS/SSL server certificates and client certificates, and message certificates.

This document is intended for to all related Participant personnel.

2. Reference

The following document, in whole or in part, is referenced in this document and is indispensable for its application. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

ITU-T Rec. X.509 2008, *Information technology – Open systems interconnection – The Directory: Public-key and attribute certificate frameworks*

3. Terms, definitions, and, abbreviations

This chapter introduces the terms, definitions, and abbreviations used in this specification.

3.1 Terms and definitions

The following terms and definitions are used in this specification.

Table 3-1 Terms and definitions

Term	Definition
Binding	The operation of establishing relationship between items of information that is provided by cryptographic means.
Certificate Authority	An authority trusted by one or more entities to create and sign public-key certificates. Optionally, the certification authority can create the subjects' keys.
Certificate Signing Request	In public key infrastructure (PKI) systems, a certificate signing request (or certification request) is a message or file following PKCS#10 sent from an applicant to a certificate authority in order to apply for a public-key certificate.
Certificate Update	The act or process by which data items bound in an existing certificate, especially authorizations granted to the subject, are changed by issuing a new certificate.
Distinguished Name	A Distinguished Name (DN) is used in a certificate to identify a certificate owner, or a certificate issuer (certificate authority).
Key Rotation	Changing the key, or, replacing the key by a new key. The places that use keys derived from the old key (such as authorized keys derived from an identity key, legitimate copies of the identity key, or certificates granted for a key) typically need to be correspondingly updated. With SSH user keys, key rotation means replacing an identity key by a newly generated key and updating authorized keys correspondingly.
Public-key Certificate	The public key of an entity, together with some other information, rendered unforgeable by digital signature with the private key of the certification authority (CA) that issued it.
RSA digital Signature Mechanism	A digital signature mechanism defined in PKCS#1.
SM2 Digital Signature Mechanism	A digital signature mechanism defined in ISO/IEC14888-3:2016/AMD1.
SM3 Hash Algorithm	A hash algorithm defined in ISO/IEC10118-3:2018.
TLS/SSL Client Certificate	A certificate that is used to verify the authentication of an End-Entity to a server when a connection is being established through a Secure Socket Layer/Transport Layer Security (SSL/TLS) session (secure channel).
TLS/SSL Server Certificate	A certificate that is used to verify the authentication of a web or application server to the client when a connection is being established through a Secure Socket Layer/Transport Layer Security (SSL/TLS) session (secure channel).
Unbinding	The operation of removing the established relationship between items of information that is provided by cryptographic means.

3.2 Abbreviations

The following table contains acronyms and abbreviations that apply to this documentation:

Table 3-2 Acronyms and abbreviations

Abbreviation	Meaning
AFCA	Ant Financial Certification Authority
CA	Certificate authority
CN	Common name
CSR	Certificate signing request
DN	Distinguished name
ITU-T Rec. X.509	ITU-T Recommendation X.509
PEM	Privacy enhanced mail
RSA	Rivest, Shamir and Adleman (public key algorithm)
SAN	Subject alternative name
SHA-256	Secure Hash Algorithm 256-bit
SSL	Secure socket layer
TLS	Transport layer security

4. Keys and certificates

Participant can select either certificates or key pairs at the application layer. However, certificates are preferred.

The following table contains the keys and certificates used during the interaction between Platform and Participant system:

Table 4-1 Certificates and key pairs

Layer	Type	Name	Usage
Transport layer (http)	TLS/SSL server certificate	Platform TLS/SSL server certificate	Identity verification for Platform in HTTPS communication
		Participant TLS/SSL server certificate	Identity verification for Participant in HTTPS communication
	TLS/SSL client certificate	Platform TLS/SSL client certificate	Platform identity verification. The certificate is used when Platform calls Participant URI and mutual HTTPS authentication is required.
		Participant TLS/SSL client certificate	Identity verification for Participant when calling Platform URI where mutual HTTPS authentication is required.
Application layer	Message certificate	Platform message certificate	Participant uses the certificate to verify messages from Platform. Participant also uses the certificate to encrypt messages when encryption is required.
		Participant message certificate	Platform uses the certificate to verify messages from Participant. Platform also uses the certificate to encrypt messages when encryption is required.
	Message key pair	Platform message key pair	Message verification. Participant uses the public key to verify Platform messages signed with the private key. Participant also uses the public key to encrypt messages.
		Participant message key pair	Message verification. Platform uses the public key to verify Participant messages signed with the private key. Platform also uses the public key to encrypt messages.
Transport layer (ftp)	Key pair	SFTP SSH key pair	Identity verification. SFTP servers use the key pair to verify the identity of visitors.

5. Certificate specification

This chapter introduces the specifications of TLS/SSL certificates and message certificates.

5.1 TLS/SSL server certificate

All API requests and other URIs from Platform and Participant must use Hyper Text Transfer Protocol Secure (HTTPS). This section defines the specifications for HTTPS server certificate and applies to both Platform and Participant certificates.

5.1.1 Certificate format

TLS/SSL server certificates must conform to X.509 v3 format.

5.1.2 Certificate subject

Subject distinguished names (DNs) contain the following attributes:

- Common Name (CN)

- Organizational unit (OU)
- Organization (O)
- State or Province (S)
- Country (C)

dNSName in subjectAltName extension or Subject CN must contain URI's domain name. The values of O, S, and C must be consistent with those registered in Platform.

5.1.3 Key algorithm and size
Use RSA algorithm. The keys must be 2048-bit in length.

5.1.4 Signature algorithm
Use SHA-256 as the hash algorithm.

5.1.5 Key usage
Keys contained in certificates can be used for digital signature and key encipherment.

5.1.6 Validity period
The certificate validity period must not exceed three years. The recommended validity period is one year.

5.1.7 Certificate issuer
TLS/SSL server certificates must be issued by CAs trusted by Platform. Visit Platform Portal for more information.

5.2 TLS/SSL client certificate
This section introduces the specifications of TLS/SSL client certificates.

5.2.1 Certificate format
TLS/SSL client certificates must conform to X.509 v3 format.

5.2.2 Certificate subject
Subject distinguished names (DNs) contain the following attributes:

- Common Name (CN)
- Organizational unit (OU)
- Organization (O)
- State or Province (S)
- Country (C)

CN must be the organization name or organization domain name. OU must be the identifier in Platform. O must be the organization name. The values of attributes must be consistent with those registered in Platform.

5.2.3 Key algorithm and size
Use RSA algorithm. The keys must be 2048-bit in length.

5.2.4 Signature algorithm
Use SHA-256 as the hash algorithm.

5.2.5 Key usage
Keys contained in certificates can be used for digital signature and key encipherment.

5.2.6 Validity period
The certificate validity period must not exceed three years. The recommended validity period is one year.

5.2.7 Certificate issuer
TLS/SSL client certificates must be issued by CAs trusted by Platform. Visit Platform Portal for more information.

5.3 Message certificate
This section introduces the specifications of message certificates.

5.3.1 Certificate format
Message certificates must conform to X.509 v3 format.

5.3.2 Certificate subject

Subject distinguished names (DNs) contain the following attributes:

- Common Name (CN)
- Organizational unit (OU)
- Organization (O)
- State or Province (ST/S)
- Country (C)

CN must be the organization name or organization domain name. OU must be the identifier in Platform. O must be the organization name. S must be the state or province where the organization Participant belongs to is registered. C must be the country where the organization Participant belongs to is registered. The values of attributes must be consistent with those registered in Platform.

5.3.3 Key algorithm and size

The following options can be used:

- RSA algorithm. The keys must be 2048-bit in length.
- SM2 algorithm. Use the recommended parameters published by Chinese Commercial Cryptography Administration Office.
- ECC algorithm. Use secp256k1 curve or X9.62 prime256v1 curve.

5.3.4 Signature algorithm

When RSA keys are used, SHA-256, SHA-384, SHA-512, and SHA3 can be used as the hash algorithm.

When SM2 keys are used, use SM3 as the hash algorithm.

When ECC keys are used, SHA-256, SHA-384, SHA-512, and SHA3 can be used as the hash algorithm.

5.3.5 Key usage

Keys contained in certificates can be used for digital signature.

5.3.6 Certificate issuer

Message certificates can be issued by the following CAs:

- Ant Financial Certificate Authority
- CAs trusted by Platform

5.3.7 Validity period

The certificate validity period must not exceed three years. The recommended validity period is two years.

6. TLS/SSL certificate operational requirements

This chapter introduces the operational requirements of TLS/SSL certificates.

6.1 TLS/SSL server certificate

This section introduces the operational requirements of TLS/SSL server certificates.

6.1.1 Applying certificates

Participant and Platform apply for certificates from CAs trusted by Platform.

6.1.2 Updating certificates

Platform and Participant must ensure that their TLS/SSL server certificates are within the validity period.

Platform and Participant maintain the configuration of their TLS/SSL server certificates independently in their gateway servers.

Platform and Participant must exchange and verify the certificates during the TLS/SSL handshake process according to the TLS/SSL specifications.

6.1.3 Verifying certificates

A certificate must conform to all the following specifications. Check from step 1 to step 6:

1. The certificate issuer conforms to the specifications in section 5.1.6.
2. The certificate signature algorithm conforms to the specifications in section 5.1.4, and the signature value is correct.
3. The certificate subject conforms to the specifications in section 5.1.2.
4. The certificate is in validity period and conforms to the specifications in section 5.1.7.
5. The key usages conform to the specifications in section 5.1.5.
6. The certificate is not revoked.

6.2 TLS/SSL client certificate

This section introduces the operational requirements of TLS/SSL client certificates.

6.2.1 Applying certificates

Eligible Participant can log in to Platform Portal to check the following information:

- Whether or not a TLS/SSL client certificate is required.
- The DN of the required TLS/SSL client certificate.

The following figure illustrates the how Participant applies for TLS/SSL client certificates:

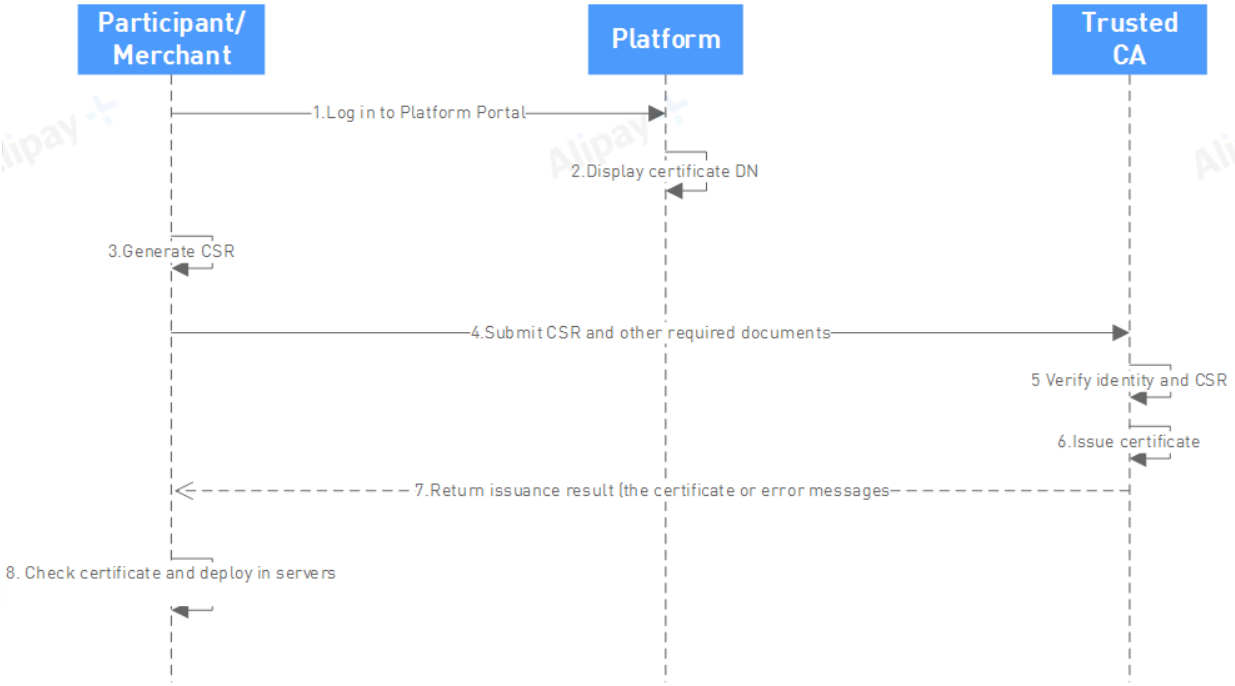


Figure 6-1 TLS/SSL client certificate application

Complete the following steps to apply for TLS/SSL client certificates:

1. Participant logs in to Platform Portal.
2. Platform displays the TLS/SSL client certificate DN.
3. Participant generates the Certificate Signing Request (CSR) according the DN.
4. Participant submits the CSR and other required documents to the CA.
5. The CA verifies Participant identity and the CSR, and continues if the verification is passed. Otherwise the process is ended.
6. The CA issues the certificate.
7. The CA returns the certificate issuance result to Participant.
8. Participant reviews and deploys the certificate in the related servers.

6.2.2 Updating certificates

Platform and Participant must ensure that their TLS/SSL client certificates are within the validity period.

Platform and Participant maintain the configuration of their TLS/SSL client certificates independently in the related servers.

Platform does not notify Participant of the TLS/SSL client certificate update, because that the certificates are exchanged during the TLS/SSL handshake process.

6.2.3 Verifying certificates

A certificate must conform to all the following specifications. Check from step 1 to step 6:

1. The certificate issuer conforms to the specifications in section 5.2.6.
2. The certificate signature algorithm conforms to the specifications in section 5.2.4, and the signature value is correct.
3. The certificate subject conforms to the specification in section 5.2.2.
4. The certificate is in validity period and conforms to the specifications in section 5.2.7.
5. The key usages conform to the specifications in section 5.2.5.
6. The certificate is not revoked.

7. Message certificate operational requirements

This chapter introduces the operational requirements of message certificates.

7.1 Participant message certificate

This section introduces the operational requirements of Participant message certificates.

7.1.1 Applying certificates

Participant can apply for message certificates from Ant Financial Certification Authority (AFCA) or other CAs trusted by Platform.

The following figure illustrates how Participant applies for message certificate from AFCA:

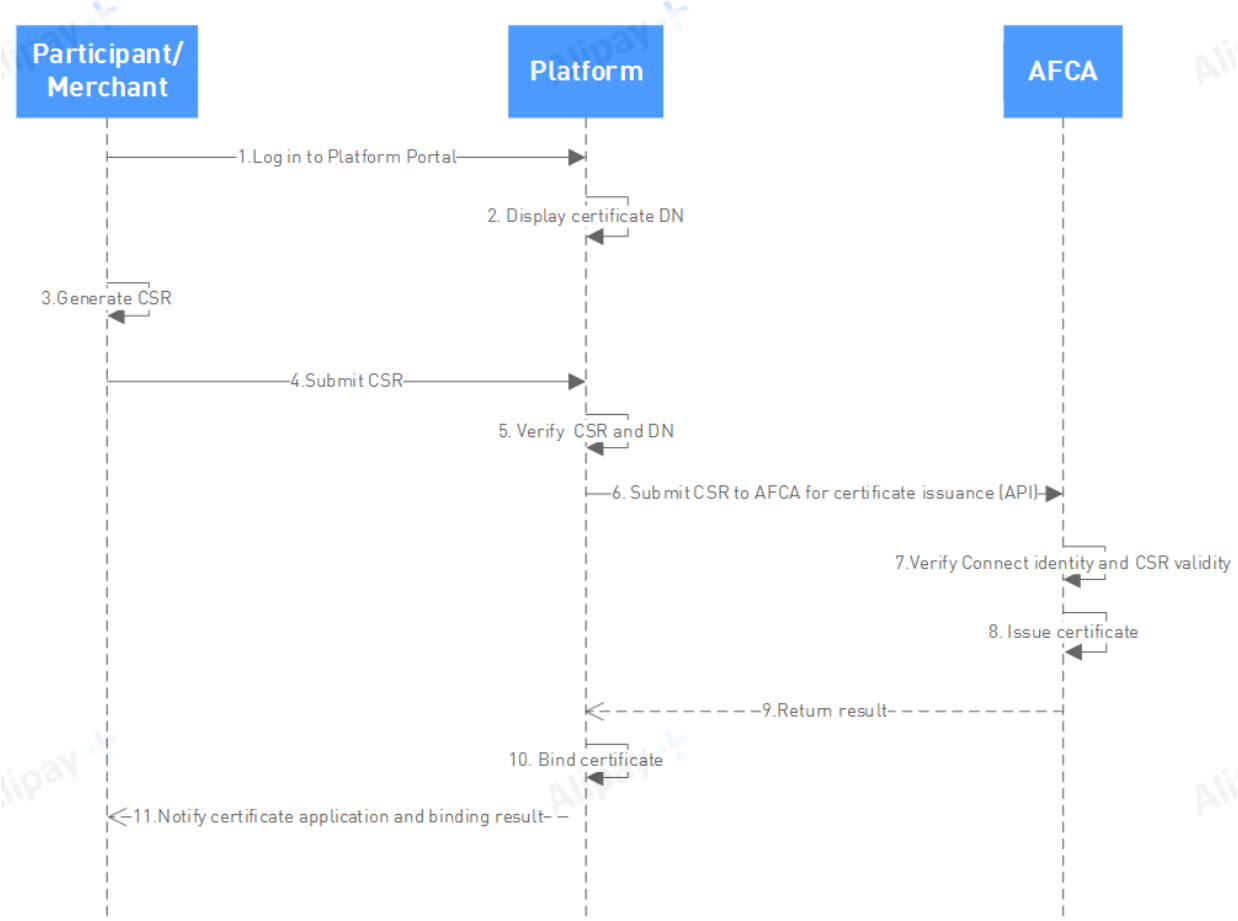


Figure 7-1 Participant message certificate application and issuance (AFCA)

Complete the following steps to apply for message certificates from AFCA:

1. Participant logs in to Platform Portal.
2. Platform Portal generates Participant message certificate DN according to the specifications in section 5.3.2.
3. Participant generates the Certificate Signing Request (CSR) according the DN and the specifications in section 5.3.

4. Participant submits the CSR on Platform Portal.
5. Platform verifies the consistency of the DN in the CSR with the DN provided in step 2, and only continues if the DNs are consistent.
6. Platform submits the CSR to AFCA for certificate issuance.
7. AFCA verifies Platform identity and CSR validity, and continues if the verification is passed. Otherwise the process is ended.
8. AFCA issues the certificate.
9. AFCA returns the certificate issuance result to Platform.
10. Platform binds the certificate to Participant.
11. Platform notifies the certificate application and binding result to Participant.

After the application process is completed, Participant can view and download the certificate at Platform.

The following figure illustrates how Participant applies for message certificate from CAs trusted by Platform:

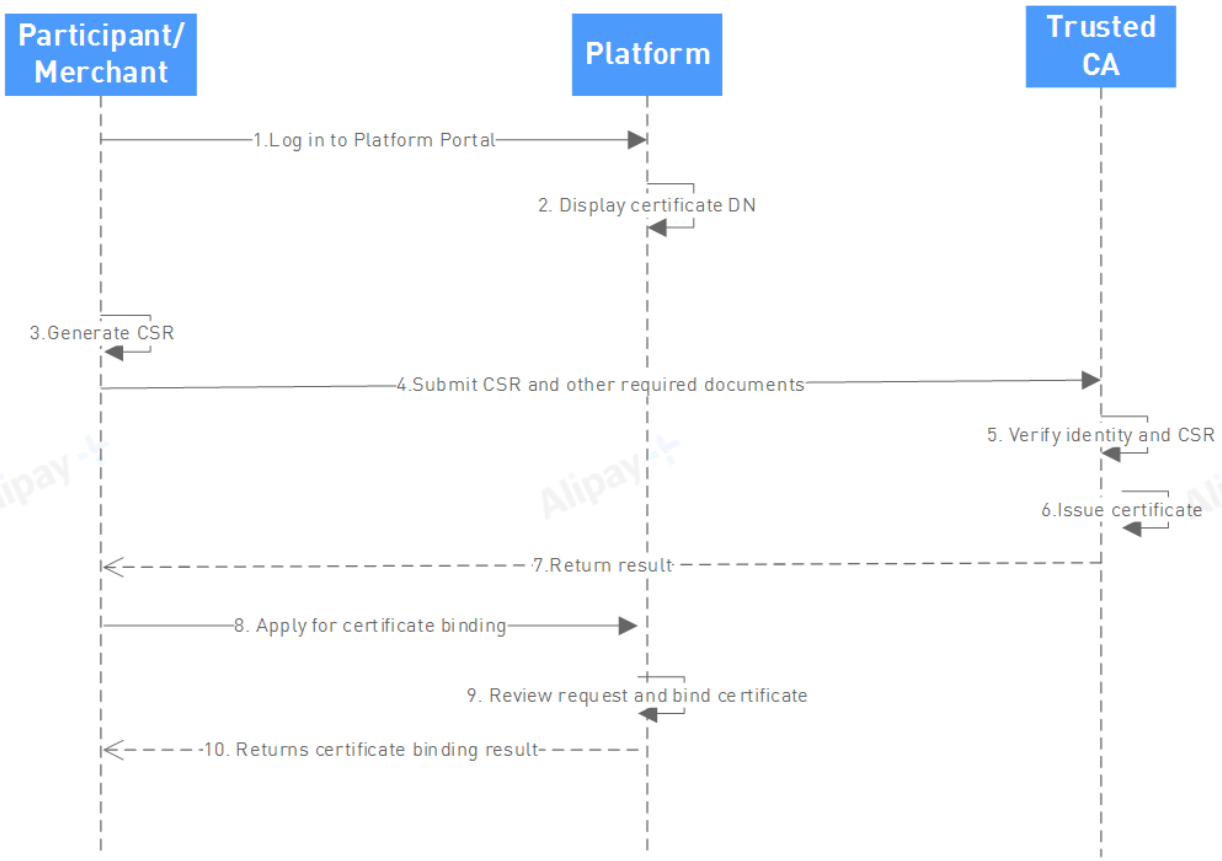


Figure 7-2 Participant message certificate application and issuance (Trusted CAs)

Complete the following steps to apply for message certificates from trusted CAs:

1. Participant logs in to Platform Portal.
2. Platform displays the message certificate DN.
3. Participant generates the Certificate Signing Request (CSR) according the DN and the specifications in section 5.3.
4. Participant submits the CSR and other required documents to the CA. The documents required are subject to the CA.
5. The CA verifies Participant identity and the CSR.
6. The CA issues the certificate.
7. The CA notifies Participant of the issuance result.
8. Participant submits the certificate binding request to Platform.
9. Platform verifies the request and the certificate, and binds the certificate to Participant if the verification is passed.
10. Platform notifies Participant of the certificate binding result.

7.1.2 Binding certificates

Participant can bind to a certificate by using Platform API or Platform Portal. See section 7.1.2.1 and 7.1.2.2 for details.

7.1.2.1 Binding certificates by using API

Perform the following steps to bind a Participant message certificate by using API:

1. Participant sends a request to the Platform certificate binding API. See section 10.1.2 for details.
2. Platform processes the request and returns the result.

Note: If no result is received or the binding is not successful, Participant must retry. If the problem persists, contact Platform.

7.1.2.2 Binding certificates by using Platform Portal

Perform the following steps to bind a Participant message certificate by using Platform Portal:

1. Participant logs in to Platform Portal.
2. Participant submits the certificate.
3. Platform verifies the certificate according to the specifications described in section 7.3.
4. If the verification is passed, Platform completes the binding and returns the binding success result. Otherwise, Platform returns the binding failure result.
5. Platform returns the result to Participant.

7.1.3 Unbinding certificates

When Participant is to stop using a message certificate that is still in the validity period, Participant must inform Platform. Certificate unbinding can be completed by using Platform API or Platform Portal. See section 7.1.3.1 and 7.1.3.2 for details.

7.1.3.1 Unbinding certificates by using API

Perform the following steps to unbind Participant message certificate by using API:

1. Participant sends a request to the Platform certificate unbinding API. See section 10.1.3 for details.
2. Platform processes the request and returns the result.
3. Participant confirms the result.

Note: If no result is received or the unbinding is not successful, Participant must retry. If the problem persists, contact Platform.

7.1.3.2 Unbinding certificates by using Platform Portal

Perform the following steps to unbind a Participant message certificate by using Platform Portal:

1. Participant logs in to Platform Portal.
2. Participant requests a certificate unbinding.
3. Platform processes the request and returns the unbinding result.

7.1.4 Updating certificates

Certificate update must be completed before the old certificate expires. To update a Participant message certificate, complete the following steps:

1. Participant binds to the new certificate. See section 7.1.2 for details.
2. Participant deploys the new certificate in Participant system. The old certificate remains effective before certificate unbinding.
3. Participant requests that Platform unbinds the old certificate. See section 7.1.3 for details.

7.2 Platform message certificate

This section introduces the operational requirements of Platform message certificates.

7.2.1 Applying certificates

Platform message certificates are issued by AFCA. Platform message certificates are generated after Participant completes the steps in section 7.1.1.

7.2.2 Binding certificates

When a new Platform message certificate is generated, Platform must inform Participant. Complete Platform message certificate binding by using one of the following methods:

- If Participant provides certificate binding API to Platform, perform the binding according to section 7.2.2.1.
- If no API is provided, perform the binding according to section 7.2.2.2.

7.2.2.1 Binding certificates by using API

Perform the following steps to bind a Platform message certificate by using API:

1. Platform sends a request to certificate binding API provided by Participant. See section 10.2.2 for details.
2. Participant processes the request and returns the result to Platform.
3. Platform confirms the result.

Note: If no result is returned or the binding is not successful, Platform must retry. If the problem persists, contact Participant.

7.2.2.2 Binding certificates by using Platform Portal

Perform the following steps to bind a Platform message certificate by using Platform Portal:

1. Platform sends a request to Participant by using the Platform Portal and email.
2. Participant downloads the new Platform message certificate on the portal.
3. Participant deploys the new Platform message certificate in Participant system and binds the certificate to Platform.
4. Participant logs in to Platform Portal to check and ensure that the binding is successful.

Note: Participant must notify Platform if the binding is not successful.

7.2.3 Unbinding certificates

When Platform is to stop using a message certificate that is still in the validity period, Platform must inform Participant. Complete Platform message certificate unbinding by using one of the following two methods:

- If Participant provides certificate unbinding API to Platform, perform the unbinding according to section 7.2.3.1.
- If no API is provided, perform the unbinding according to section 7.2.3.2.

7.2.3.1 Unbinding certificates by using API

Perform the following steps to unbind a Platform message certificate by using API:

1. Platform sends a request to Participant certificate unbinding API. See 10.2.3 for details.
2. Participant processes the request and returns the result to Platform.
3. Platform confirms the result.

Note: If no result is received or the unbinding is not successful, Platform must retry. If the problem persists, contact Participant.

7.2.3.2 Unbinding certificates by using Platform Portal

Perform the following steps to unbind a Platform message certificate by using Platform Portal:

1. Platform notifies Participant by using Platform Portal and email.
2. Participant completes the certificate unbinding in its system.

7.2.4 Updating certificates

By default, Platform starts the update process of a message certificate 90 days before the certificate expiration date. If the certificate is renewed before that period, Platform will contact Participant in advance.

To update a Platform message certificate, complete the following steps:

1. Platform binds to a new message certificate. See section 7.2.2 for details.
2. Platform deploys the new certificate. The old certificate remains effective before certificate unbinding.
3. Platform requests that Participant unbinds the old message certificate. See section 7.2.3 for details.

7.3 Verifying message certificates

A certificate must conform to all the following specifications. Check from step 1 to step 6:

1. The certificate issuer conforms to the specifications in section 5.3.6.
2. The certificate signature algorithm conforms to the specifications in section 5.3.4, and the signature value is valid.

3. The certificate subject conforms to the specifications in section 5.3.2.
4. The certificate is in validity period and conforms to the specifications in section 5.3.7.
5. The key usages conform to the specifications in section 5.3.5.
6. The certificate is not revoked.

8. Message key pair operational requirements

This chapter introduces the operational requirements of message key pairs.

8.1 Participant message key pair

This section introduces the operational requirements of Participant message key pairs.

8.1.1 Generating keys

Participant independently generates message key pairs. The following algorithm and key length options can be used:

- RSA algorithm. The keys must be 2048-bit in length.
- SM2 algorithm. Use the recommended parameters published by Chinese Commercial Cryptography Administration Office.
- ECC algorithm. Use secp256k1 curve or X9.62 prime256v1 curve

8.1.2 Binding keys

If no key has been bound before, complete key binding according to section 8.1.2.2. Otherwise, complete key binding according to section 8.1.2.1 or section 8.1.2.2.

8.1.2.1 Binding keys by using API

Perform the following steps to bind a Participant message public key by using API:

1. Participant sends a request to the Platform public key binding API. See section 10.1.4 for details.
2. Platform processes the request and returns the result.
3. Participant confirms the result.

Note: If no result is received or the binding is not successful, Participant must retry. If the problem persists, contact Platform.

8.1.2.2 Binding keys by using Platform Portal

Perform the following steps to bind a Participant message public key by using Platform Portal:

1. Participant logs in to Platform Portal.
2. Participant uploads the new public key and designates the key validity period.
3. Platform verifies the public key according to section 8.3.
4. If the verification is passed, Platform binds the key to Participant. Otherwise, no key is bound.
5. Platform Portal notifies Participant of the result.

8.1.3 Unbinding keys

When Participant is to stop using a public key that is still in the validity period, Participant must inform Platform. Complete key unbinding according to section 8.1.3.1 or section 8.1.3.2.

8.1.3.1 Unbinding keys by using API

Perform the following steps to unbind a Participant message public key using API:

1. Participant sends a request to the Platform public key unbinding API. See section 10.1.5 for details.
2. Platform processes the request and returns the result.
3. Participant confirms the result.

Note: If no result is received or the unbinding is not successful, Participant must retry. If the problem persists, contact Platform.

8.1.3.2 Unbinding keys by using Platform Portal

Perform the following steps to unbind a Participant message public key by using Platform Portal:

1. Participant logs in to Platform Portal.

2. Participant requests Platform to unbind a public key.
3. Platform process the request.
4. Platform notifies Participant of the result.

8.1.4 Rotating keys

The key rotation must be completed before old key pair expiration. To rotate a key, complete the following steps:

1. Participant binds to a new public key. See section 8.1.2 for details.
2. Participant starts using the new key pair.
3. Participant requests Platform to unbind the old public key. See section 8.1.3 for details.

Note: The old key pair remains effective before unbinding.

8.2 Platform message key pair

This section introduces the operational requirements of Platform message key pairs.

8.2.1 Generating keys

After Participant completes the steps in section 8.1.2, Platform message key pair is generated automatically, with a validity period of two years.

8.2.2 Binding keys

If no key has been bound before, or no Participant public key binding API is provided, complete key binding according to section 9.2.2.2. Otherwise, complete key binding according to section 8.2.2.1.

8.2.2.1 Binding keys by using API

Perform the following steps to bind a Platform message public key by using API:

1. Platform sends a request to Participant public key binding API. See section 10.2.4 for details.
2. Participant processes the request and returns the result.
3. Platform confirms the result.

Note: If no result is received or the binding is not successful, Platform must retry. If the problem persists, contact Participant.

8.2.2.2 Binding keys by using Platform Portal

Perform the following steps to bind a Platform message public key by using Platform Portal:

1. Participant logs in to Platform Portal and downloads the new public key.
2. Participant binds the key to Platform.
3. Participant confirms on Platform Portal that the binding is completed.

8.2.3 Unbinding keys

When Platform is to stop using a key pair that is still in the validity period, Platform must inform Participant. If Participant public key unbinding API is provided, complete key unbinding according to section 8.2.3.1. Otherwise, complete key unbinding according to section 8.2.3.2.

8.2.3.1 Unbinding keys by using API

Perform the following steps to unbind a Platform message public key by using API:

1. Platform sends a request to Participant public key unbinding API. See section 10.2.5 for details.
2. Participant processes the request and returns the result.
3. Platform confirms the result.

Note: If no result is received or the unbinding is not successful, Participant must retry. If the problem persists, contact Platform.

8.2.3.2 Unbinding keys by using Platform Portal

Perform the following steps to unbind a Platform message public key by using Platform Portal:

1. Participant logs in to Platform Portal.
2. Participant requests Platform to unbind a public key.

3. Platform process the request and returns the result.
4. If the unbinding is successfully, Participant then unbinds the public key in Participant system.

8.2.4 Rotating keys

The key rotation must be completed before old key pair expiration. By default, Platform starts the rotation process of a message key pair 90 days before the key pair expiration date. If the key pair is renewed before that period, Platform will contact Participant in advance.

To rotate a Platform message key pair, complete the following steps:

1. Platform binds to a new public key. See section 8.2.2 for details.
2. Platform starts using the new key pair.
3. Platform requests Participant to unbind the old public key. See section 8.2.3 for details.

Note: The old key pair remains effective before unbinding.

8.3 Verifying public keys

A public key must conform to all the following specifications. Check from step 1 to step 3:

1. The public key is in X.509 format and PEM encoded.
2. If RSA keys are used, the modulus must be 2048-bit.
3. If SM2 keys are used, the keys must be in the correct group specified by the associated domain parameters.

9. SFTP SSH key pair operational requirements

This section introduces the operational requirements of SFTP SSH key pairs used by Platform and Participant.

9.1 Generating keys

SSH keys are RSA-2048 keys generated by the visitors. Email addresses registered in Platform Portal by Participant must be included in the key file.

9.2 Delivering keys

If the SFTP server is provided by Platform, Participant must upload the SFTP SSH public key to Platform Portal.

If the SFTP server is provided by Participant, Participant must download the Platform SFTP SSH public key in Platform Portal.

9.3 Rotating keys

SFTP SSH keys must be regularly changed. The frequency for rotating Platform SFTP SSH keys are once per two year. For Participant SFTP SSH keys, the minimum frequency is once per three years.

9.3.1 Rotating Participant SFTP SSH keys

Perform the following steps to rotate Participant SFTP SSH keys:

1. Participant generates a new key pair.
2. Participant delivers the new SFTP SSH public key to Platform Portal according to section 9.2, and designates the time period for the key rotation process. Both the Platform server and Participant server must calibrate the system time.
3. Platform reviews the designated time period, and only continues if the designated time period is agreed.
4. Participant stops visiting the Platform SFTP service at the beginning time of the designated time period.
5. After the end time of the designated time period, Platform enables the new public key, and Participant can visit the Platform SFTP service with the new private key.

9.3.2 Rotating Platform SFTP SSH keys

Perform the following steps to rotate Platform SFTP SSH keys:

1. Platform generates a new key pair and notifies Participant.
2. Participant downloads the new SFTP SSH public key in Platform Portal, and designates the time period for the key rotation process. Both the Platform server and Participant server must calibrate the system time.
3. Platform reviews the designated time period, and only continues if the designated time period is agreed.
4. Platform stops visiting Participant SFTP service at the beginning time of the designated time period.

5. After the end time of the designated time period, Participant enables the new public key, and Platform can visit Participant SFTP service with the new private key.

10. APIs

Message certificate and key pairs can be managed by using APIs.

10.1 Platform APIs

Platform-provided APIs are described in the following sections.

10.1.1 Query

Participant can use this API to query Platform message certificates or public keys.

Request

The main request information includes Participant unique identifier, timestamp, and signature.

Response

The main response information includes the public key that is paired with the current-effective private key and public key version number, or certificate information and certificate version number.

The actual response information varies depending on the specific request.

10.1.2 Certificate binding

Participant can use this API to request binding to a certificate.

Request

The main request information includes Participant unique identifier, information of the new certificate, timestamp, and signature.

Signature

For the first binding, use the certificate private key to generate the signature (Signature 1). In other scenarios, generate Signature 1 with the new certificate private key, and Signature 2 with the old certificate private key. Platform verifies both signatures.

Response

The main response information includes the binding result (success or failure), timestamp, and signature. If the binding is successful, the certificate issuer, certificate serial number, and certificate version number are also returned.

Process

Platform verifies the new certificate according to the specifications in section 5.3. If old certificate exists, Platform also checks if the new certificate is different to the old one. If the verification is passed, the process continues. Otherwise, the process is ended and a response "Invalid certificate" is returned.

Platform then verifies Signature 1 with the new certificate and continues if the verification is passed. Otherwise, the process is ended and a response "Signature 1 is invalid" is returned.

If Signature 2 exists, Platform verifies Signature 2 with the old certificate, and continues if the verification is passed. If the verification fails, the process is ended and a response "Signature 2 is invalid" is returned.

Finally, Platform performs the binding.

10.1.3 Certificate unbinding

Participant can use this API to request unbinding a message certificate.

Prerequisite

Participant is bound to two effective message certificates.

Request

The main request information includes Participant unique identifier, certificate version number, timestamp, and signature.

Signature

Sign the request with the private key in the certificate that requires unbinding.

Response

The main response information includes the unbinding result (success or failure), certificate issuer, certificate serial number, certificate version number, timestamp, and signature.

Process

Platform verifies the certificate according to Participant unique identifier and certificate version number, and continues if the verification is passed. Otherwise, the process is ended and a response "Certificate information not found" is returned.

Platform then verifies the request signature with the certificate, and continues if the verification is passed. Otherwise, the process is ended and a response "Invalid signature" is returned.

Finally, Platform performs the unbinding.

10.1.4 Public key binding

Participant can use this API to request binding to a public key.

Prerequisite

Effective Participant public keys are stored in Platform.

Request

The main request information includes Participant unique identifier, information of the new public key, expiration date of the new public key, timestamp, and signature.

Signature

Use the new private key to generate Signature 1 and the old private key, Signature 2. Platform verifies both signatures.

Response

The main response information includes the binding result (success or failure), timestamp, and signature. If the binding is successful, public key information and version number are also returned.

Process

Platform verifies the new public key according to requirements specified in section 5.3.3, and checks if the new key is different to the old one. If the verification is passed, the process continues. Otherwise, the process is ended, and a response "Invalid public key" is returned.

Platform then verifies Signature 1 with the new public key and continues if the verification is passed. Otherwise, the process is ended, and a response "Signature 1 is invalid" is returned.

Platform then verifies Signature 2 with the old public key, and continues if the verification is passed. Otherwise, the process is ended, and a response "Signature 2 is invalid" is returned.

Finally, Platform performs the binding.

10.1.5 Public key unbinding

Participant can use this API to request unbinding a public key.

Prerequisite

Participant is bound to two effective public keys.

Request

The main request information includes Participant unique identifier, public key version number, time stamp, and signature.

Signature

Sign the request with the private key paired with the public key that requires unbinding.

Response

The main response information includes the unbinding result (success or failure), key version number, timestamp, and signature.

Process

Platform verifies the public key according to Participant unique identifier and public key version number, and continues if the verification is passed. Otherwise, the process is ended, and a response "Public key information not found" is returned.

Platform then verifies the request signature with the public key, and continues if the verification is passed. Otherwise, the process is ended and a response "Invalid signature" is returned.

Finally, Platform performs the unbinding.

10.2 Participant APIs

Participants (merchants) provided APIs are specified in this section. Public key and certificate management can be more efficient with Participant-provided APIs.

10.2.1 Query

Platform can use this API to query effective Participant message certificates or public keys.

Request

The main request information includes Platform unique identifier, timestamp, and signature.

Response

The main response information includes the public key that is paired with the effective private key and public key version number, or certificate information and certificate version number.

The actual response information varies depending on the specific request.

10.2.2 Certificate binding

Platform can use this API to request binding to a message certificate.

Request

The main request information includes Platform unique identifier, information of the new certificate, timestamp, and signature.

Signature

For the first binding, use the certificate private key to generate the signature (Signature 1). In other scenarios, generate Signature 1 with the new private key and Signature 2 with the old private key. Participant verifies both signatures.

Response

The main response information includes the binding result (success or failure), timestamp, and signature. If the binding

is successful, the certificate version number is also returned.

Process

Participant verifies the new certificate according to the specifications in section 5.3. If old certificate exists, Participant also checks if the new certificate is different to the old one. If the verification is passed, the process continues. Otherwise, the process is ended, and a response "Invalid certificate" is returned.

Participant then verifies Signature 1 with the new certificate and continues if the verification is passed. Otherwise, the process is ended, and a response "Signature 1 is invalid" is returned.

Participant hen verifies Signature 2 with the old certificate, and continues if the verification is passed. Otherwise, the process is ended and a response "Signature 2 is invalid" is returned.

Finally, Participant performs the binding.

10.2.3 Certificate unbinding

Platform can use this API to request unbinding a message certificate.

Prerequisite

Platform is bound to two effective message certificates, which must be verified by both Platform and Participant.

Request

The main request information includes Platform unique identifier, certificate version number, timestamp, and signature.

Signature

Sign the request with the private key in the certificate that requires unbinding.

Response

The main response information includes the unbinding result (success or failure), certificate version number, timestamp, and signature.

Process

Participant verifies the certificate according to Platform unique identifier and then certificate version number, and continues if the verification is passed. Otherwise, the process is ended and a response "Certificate information not found" is returned.

Participant then verifies the request signature with the certificate, and continues if the verification is passed. Otherwise, the process is ended, and a response "Invalid signature" is returned.

Finally, Participant performs the unbinding.

10.2.4 Public key binding

Platform can use this API to request binding to a public key.

Prerequisite

Platform has provided effective public key to Participant.

Request

The main request information includes Platform unique identifier, information of the new public key, expiration date of the new public key, timestamp, and signature.

Signature

Use the new private key to generate Signature 1 and the old private key, Signature 2. Participant verifies both signatures.

Response

The main response information includes the binding result (success or failure), timestamp, and signature. If the binding is successful, public key information and version number are also returned.

Process

Participant verifies the new public key according to requirements specified in section 5.3.3, and checks if the new key is different to the old one. If the verification is passed, the process continues. Otherwise, the process is ended, and a response "Invalid public key" is returned.

Participant then verifies Signature 1 with the new public key and continues if the verification is passed. Otherwise, the process is ended, and a response "Signature 1 is invalid" is returned.

Participant then verifies Signature 2 with the old public key, and continues if the verification is passed. Otherwise, the process is ended, and a response "Signature 2 is invalid" is returned.

Finally, Participant performs the binding.

10.2.5 Public key unbinding

Platform can use this API to request unbinding a public key.

Prerequisite

Platform has two effective public keys in use.

Request

The main request information includes Platform unique identifier, public key version number, timestamp, and signature.

Signature

Sign the request with the private key paired with the public key that requires unbinding.

Response

The main response information includes the unbinding result (success or failure), key version number, timestamp, and signature.

Process

Participant verifies the public key according to Platform unique identifier and public key version number, and continues if the verification is passed. Otherwise, the process is ended, and a response "Public key information not found" is returned.

Participant then verifies the request signature with the public key, and continues if the verification is passed. Otherwise, the process is ended and a response "Invalid signature" is returned.

Finally, Participant performs the unbinding.